CAST: 리전 용량 제약을 고려한 탄소 인지 시공간 워크로드 스케줄링

CAST: Carbon-Aware Spatio-Temporal Scheduling under Capacity Constraints

초록 CAST는 배치 작업을 여러 리전에 나눠 탄소를 줄이는 스케줄러로, 마감은 지키고 리전별 동시 실행 상한을 넘기면 최대 벌점을 준다. 매시간 대기 작업 전부의 리전과 시각을 선형 계획으로 다시 풀고 지금 시작할 작업만 확정한다. 8개 리전 1년 실험에서 기본 설정은 제출 리전 즉시 실행보다 배출을 합성 작업 71.9%, Azure VM 기록 51.3% 줄였고 대부분 리전 이동 덕이다(평균 배출계수·전송 제외; 한계 근사 18~39%). 위반은 Azure 이용률 53.5%까지 없었다. 같은 지연의 고정 가중치에서 위반 없는 단순 설계 둘보다 0.9~2.5%p 덜 배출했으나 한계 근사 하나에서 합성은 뒤집히고 Azure 한 짝은 사라졌으며, 기본 가중치로는 합성 배출 차가 유의하지 않으나(점추정은 반대) 대기가 25% 짧았다. 탄소만 보면 합성 상한 안 95.7%로, 한 리전에 평소 2.2·9.9배 용량이 드는 CASPER·Sukprasert 재구현(95.3%·96.6%)에 버금갔다.

주제어: 탄소 인지 스케줄링, 워크로드 이동, 리전 용량, 선형 계획법, 롤링 지평

Abstract We propose CAST, a batch scheduler that cuts carbon emissions under hard deadlines and soft per-region caps on concurrent jobs. Hourly, it re-plans the region and start time of all waiting jobs with linear programs and commits only jobs starting now. In a one-year, eight-region simulation, CAST's default configuration (knee weighting, LSTM forecasts) cut emissions by 71.9% on a synthetic workload (8.4% utilization) and 51.3% on an Azure VM trace (26.1%) versus immediate home-region execution, mostly by moving regions (average emission factors, transfer excluded; 21–39% and 18–29% under marginal approximations), and had no cap violations up to 53.5% Azure utilization. At latency-matched fixed weights it emitted 0.9–2.5 percentage points less than the two closest violation-free designs (one marginal approximation reverses the synthetic gaps and erases one Azure gap); under the default weighting the synthetic gap to the fix-on-arrival design was not significant (point estimate reversed) while waits were 25% shorter. Optimizing carbon alone on the synthetic workload, it saved 95.7% within the caps, on par with reimplemented CASPER (95.3%) and perfect-foresight Sukprasert et al. (96.6%), which need 2.2 and 9.9 times the home-region peak in one region.

Keywords: Carbon-aware scheduling, Workload shifting, Regional capacity, Linear programming, Rolling horizon

1. 서론

본 논문은 마감을 지키면서 리전(클라우드 사업자가 데이터센터를 묶어 운영하는 지역 단위)의 용량을 되도록 넘지 않고 탄소 배출을 줄이는 스케줄러 CAST(Carbon-Aware Spatio-Temporal scheduler)를 제안한다. 전 세계 데이터센터 전력 소비는 2024년 약 415TWh에서 2030년 945TWh로 늘 것으로 전망되고[1], 유럽연합은 일정 규모 이상의 데이터센터에 에너지 사용 보고를 의무화했다[2]. 이미 가동 중인 인프라에 적용할 수 있는 수단의 하나가 작업을 탄소가 적게 나오는 시각과 리전으로 옮기는 탄소 인지 스케줄링이며, 본 논문은 이때 작업이 같은 시각과 리전으로 몰려 용량을 넘기는 문제를 다룬다.

전력 1 kWh를 만들 때 나오는 탄소량, 곧 탄소집약도는 발전원 구성에 따라 시각과 지역마다 다르다. 본 연구의 여덟 리전(미국 캘리포니아·텍사스·뉴욕, 프랑스, 독일, 한국, 일본, 인도)에서 2025년(1월 8일부터) 평균은 인도 543.2, 프랑스 13.6 gCO_2/kWh로 약 40배 벌어지고, 캘리포니아는 하루 최댓값이 최솟값의 평균 3.93배다(4.6절). 배치 작업, 곧 마감 안에만 끝나면 되는 계산은 탄소집약도가 낮은, 곧 깨끗한 시각과 리전으로 옮길 여유가 있다.

선행 연구는 이 격차를 시간 축(구글의 CICS[3])이나 공간 축(CASPER[4], VMware의 전역 부하 분산[5])에서 써 왔다. 두 축을 함께 다루어 절감의 이론적 최대를 구한 Sukprasert 등[6]은 공간 이동이 지배적이라고 결론지었고, 용량이 공간 이동의 이득을 좌우함을 스스로 확인하고도 두 축을 결합한 분석에는 리전 용량을 두지 않았다. 용량을 다룬 연구는 다른 것을 양보한다. Flex(옛 이름 CarbonFlex)는 단일 클러스터 안의 시간 이동에 한정되고[7], Caspian[8]은 마감을 목적함수의 벌점으로 완화한다. 리전별 용량과 반드시 지키는 마감을 함께 둔 Zanotto 등[9]은 작업이 도착할 때 이미 정한 배정을 모두 보고 그 작업의 자리를 한 번 정하며, 용량을 얼마로 둘지는 향후 과제로 남겼다(2장, 표 1).

본 논문의 리전 용량은 한 운영 주체가 리전마다 확보한 전용 용량, 곧 동시에 돌릴 수 있는 작업 수의 상한이며, 상한 위반은 리전이 이미 상한만큼 차 있는 순간에 작업을 시작한 것이다. 작업이 제출된 리전(홈 리전)에서 모든 작업을 바로 실행할 때의 리전별 최대 동시 실행 수를 기준 피크라 하고, 사업자가 설비 규모를

공개하지 않으므로 상한은 실험 변수로 둔다. 실험은 리전마다 하루 50건씩 만든 합성 워크로드와 공개 Azure VM 기록에서 가져온 Azure 워크로드로 하며, 가장 큰 기준 피크의 1.25배를 올림해 모든 리전에 준 대표 상한은 각각 17건과 43건, 이때 이용률(모든 작업의 실행 시간 합 ÷ (상한 × 리전 수 × 1년))은 8.4%와 26.1%다(4.1절).

용량을 빼면 무슨 일이 생기는지는 본 연구의 자료에 드러난다. 1년치 탄소집약도를 미리 아는 Sukprasert 등의 정책을 원문 수식대로(바꾼 점은 4.8절) 합성 워크로드에 돌리면, 작업 143,200건 가운데 99.75%가 프랑스 한 리전으로 몰려 배출이 96.63% 준다. 그러나 이 배정이 위반 없이 돌려면 리전 하나가 동시에 129건, 곧 가장 큰 기준 피크(13건)의 9.9배를 돌려야 한다. 용량을 제약으로 두면 물음은 어느 리전이 깨끗한가가 아니라, 받아 줄 수 있는 만큼만 보내고 남은 작업을 어디에 언제 둘 것인가로 바뀐다.

이 문제가 어려운 것은 가장 깨끗한 리전의 가장 깨끗한 시각이 모든 작업에게 같기 때문이다. 작업이 도착할 때 리전과 시작 시각을 확정하되, 빈자리는 이미 시작한 작업만 세고 앞서 정해 둔 다른 작업의 계획은 세지 않으며, 시작할 때 다시 확인하지도 않는다고 하자. 그러면 작업들이 하루 중 탄소집약도가 가장 낮은 같은 시간대를 골라 두었다가 한꺼번에 시작해 상한을 넘긴다. 본 논문은 이를 시작한 작업만 확인형이라 부른다. Carbon Aware SDK[10]처럼 다른 작업의 계획을 보지 않는 도구로 시각을 정하면 같은 몰림이 생길 수 있고, 단일 클러스터[7], 고성능 계산(HPC) 시설[11], 실시간 수요반응[12]에서도 같은 몰림이 보고되었다.

몰림을 상한에서 막으려는 간단한 방법은 둘이다. 하나는 도착 때 확정하되 앞서 확정된 다른 작업의 배정을 모두 세는 것으로, Zanotto 등의 결정 방식을 CAST와 같은 선형 계획 위에 구현한 이것을(그 시스템 자체는 아니다) 도착 고정형이라 부른다. 다른 하나는 시작할 때 다시 확인해 자리가 없으면 미루다가, 더 미룰 수 없게 되면 처음 정한 리전에 밀어 넣거나(입장 통제형) 그 순간 빈 리전 가운데 가장 깨끗한 곳으로 보내는(대피형) 것이며, 실제 클러스터 스케줄러가 시작 때 확인하면 이 둘에 가깝다. 확인 방식만 바꾸어 1년을 돌리면 시작한 작업만 확인형은 합성 작업의 9~17%가 상한 위반으로 시작했고 입장 통제형도 위반이 남았으나, 도착 고정형과 대피형은 위반이 없었다(4.4절). 그러나 둘 다 도착 때 정한 자리를 따르므로 나중에 더 깨끗한 자리가 비어도 옮기지 않는다.

CAST는 미래 시각을 확정하지 않는다. 한 시간을 한 슬롯으로 보고, 매 슬롯 아직 시작하지 않은 모든 작업의 리전과 시작 시각을 하나의 선형 계획으로 함께 풀어 지금 시작할 작업만 시작시키고, 나머지는 새로 도착한 작업과 함께 다음 슬롯에 다시 푼다. 다른 작업의 계획을 모두 세어 용량을 본다는 점은 Zanotto 등[9]과 같고, 가장 큰 다른 점은 매 슬롯 다시 푼다는 것이다. 그래서 깨끗한 시각이 차면 남은 작업은 다음으로 깨끗한 자리로 가고, 더 깨끗한 자리가 생기면 그리로 옮긴다. 마감은 하드 제약으로 지키고, 마감이 코앞인 작업이 상한보다 많은 순간에도 해가 있도록 상한은 넘긴 양에 가장 큰 벌점을 매기는 소프트 제약으로 둔다(3.3절). 미룰 수 없는 작업을 위해 미래 슬롯에 몇 건분을 비워 두는 예약도 둘 수 있으나, 예약을 바꿔 본 부하에서는 예약 없이도 위반이 없었다(4.7절).

목적식은 초과 벌점 다음으로 예측 배출과 왕복 지연(홈 리전과 실행 리전 사이를 오가는 시간으로, 원격 실행 중 자료 교환과 장애 노출의 비용)의 가중합이다. 이하 지연은 실행 시간으로 가중한 평균 왕복 지연이고, 시작을 미룬 시간은 대기 시간이라 부른다. 사람이 가중치 α를 정하지 않아도 되도록 CAST는 매 슬롯 $\alpha = 0$, 0.1, …, 1로 각각 풀고, 배출과 지연이 모두 가장 작은 이상점에 가장 가까운 후보를 고른다. 이 기본 운용 규칙을 편의상 무릎점 규칙이라 부른다(3.4절). 탄소집약도는 LSTM으로 24시간 앞까지 예측하고 배출은 실측으로 센다. 매 슬롯 다시 풀고 지금 결정만 확정하는 롤링 지평 자체는 운영연구와 모형 예측 제어의 표준 기법이고[13–15], Caspian도 매시간 다시 풀되 마감을 벌점으로 완화한다. CAST는 이 구조를 마감을 지키며 리전 용량을 나눠 쓰는 배정에 적용한다.

평균 배출계수(그 시각 그 리전 전력망 전체의 평균 탄소집약도)로 세고 왕복 지연 상한을 두지 않은 주 실험에서, LSTM 예측과 무릎점 규칙, 대표 상한(합성은 예약 3건)의 CAST는 홈 리전 즉시 실행보다 배출을 합성 워크로드에서 71.93%, Azure 워크로드에서 51.35% 줄였고, 모든 작업이 마감 안에 끝났으며 상한 위반은 없었다. Azure 상한을 줄여 이용률을 53.5%까지 올려도 위반이 없었고, 66.1%에서 처음으로 모든 리전이 찬 순간에 작업의 0.32%가 넘쳤는데 도착 고정형(0.59%)의 절반 남짓이다(4.4절). 합성에서 상한을 지운 같은 설계는 1.47%포인트(%포인트는 절감률끼리의 차) 더 줄이지만 리전 하나에 동시 63건, 상한의 3.7배를 요구하므로 이 부하에서 용량을 지키는 비용은 작다(4.3절). 절감의 대부분은 리전 이동에서 오며, 시각을 그대로 두고 리전만 옮겨도 합성에서 61.57%가 준다. 슬롯당 계산은 평균 223 ms다(4.9절).

지연을 맞춘 고정 가중치끼리 견주면 CAST는 위반이 없는 두 설계보다도 덜 배출한다. 예약 없이, 지연이 0.6 ms 안으로 같아지는 고정 α끼리 짝지으면 CAST는 도착

고정형보다 합성 0.96%포인트(다시 뽑은 워크로드 네 벌에서도 1.23~1.56), Azure 2.53%포인트, 대피형보다 합성 0.91%포인트, Azure 1.99%포인트 덜 배출했다(4.4절). 도착 고정형과의 차는 새로 도착한 작업과 함께 다시 짜는 데서 오며, 깨끗한 리전의 상한이 빠듯할수록 커서 합성에서 상한 13건이면 1.46%포인트, 깨끗한 세 리전만 작게 두면 2.10%포인트였다(넉넉한 26건에서는 0.34%포인트로 신뢰구간이 0을 포함). 한 주를 미리 다 알고 푼 오프라인 하한에도 계절이 다른 네 주 모두 CAST가 도착 고정형보다 가까웠다(합성, 예약 3, 4.5절). 기본 설정인 무릎점 규칙에서도 CAST는 같은 지연의 도착 고정형보다 대기 시간이 25% 짧다(배출 차는 아래 둘째 단서).

이 우위에는 두 단서가 붙는다. 첫째, 평균 배출계수로 센 것이다. 프랑스로 옮긴 작업만 같은 시각 독일의 탄소집약도로 센 한계 근사(프랑스만)에서는 합성의 순서가 뒤집혀 도착 고정형이 0.89%포인트, 대피형이 2.15%포인트 덜 배출했고, Azure에서는 도착 고정형보다 1.48%포인트 덜 배출하는 우위만 남았다(4.1절). 둘째, 고정 가중치에서의 값이다. 무릎점 규칙은 같은 지연의 고정 α보다 합성 아홉 조건 가운데 여덟에서 1.77~2.64%포인트 덜 줄이므로(4.5절), 무릎점 규칙의 CAST(예약 없이 72.06%)는 같은 지연의 도착 고정형과 합성에서 배출 차가 유의하지 않다(점추정은 도착 고정형이 0.94%포인트 낮다). 도착 고정형에는 시작 시각을 미리 알려 줄 수 있고 계산이 적다는 장점도 있다(4.4절).

탄소만 보면 CAST는 상한 안에서 선행 정책에 버금간다. 합성 워크로드에서 선행 정책을 재구현해(원 논문의 수치를 재현한 것은 아니다) 위반 없이 돌기 위해 요구하는 용량, 곧 필요 용량으로 다시 재면, α=1의 CAST는 상한 17건 안에서 95.65%를 줄이는데 CASPER는 95.30%를 줄이려고 가장 붐빈 리전에 가장 큰 기준 피크의 2.2배를, 1년치를 미리 아는 Sukprasert 등의 정책은 96.63%를 줄이려고 9.9배를 요구한다(4.8절). 다만 리전별 최댓값을 더한 설비 총량으로는 α=1의 CAST와 CASPER가 46건으로 같고 무릎점 규칙의 CAST가 117건으로 크며, 평균 부하가 가장 깨끗한 리전의 상한을 넘는 Azure에서는 α=1의 CAST도 용량을 무시한 배정보다 27.37%포인트 덜 줄인다(4.7절).

본 논문의 기여는 세 가지다. 첫째, 확정하지 않고 매 슬롯 다시 푸는 CAST를 제시하고, 1년 두 워크로드에서 위반 없이(Azure는 이용률 53.5%까지) 배출을 크게 줄이며, 평균 배출계수로는 지연을 맞춘 고정 가중치에서 위반 없는 대조군보다 덜 배출함을 보인다(3장, 4.3~4.5절). 둘째, 용량을 확인하는 방식만 바꾼 설계들을 견주어, 미래 시작을 서로 모른 채 확정하고 시작 때 다시 확인하지 않으면 위반이 크게 나고, 확정 계획을 모두 보거나 시작 때 빈 리전으로 보내거나 다시 풀면 대표 상한에서 위반이 0이며, 다시 풀기의 몫은 배출에 있음을 보인다(4.4절). 셋째, 선행 정책을 필요 용량으로 다시 재어 그 큰 절감이 리전 하나에 기준 피크의 2.2~9.9배를 전제함을 보인다(4.8절).

절감률은 작업당 1 kW의 실행 전력으로 세며 리전 사이 전송을 넣지 않았다. 출발지 기준으로 작업당 약 9 GB를 보내면 합성 무릎점 절감이 시각만 옮기는 방법(18.78%)과 같아지고(4.3절), 한계 근사들로 세면 합성 21~39%, Azure 18~29%(4.1절), 모든 작업에 왕복 지연 100 ms 제한을 두면 합성 32.98%다(4.7절). 주 실험은 슬롯 안 도착과 지금 슬롯의 탄소를 한 시간 미리 알고 세 리전의 LSTM에 미래 기상 실측을 넣으며, 이를 빼면 예약 없는 합성 절감이 72.06%에서 68.88%로 줄지만 위반은 그대로 0이고 설계 비교의 순서도 같다(4.4절). 워크로드의 등급 비율과 도착 분포, Azure의 홈 리전과 미룸 허용치는 모형값이다(4.1절, 5장).

2장은 관련 연구, 3장은 CAST의 구성, 4장은 실험과 분석, 5장은 결론과 한계를 다룬다.

2. 관련 연구

선행 연구는 탄소 인지 스케줄링이 배출을 얼마나 줄일 수 있는지를 여러 방향에서 보여 왔다. 시간 축만 쓰는 연구가 먼저 나왔다. Wiesner 등은 한 데이터센터 안에서 작업을 미루는 것만으로 얻는 절감을 네 지역의 전력 자료로 보였고 자원 제약은 두지 않았다고 밝혔다[16]. CarbonScaler는 한 리전 안에서 작업 하나의 서버 수를 탄소 예보에 맞춰 늘리고 줄이며[17], GAIA는 한 리전 클러스터에서 탄소, 완료 시간, 비용 사이의 맞바꿈을 다룬다[18]. GREEN은 기계학습 클러스터의 스케줄러 안에서 하루 총 용량을 지키며 작업을 저탄소 시각으로 옮긴다[19]. 넷 모두 작업이 어디서 도는지는 바꾸지 않는다. 리전과 시각을 함께 옮기면서 리전 용량과 작업 마감을 함께 제약으로 넣으려 하면, 선행 연구의 설계에서 다음 세 가지가 걸린다.

첫째, 두 축을 결합한 선행 연구는 리전 용량과 작업 마감 가운데 하나를 양보한다. Sukprasert 등은 용량이 공간 이동의 이득을 좌우함을 스스로 확인하고도 결합 분석에서는 용량을 두지 않았고[6], Attenni 등도 자원이 항상 가용하다고 가정한다[20]. Lechowicz 등의 CarbonClipper는 작업 하나를 마감 안에서 리전 사이로 옮기는 온라인 문제에 경쟁비 보장, 곧 미래를 다 아는 최적해와 견준 최악의 경우 비율에 대한 보장을 주었으나, 여러 작업이 자원을 두고 다투는 상황과

데이터센터 용량은 정식화에 없다고 적었다[21]. Hall 등은 하루 전 계획과 실시간 배정으로 부하를 시공간으로 옮기며 확률적 성능 보장을 주지만, 대상은 개별 작업이 아니라 여러 작업을 합친 집계 부하다[22]. Caspian은 반대로 용량을 하드 제약, 곧 어떤 경우에도 어길 수 없는 제약으로 두는 대신 마감을 목적함수의 벌점으로 완화한다[8]. 본 연구 자료에서 재현하면 채점 작업의 3.90%인 5,582건이 마감을 넘긴다(4.8절). Caspian은 매시간 다시 풀지만 용량을 지키려고 마감을 양보한다. 탄력적 확장으로 용량이 늘 수 있다고 볼 수도 있으나, 본 연구는 운영 주체가 리전마다 확보한 전용 용량을 상한으로 두고 시각마다 바뀌는 공용 여유는 넣지 않는다.

둘째, 용량을 두는 연구 가운데 여럿은 작업이 들어오는 순간에 한 번 판단하고 다시 보지 않는다. Zanotto 등은 본 연구와 가장 가깝다. 리전별 동시 실행 상한, 작업별 하드 마감, 자체 예측기를 갖춘 쿠버네티스 구조다. 요청이 도착할 때마다 이미 정한 다른 작업의 배정을 모두 보고 정수 계획으로 그 요청의 리전과 시각을 정하며, 그 뒤에는 다시 풀지 않는다. 다만 평가는 1개월 동안 1,000건 묶음을 반복한 것이고 비교 대상은 라운드로빈 하나이며, 상한을 어떻게 정할지는 향후 과제로 남겼다[9]. 이 결정 방식은 CAST와 같은 선형 계획 위에 구현해 4.4절의 대조군인 도착 고정형으로 재현한다. CICS는 하루 전에 산정한 자원 곡선 하나로 다음 날의 유연한 부하를 묶으므로 개별 작업의 마감을 보지 못한다[3]. Flex는 단일 클러스터 안에서 용량을 보지 않는 시간 이동이 저탄소 시각에 부하를 모은다는 점을 지적했으나, 리전 사이의 이동은 다루지 않는다[7]. COAST도 작업을 각자 저탄소 시각으로 보내면 같은 슬롯에 몰린다는 데서 출발해, 혼잡을 하드 용량 제약이 아닌 소프트 벌점으로 두고 작업들 사이의 혼잡 게임으로 시작 시각을 나눈다[11]. 대상은 단일 HPC 시설의 시간 이동이며, 예측이 좋아질수록 더 몰린다는 점은 다루지 않는다. 입장 조건이란 작업 하나가 들어올 때 그 작업이 고른 시각에 자리가 있는지만 묻는 확인이다. 용량을 이 조건으로만 두면 여러 작업이 같은 저탄소 시각을 노리고 있다는 사실은 그 시각이 찬 뒤에야 드러난다. 이미 정한 배정을 모두 보는 Zanotto 등의 방식은 이 몰림을 상한에서 멈추게 하지만, 한 번 정한 자리는 나중에 더 깨끗한 자리가 보여도 바꾸지 않는다.

셋째, 탄소와 그 밖의 목적 사이의 가중치가 고정 상수다. CASPER는 탄소와 서버 프로비저닝 비용 사이의 가중치를 고정하며 저자들 스스로 다른 값에서는 결과가 달라질 것이라고 밝혔다[4]. CICS 역시 탄소 비용과 피크 전력 비용의 가중합을 쓰고[3], 두 축을 하나의 정수 계획으로 결합한 Caspian도 가중치를 사람이 고르며 그 값을 신중하게 정해야 한다고 결론지었다[8]. 그 값을 미리 정할 근거는 주어지지 않는다.

지리적 부하 분산 연구는 탄소 인지 연구보다 먼저 일을 데이터센터 사이로, 또 시간 축으로 옮겨 전기요금이나 화석 에너지 사용을 줄이는 문제를 다뤘다. Liu 등은 요청을 리전 사이로 나눠 보내 화석 에너지를 줄이는 조건을 보였다[23]. Lin 등은 데이터센터별로 켜 둘 서버 수를 정하는 문제에서, 예측한 부하 창 위에서 매번 다시 최적화하고 지금 결정만 실행하는 이동 지평 제어(RHC)가 나빠질 수 있는 조건과 이를 막는 평균 고정 지평 제어(AFHC)를 보였다[24]. Ren 등은 여러 조직의 배치 작업을 대기열 지연 제약 아래 여러 데이터센터로 보내는 온라인 알고리즘을[25], GreenSlot은 한 데이터센터 안에서 태양광 예측에 맞춰 배치 작업을 마감 안의 시각으로 옮기는 스케줄러를 제시했다[26]. CAST는 이 흐름 위에서 다음 경우를 다룬다. 작업마다 정해진 마감을 지켜야 하고, 리전마다 정한 동시 실행 상한을 대기 중인 모든 작업이 함께 나눠 쓴다. 이 조건에서 작업마다 리전과 시작 시각을 한 선형 계획으로 함께 고르고, 실제로 시작하는 순간에만 확정한다. Lin 등의 분석은 서버를 켜고 끄는 전환 비용을 두지만, 본 정식화에는 전환 비용 항이 없다.

매 기간 앞날 전체를 다시 최적화하되 당장의 결정만 실행하는 롤링 지평 방식은 운영연구의 생산계획과 용량 계획[13, 15], 모형 예측 제어[14]에서 오래 쓰여 온 표준 구조이고, 온라인 스케줄링에서는 작업을 도착 때 확정하는 것과 시작할 때 확정하는 것을 나누는 확정 시점의 분류가 알려져 있다[27]. 이 분류로 보면 Zanotto 등의 방식은 도착 때 확정, CAST는 시작 때 확정에 해당한다. 다만 [27]은 단일 서버의 경쟁비 분석이므로 본 연구는 분류만 빌리고 성능 보장은 주장하지 않는다.

본 연구는 이 세 가지를 다음과 같이 다룬다. 마감은 하드 제약(식 (4))으로 양보하지 않고, 용량은 초과량에 가장 큰 벌점을 둔다(식 (3), (6)). Azure 워크로드에서 이용률 53.5%까지는 상한을 넘긴 적이 없고, 66.1%에서 넘긴 465건은 모두 그 순간 모든 리전이 상한까지 찬 때였다(4.4절). 용량은 입장 조건이 아니라 모든 대기 작업이 함께 보는 제약으로 두어 리전과 시작 시각을 매 슬롯 한 문제로 다시 푼다(3.4절). 가중치는 슬롯마다 무릎점 규칙, 곧 배출과 지연이 모두 가장 작은 이상점에 가장 가까운 후보를 고르는 규칙으로 정한다(3.4절). Zanotto 등과 비교하면 본 연구는 1년 동안 이어지는 작업 흐름, 탄소 인지 선행 정책 넷과의 같은 회계 비교, 상한 값을 바꿔 가며 본 분석을 더 갖추었고, 실제 클러스터 프로토타입은 갖추지 못했다. 본 연구가 재현한 선행

표 1 선행 연구의 설계 성질. ○ 있음, △ 벌점·부분 분석·집계 부하, × 없음, — 해당 없음. CAST의 용량은 가장 큰 벌점을 둔 소프트 제약이며 이용률 53.5%까지 초과는 0이었고, COAST의 마감은 권고형이다.

Table 1 Design properties of prior work. ○ yes, △ penalty, partial analysis or aggregate load, × no, — not applicable. CAST's capacity is a soft constraint with the largest penalty (no excess up to 53.5% utilization); COAST's deadlines are advisory.

Work	Time	Space	Region cap	Hard deadline	Re-solve
Wiesner, CarbonScaler, GAIA [16–18]	○	×	×	△	×
GREEN [19]	○	×	△	×	○
CICS [3]	○	×	△	×	×
CASPER [4]	×	○	△	—	—
Sukprasert [6]	○	○	△	○	×
CarbonClipper [21]	○	○	×	○	—
Hall [22]	○	○	△	△	×
Flex [7]	○	×	○	△	○
COAST [11]	○	×	△	○	×
Caspian [8]	○	○	○	△	○
Zanotto [9]	○	○	○	○	×
CAST	○	○	△	○	○

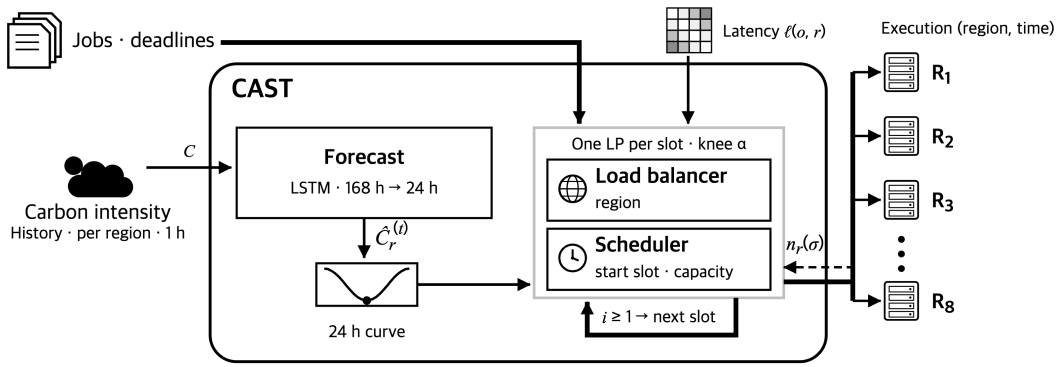


그림 1 CAST의 전체 파이프라인. 굵은 선은 작업, 가는 선은 탄소·지연 자료, 파선은 리전별 점유가 스케줄러로 다시 들어가는 길이며, 회색 테두리 안의 로드밸런서와 스케줄러는 매 슬롯 하나의 선형 계획으로 함께 풀린다.

Fig. 1 Overall pipeline of CAST. Thick lines carry jobs, thin lines carbon and latency data, and the dashed line returns each region's occupancy to the scheduler; the load balancer and scheduler inside the gray frame are solved together as one linear program per slot.

정책 가운데 리전 용량을 입장 조건으로 두는 것은 없다. CICS는 자기 자원 곡선으로, CASPER는 스스로 정한 서버 용량으로 묶을 뿐 리전 상한을 두지 않는다. Sukprasert는 용량을 두지 않고, Caspian은 용량을 하드 제약으로 둔다. 그래서 용량을 시작할 때만 보는 설계에서 생기는 몰림은 이들의 평가에서 드러날 자리가 없었다. 표 1에 이 성질을 모았다.

3. CAST 시스템

CAST는 예측기, 로드밸런서, 스케줄러의 세 모듈로 구성된다. 예측기는 리전별로 향후 24시간의 탄소집약도를 산출한다. 로드밸런서는 각 작업을 어느 리전에서 돌릴지를, 스케줄러는 그 리전에서 언제 시작할지를 정한다. 다만 로드밸런서와 스케줄러는 차례로 도는 두 단계가 아니라, 매 1시간 슬롯마다 하나의 선형 계획 안에서 함께 풀리는 두 축이다. 지금 시작하는 작업만 확정해 실행 기록, 곧 리전마다 지금 도는 작업 수를 적어 두는 기록에 올리고, 나머지는 다음 슬롯에 새로 발행된 예측으로 다시 푼다. 그림 1이 세 모듈과 모듈 사이를 오가는 값을 보인다.

설계 원칙은 셋이다. 첫째, 용량은 입장 때 확인하는 조건이 아니라 대기 중인 모든 작업이 함께 보는 제약으로 두고, 리전과 시각도 그 제약 아래에서 한 문제로 정한다(3.4절). 둘째, 확정 계획을 유지하지 않고 매 슬롯 다시 풀어 지금 시작하는 작업만 확정한다. 셋째, 배정은 예측값으로 정하고 배출량은 실측값으로 센다. 둘 다 예측값을 쓰면 예측 오차의 대가가 결과에서 사라지기 때문이다.

세 모듈이 지향하는 문제를 먼저 1년치를 한 번에 보는 참조 문제로 적는다. 작업 집합을 J, 리전 집합을 R로 둔다. 슬롯 길이는 탄소집약도 자료의 해상도에 맞춘 1시간이다. 작업 j는 홈 리전 o_j(작업이 제출된 리전, 곧

출발 리전), 제출 시각 s_j, 실행 시간 d_j, 미룸 등급 k_j(미룰 수 있는 정도로 나눈 작업 종류), 미룸 허용치 L_j(시작을 미룰 수 있는 시간)를 가지며, 마감은 $D_j = s_j + L_j + d_j$다. 왕복 지연 상한 ℓ_j는 그 작업이 허용하는 왕복 지연의 최댓값으로, 주 실험에서는 두지 않는다($\ell_j = \infty$). 그래서 주 실험에서는 어느 리전으로든 옮길 수 있어 결과가 리전 이동에 크게 좌우되며, 모든 작업에 공통 왕복 지연 상한 50~150 ms를 둔 결과는 4.7절에 있다. 리전 r은 동시에 돌릴 수 있는 작업 수의 상한 cap_r을 가진다. 정하는 값은 실행 리전 $r(j)$와 시작 시각 τ_j 둘이고, 리전 r에서 시각 t에 돌고 있는 작업 수를 $n_r(t)$로 쓴다. 3.3절에서 매 슬롯 푸는 문제는 $n_r(t)$를 두 몫으로 나누어 센다. 이미 시작해 실행 기록에 오른 확정 작업은 상수 $n_r(\sigma)$(슬롯 σ를 조금이라도 덮는 확정 작업 수)와 $n_r(u)$(순간 u에 실행 중인 확정 작업 수)로 세고, 아직 시작하지 않은 작업은 식 (11)(12)의 Σx 항이 센다. t는 식 (1)(3)(6)에서 연속 시각이고, 3.3절부터는 슬롯 번호다.

목적은 사전식, 곧 앞 목표를 먼저 최소화하고 그 값을 지키는 범위에서 다음 목표를 줄이는 순서다. 먼저 상한을 넘긴 양 Z를 최소화하고, 그 위에서 지연 Λ가 주어진 지연 예산 $\bar{\Lambda}$를 넘지 않는 배정 가운데 배출 E를 최소화한다. 한 줄로 쓰면 $r(j)$와 τ_j에 대해 $(Z,\ E)$를 사전식으로 최소화하되 $\Lambda \le \bar{\Lambda}$와 식 (4)~(6)을 지키는 문제이며, 이를 참조 문제 (P0)라 부른다. (P0)는 평가의 틀이다. 4장에서 고정 α를 같은 지연 짝으로 견주는 평가, 곧 지연을 같게 맞춘 뒤 배출을 견주는 평가가 이 꼴이다. 매 슬롯 실제로 푸는 3.3절의 식 (9)~(12)는 참조 문제를 대신해 푸는 문제이고, 예약을 침범한 양 z^{B}에 벌점을 매기는 단계는 (P0)에 없는 운용 장치다.

$$E = \sum_{j \in J} P \int_{\tau_j}^{\tau_j + d_j} C_{r(j)}(t)\,dt \tag{1}$$

$$\Lambda = \sum_{j \in J} d_j\, \ell(o_j, r(j)) \tag{2}$$

$$Z = \sum_{r \in R} \int \max\{0, n_r(t) - \mathrm{cap}_r\}\,dt \tag{3}$$

여기서 $C_r(t)$는 리전 r의 실측 탄소집약도, P는 작업의 소비 전력, $\ell(o, r)$은 홈 리전 o에서 실행 리전 r까지의 왕복 지연이다. E는 총 배출량이고, Z는 상한을 넘긴 작업 수를 시간으로 적분한 초과량이다. Λ는 작업마다 실행 시간 d_j와 왕복 지연을 곱해 더한 원격 실행 비용이며, 이하 지연이라 부른다(무릎점 규칙과 같은 지연 짝은 이를 실행 시간의 합으로 나눈 평균으로 재며, 작업당 평균은 따로 밝힌다). 지연은 시작을 미룬 시간인 미룸과 다르며, 미룸은 4장의 대기 시간과 같은 지표다. 4.1절의 위반 건수(이미 상한만큼 찬 리전에서 시작한 배정 수)와 초과 시간(상한을 넘긴 시간의 합)은 Z와 같은 초과를 건수와 시간으로 센 것이다. 매 슬롯 푸는 식 (7)~(9)는 $\bar{\Lambda}$ 대신 가중치 α로 배출과 지연을 맞바꾼다. 제약은 세 가지다.

$$s_j \le \tau_j, \quad \tau_j + d_j \le D_j \qquad \forall j \in J \tag{4}$$

$$\ell(o_j, r(j)) \le \bar{\ell}_j \qquad \forall j \in J \tag{5}$$

$$n_r(t) \le \mathrm{cap}_r + z_r(t),\ z_r(t) \ge 0 \quad \forall r \in R,\ \forall t \tag{6}$$

식 (4)는 실행이 제출 이후에 시작해 마감 이내에 끝나야 한다는 뜻으로, 마감은 무를 수 없는 하드 제약이다. 식 (5)는 왕복 지연 상한을, 식 (6)은 용량을 정한다. 왕복 지연은 응답 경로의 지연으로만 세고 완료 시각에는 더하지 않는다. $z_r(t)$는 상한을 넘긴 양을 나타내는 변수이며, 이를 모든 리전과 시각에 걸쳐 적분한 것이 식 (3)의 Z다.

식 (6)이 본 연구의 형식화가 선행 연구와 갈라지는 지점이다.

용량을 두지 않으면 최적해는 가장 깨끗한 리전으로 몰린다. 용량을 무를 수 없는 제약으로 두면 마감이 코앞인 작업이 상한보다 많이 몰리는 순간 해가 없다. 초과를 변수로 두면 항상 해가 있고, 용량을 얼마나 어겼는지가 수식 밖의 사정이 아니라 목적함수의 값이 된다.

작업은 둘로 갈린다. 제출 슬롯 말고는 시작할 정수 슬롯이 없는 작업, 곧 $\lfloor D_j - d_j \rfloor \le \lfloor s_j \rfloor$인 작업을 미룰 수 없는 작업이라 부르고, 나머지를 미룰 수 있는 작업이라 부른다. 합성 워크로드에서 미룰 수 없는 작업은 건수로 59.1%지만 실행 시간이 초에서 분 단위라 총 작업시간의 1.3%에 그친다.

식 (1)~(6)을 1년치 전체에 대해 한 번에 풀지 않은 것은 시스템을 작업이 도착하는 대로 결정하는 온라인 방식으로 두기 위해서다. 한 번에 풀면 모든 작업과 1년치 탄소집약도를 미리 알아야 한다. 그래서 (1)~(6)은 참조 문제로 두고, 실제로는 매 슬롯 3.3절의 식 (9)~(12)를 푼다.

3.1 예측기

예측기는 리전마다 독립인 2층 LSTM이며, 은닉 크기는 64, 드롭아웃은 0.2이고 마지막 은닉 상태를 선형층으로 24개 값에 잇는다. 과거 168시간의 탄소집약도, 무탄소·재생 전력 비율, 시각, 요일, 월의 주기 변수, 공휴일 여부를 받아 향후 24시간을 한 번에 출력한다. 캘리포니아, 텍사스, 독일의 세 리전은 풍속, 일사량, 기온(Open-Meteo 과거 기상 API의 시간 단위 값)을 입력에 더하고, 향후 24시간의 같은 기상 변수(24×3=72개)도 출력층에 함께 넣는다.

학습은 2021~2023년, 검증은 2024년, 시험은 2025년으로 연도를 끊었고, 스케줄링 실험은 시험 연도

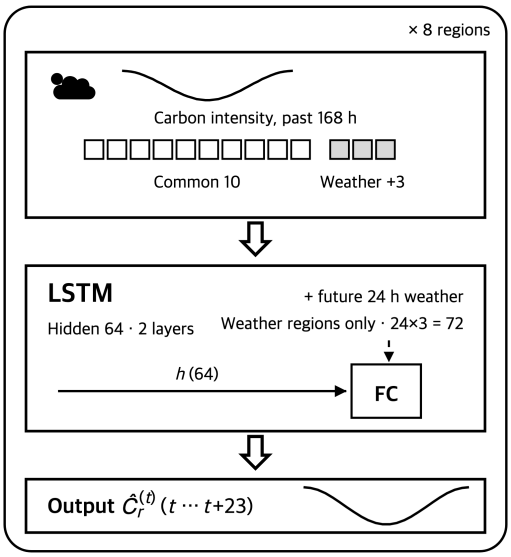


그림 2 리전별 LSTM 구조. 모델은 리전마다 독립이다.
Fig. 2 Per-region LSTM architecture. One model per region.

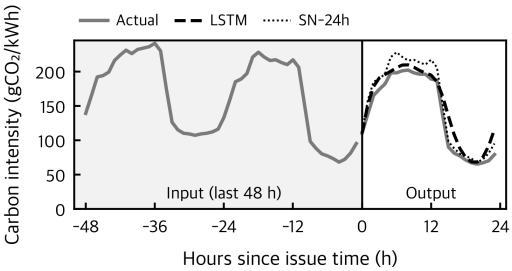


그림 3 캘리포니아의 탄소집약도 예측 예. 한 구간이며 왼쪽은 입력 168시간 중 마지막 48시간이다. 이 구간의 평균 절대 오차는 LSTM 12.2 gCO₂/kWh, 점선인 24시간 전 값(SN-24h, 어제 같은 시각의 값) 14.0이다.
Fig. 3 Example carbon intensity forecast for California. One window; the left part shows the last 48 of the 168 input hours; the mean absolute error over this window is 12.2 gCO₂/kWh, against 14.0 for the dotted SN-24h forecast (the value at the same hour on the previous day).

위에서만 돈다. 입력은 학습 연도에만 맞춘 최솟값–최댓값 척도로 0~1에 옮긴다. 손실은 정규화한 값의 평균 제곱 오차이고, Adam(학습률 0.001, 가중치 감쇠 0.0001)으로 배치 64씩 학습하며 기울기 노름을 1.0으로 자른다. 검증 오차가 8 학습 회차(epoch) 동안 줄지 않으면 학습률을 절반으로 낮추고, 15 회차 동안 줄지 않거나 가장 좋았던 회차 뒤 30 회차가 지나면 멈추되, 최소 20 회차, 최대 150 회차를 돈다. 모델은 원래 단위로 되돌린 검증 평균 절대 오차가 가장 작은 회차의 것을 쓰며, 난수 시드는 42다. 구조는 그림 2와 같고, 그림 3은 그 모델이 실제로 낸 예측 한 구간이다.

하루 앞 탄소집약도 예측은 선행 연구가 따로 다룬 문제이고[28] 본 연구의 기여가 아니다. 예측기가 뒤의 두 모듈에 넘기는 것은 값 하나하나보다 곡선의 모양, 곧 리전과 시각에 따라 탄소가 어디서 얼마나 깊이 내려가는지다. 로드밸런서와 스케줄러는 그 모양을 보고 자리를 고르기 때문이다. 예측 오차가 스케줄링 결과에 어떻게 이어지는지, 그리고 24시간 전 값을 그대로 쓰는 단순한 예측기와 비교한 결과는 4.2절에 있다.

여기까지가 예측기다. 이제부터는 그 예측을 받아 자리를 고르는 두 모듈을 적는다.

3.2 로드밸런서

로드밸런서는 작업마다 실행 리전을 고른다. 후보는 여덟 리전 전부다. 원리상 식 (5)를 어기는 리전, 곧 홈 리전에서의 왕복 지연이 그 작업의 왕복 지연 상한 ℓ_j보다 긴 리전은 후보에서 빠진다. 주 실험은 왕복 지연 상한을 두지 않으므로 이 제약이 걸리지 않지만, 왕복 지연 상한을 두는 운영에서는 이것이 공간 이동의 범위를 정한다(4.7절). 왕복 지연은 전 구간에서 고정된 8×8 실측 행렬에서 읽는다(4.1절).

후보는 리전 r과 시작 시점 i의 쌍이며, i는 지금부터 몇 슬롯 뒤에 시작하는지를 뜻한다(자세히는 3.3절). 후보의 비용은 정규화한 탄소 비용과 지연 비용을 가중치 α로 더하고 실행 시간을 곱한 값이다. α는 탄소 비용에 두는 무게이고, 나머지 몫이 지연 비용에 간다.

$$c_j(r,i) = d_j\left[\alpha \tilde{C}_j(r,i) + (1-\alpha)\tilde{\ell}(o_j,r)\right] \qquad (7)$$

$$\tilde{C}_j(r,i) = \frac{\hat{C}_r^{(t)}(t+i,d_j)}{\max_{r',t'} \hat{C}_{r'}^{(t)}(t')}, \qquad \tilde{\ell}(o,r) = \frac{\ell(o,r)}{244} \qquad (8)$$

여기서 $\hat{C}_r^{(t)}(t+i, d_j)$는 슬롯 t에 발행된 예측에서 $t+i$부터 d_j 동안의 평균 탄소집약도이고, 분모는 그 슬롯에 발행된 예측 전체, 곧 모든 리전과 예측 지평(예측이 내다보는 앞 시간 범위)을 통틀은 최댓값이다. $\tilde{C}_j(r,\ i)$는 이렇게 정규화한 탄소 비용이고, $\tilde{\ell}(o,\ r)$은 아래처럼 지연 행렬의 최댓값으로 나눈 지연 비용이다. 지금 슬롯(i=0)의 첫 한 시간은 예측 대신 그 시각의 실측을 쓴다. 평균을 내는 창은 예측 지평 끝에서 자르므로, 그보다 긴 작업은 지평 안의 평균으로 판단한다. i=24는 t+23의 예측으로 본다. 지연 비용은 전 구간에서 고정된 값인 지연 행렬의 최댓값 244 ms로 나눈다. 탄소 비용만 슬롯마다 그 슬롯의 최댓값으로 나누는 것은, 슬롯 사이 탄소집약도 차이의 절대 크기가 α의 뜻에 섞이지 않게 하려는 것이다. 그

대가로 같은 α라도 슬롯마다 배출(kg)과 지연(ms)을 맞바꾸는 비율이 다르다. 실행 시간 d_j를 곱한 것은 긴 작업이 그만큼 오래 배출하고 오래 원격 리전을 점유하기 때문이며, α=1이면 식 (7)의 합이 예측 배출량에 비례한다.

용량을 보지 않는 선행 정책은 작업을 가장 깨끗한 리전으로 몰 수 있다(1장에서 본 Sukprasert 등의 정책으로, 뒤에서는 Sukprasert 공간+시간이라 부른다). CAST의 로드밸런서는 각 리전이 받아 줄 수 있는 만큼만 보낸다. 그 '만큼'은 3.3절의 상한 행, 곧 리전별 상한을 적은 제약식이 정하고, 남은 작업은 다음으로 깨끗한 리전이나 다른 시각으로 간다. 1년 동안 실제로 만들어진 리전 간 흐름은 4.3절에서 본다.

3.3 스케줄러

스케줄러는 작업마다 시작 슬롯을 고르고, 리전별 실행 기록으로 용량을 지킨다. 매 슬롯 t마다 아직 시작하지 않은 작업을 모아 대기 집합 Q를 만든다. 미룰 수 있는 작업은 언제 도착했든 시작하기 전까지 Q에 남고, 이번 슬롯에 도착한 미룰 수 없는 작업도 Q에 들어간다. 슬롯 t에 제출되는 작업은 슬롯 시작 시점에 모두 알려진 것으로 보고 한 번에 배정한다. 슬롯 안에서 나중에 도착할 작업을 최대 한 시간 먼저 아는 셈이며, 이 가정은 4장의 모든 대조군에 같게 적용한다.

i=0은 지금, 곧 슬롯 머리(슬롯이 시작하는 정시) t와 제출 시각 s_j 가운데 늦은 때인 $\max(t,\ s_j)$에 시작하는 것이고, $i\geq$1은 슬롯 t+i가 시작하는 때에 시작하는 것이다. i의 최댓값 h_j는 마감 안에 끝낼 수 있는 마지막 시작 시각과 예측 지평의 끝 가운데 이른 쪽으로 정한다. 곧 $h_j = \lfloor \min(D_j - d_j,\ t+H) - t \rfloor$이며 지평 H는 24다. 미룰 수 없는 작업과, 여유, 곧 마지막 시작 시각 $D_j - d_j$까지 남은 시간이 한 슬롯보다 적은 작업($(D_j - d_j) - t < 1$)은 i=0만 허용한다. 후보를 예측 지평 안으로 제한한 것은 예측이 24시간까지만 있어 그 너머는 비교할 근거가 없기 때문이다. 그래서 하루를 넘는 작업의 리전도 앞 24시간 예측으로 정하고, 실행 구간이 지평 끝을 넘으면 지평 안의 평균 탄소집약도로 비용을 매긴다.

슬롯 t의 배정 문제는 다음과 같다. x_{jri}는 작업 j를 리전 r에서 i 슬롯 뒤에 시작하게 두는 변수다. 목적식은 후보 비용의 합에 두 초과 벌점과 ε항을 더한 것이다. z^A는 상한 cap_r을 넘긴 양(물리 초과), z^B는 미룰 수 있는 작업의 계획이 예약 ρ_r(미룰 수 없는 작업을 위해 비워 두는 자리)을 침범한 양(예약 침범)이며, M_A와 M_B는 각각에 매기는 벌점 계수다. ε은 이른 시작을 고르게 하는 작은 계수다.

$$\min \sum_{j\in Q}\sum_{r,i} c_j(r,i)\,x_{jri} + M_A \sum_{r,\kappa} z^A_{r\kappa} + M_B \sum_{r,\sigma\geq t+1} z^B_{r\sigma} + \varepsilon \sum_{j\in Q}\sum_{r,i} i\,x_{jri} \tag{9}$$

$$\sum_{r,i} x_{jri} = 1, \qquad \forall j \in Q \tag{10}$$

$$\sum_{(j,0)\ni u} x_{jr0} + n_r(u) \leq \mathrm{cap}_r + z^A_{ru} \quad \forall r,\ u \in U_t \tag{11a}$$

$$\sum_{(j,i)\ni \sigma} x_{jri} + n_r(\sigma) \leq \mathrm{cap}_r + z^A_{r\sigma} \quad \forall r,\ \sigma \in [t+1, t+H] \tag{11b}$$

$$\sum_{(j,i)\ni \sigma,\ j\in Q^{\mathrm{def}}} x_{jri} + n_r(\sigma) \leq \mathrm{cap}_r - \rho_r + z^B_{r\sigma} \quad \forall r,\ \sigma \in [t+1, t+H] \tag{12}$$

식 (10)은 한 작업을 한 자리에만 둔다는 뜻이다. 식 (11a)(11b)는 식 (6)의 물리 상한을 지금 슬롯의 시작 순간과 미래 슬롯에 대해 적은 것이고, 식 (12)는 미룰 수 있는 작업이 미래 슬롯을 끝까지 채우지 못하게 ρ_r 자리를 비워 두는 예약으로, 모든 리전과 $\sigma \in [t$+1, t+$H]$에 대해 성립한다. 기호는 다음과 같다. 변수는 $0 \leq x_{jri} \leq 1$, $z^A, z^B \geq 0$이고, 첨자는 $i \in \{0, \ldots, h_j\}$, $\sigma \in \{t$+1, …, t+$H\}$다. 리전 r은 왕복 지연 상한을 지키는 리전만 모은 $R_j = \{r \in R : \ell(o_j, r) \leq \ell_j\}$ 안에서 고른다. 곧 식 (5)를 변수의 정의역에 넣으며, 식 (9)(10)의 리전 합도 R_j 위에서 잡는다. σ는 슬롯 번호로, 작업의 시작 시각 τ_j와 다르다. $(j,\ i) \ni \sigma$는 작업 j가 리전 r에서 i 슬롯 뒤에 시작해 d_j 동안 도는 실행이 슬롯 σ와 겹친다는 뜻이다. $U_t = \{t\} \cup \{$이번 슬롯에 도착한 작업의 $s_j\}$는 지금 슬롯 안에서 작업이 시작하는 순간, 곧 슬롯 머리 t와 이번 슬롯에 도착한 작업의 제출 시각을 모은 집합이다. $(j,\ 0) \ni u$는 지금 시작하는 작업 j의 실행 $[\max(t,\ s_j),\ \max(t,\ s_j)+d_j)$가 순간 u를 포함한다는 뜻이고, z^A의 첨자 κ는 식 (11a)의 $u \in U_t$와 식 (11b)의 σ를 함께 가리킨다. $n_r(\sigma)$는 슬롯 σ를 조금이라도 덮는 확정 작업의 수, $n_r(u)$는 순간 u에 실행 중인 작업 수로 둘 다 실행 기록에서 읽는다. Q^{def}는 대기 작업 가운데 미룰 수 있는 작업의 집합이다. 한 슬롯 문제의 변수는 대기 작업마다 $|R_j| \times (h_j + 1)$개이므로 대기 작업 수 × 8 리전 × 최대 25 후보 슬롯 안에 있다. 변수가 가장 많았던 슬롯에서 7,848개였고(4.9절), 대기열이 이보다 수십 배 긴 규모는 재지 않았다.

초과 변수의 벌점은 $M_A \gg M_B \gg 1$로 두어 벌점에 순서를 만든다. 물리 초과를 가장 무겁게, 그다음 예약 침범을, 그 안에서 탄소와 지연을, 마지막으로 ε항의 이른 시작을 본다. 상한과 점유가 정수라 정수해에서는 z도 정수로 떨어진다. 이때 이 순서가 엄밀히 지켜지려면 M_B가 대기 작업의 실행 시간 합 $\Sigma_{j\in Q} d_j$와 ε항의 최댓값을 더한 것보다 커야 하고, M_A도 한 작업을 옮길 때 함께 바뀌는 예약 침범의 크기만큼 M_B보다 커야 한다. 실제로는 M_B를 시간 단위로 잰 가장 긴 실행 시간의 2배로 두되 10보다 작지 않게 했고(합성 워크로드 12, Azure

1,438.5), M_A는 그 100배로 두었다. 이는 순서를 보장하는 값이 아니라 실용적 선택이다. 합성 워크로드 주 실험의 최종 해에서는 두 초과 변수가 1년 8,760 슬롯 어디에서도 양수가 되지 않았고, 물리 초과 z^A는 Azure 대표 상한에서도 양수가 된 적이 없다(4.4절의 상한 0.5배에서 처음 양수). 예약을 둔 Azure(ρ=13)에서는 예약 침범 z^B가 무릎점에서 슬롯의 52.6%(4,605/8,761), α=1에서 90.5%(7,949/8,780)에서 양수다. 그래도 M_B를 0.1~10배로 바꿔 보면 절감은 46.14~46.22%로 같고 위반은 모두 0이어서, 결과가 벌점 크기에 매이지 않는다. 4.7절에서 보는 예약의 비용은 벌점이 아니라 예약 행, 곧 예약을 적은 제약식 자체에서 온다. 초과 벌점은 행마다 매기며 초과가 이어진 시간으로 가중하지 않는다. ε은 10^{-4}로 두어 대개 비용이 같은 후보 사이에서 이른 시작을 고르게 한다. 다만 분 단위 작업에서는 ε 항이 후보 사이 탄소 비용의 차보다 클 수 있다. 그래도 실행 10분 미만 작업은 작업시간의 1.4%, 배출의 1.6%라 결과에 주는 영향은 작다.

용량 행은 두 가지 해상도로 센다. 지금 슬롯만은 식 (11a)처럼 작업이 시작하는 순간마다 한 행(순간 행)을 두고, 그 순간에 돌고 있는 작업을 실제 시각 그대로 센다. 그래야 1초짜리 작업 수십 건이 한 시간 자리를 통째로 차지하는 가짜 포화가 생기지 않고, 4.1절의 위반 정의와 같은 기준으로 센다. 미래 슬롯은 슬롯마다 한 행을 둔다. 미래 계획은 모두 정수 슬롯 머리에서 시작하므로, 미래 슬롯($\sigma \geq t$+1)을 조금이라도 덮는 실행의 수는 그 슬롯 머리의 동시 실행 수와 같다. 그 대신 1시간 미만 작업도 미래 슬롯에서는 슬롯 하나를 통째로 쓴다. 이는 셈법이 아니라 시작 시각을 정시로만 둔 데서 오며, 그래서 식 (11b)의 z^A는 물리 초과와 같지 않다. 매 슬롯 다시 풀리므로 확정 시점에는 순간 행(식 (11a))으로 바로잡히지만, 그 전까지는 다른 작업의 선택에 영향을 준다. 도착 고정형도 확정 계획이 모두 슬롯 머리에서 시작하므로 슬롯 안 순간 최대로 센 판과 결과가 같았다(1년 실행에서 숫자까지 같았다). 채점 구간(결과를 세는 기간)에서 실행 시간이 1시간 미만인 작업 111,142건 가운데, 제출 슬롯보다 뒤 슬롯에 시작해 그 전까지 미래 슬롯 하나를 통째로 차지하는 계획에 올랐던 것은 13,034건(11.7%)이다. 주 실험(LSTM, 상한 17, 예약 3)의 1년 실행에서 슬롯마다 남긴 중간 계획으로 센 값이다.

식 (9)~(12)는 먼저 변수를 0~1 연속으로 두고 선형 계획으로 푼다. 용량 행과 작업별 배정 행이 함께 있어 제약 행렬이 완전 단모듈, 곧 선형 계획의 해가 저절로 정수가 되는 성질을 갖지 않으므로 정수해가 보장되지 않는다. 분수해가 난 슬롯은 3.9%였고(4.9절), 그 슬롯만 정수 계획으로 다시 푼다. 풀고 나면 i=0인 작업만 확정해 리전과 시작 시각을 실행 기록에 올린다. $i\geq$1로 배정된 작업은 계획일 뿐이며 다음 슬롯에 다시 Q에 들어간다. 그래서 어떤 미래 슬롯도 미리 잡아 두지 않지만, 지금 시작하는 긴 작업이 미래 슬롯을 덮는 것은 식 (11)(12)가 $n_A(\sigma)$로 센다.

상한 cap$_r$은 한 운영 주체가 리전 r에 확보한 전용 용량(쿼터나 클러스터 크기)이며 1년 내내 같다고 둔다. 이 값은 본래 설비 규모에서 와야 하지만 클라우드 사업자는 리전별 설비 규모를 공개하지 않는다. 그래서 상한을 하나의 상수로 유도하지 않고 실험 변수로 둔다. 상한을 재는 단위는 기준 피크, 곧 모든 작업을 홈 리전에서 즉시 실행했을 때의 리전별 최대 동시 실행 수다. 본문은 가장 큰 기준 피크의 1.25배를 올림해 모든 리전에 같게 준 값을 대표 상한으로 쓰고, 가장 큰 기준 피크의 0.5배에서 3배까지 바꾼 결과를 4.7절에 둔다.

예약 ρ_r도 자료에서 정한다. 미룰 수 없는 작업은 도착한 슬롯에 바로 돌아야 하므로 시작을 막거나 늦출 방법이 없다. 그런데 슬롯 σ가 시작될 때 이전 슬롯에 시작해 아직 돌고 있는 작업으로 상한이 차 있으면, σ에 도착하는 미룰 수 없는 작업은 들어갈 자리가 없다. 식 (12)는 미래 슬롯으로 넘어가는 미룰 수 있는 작업을 cap$_r$ − ρ_r 아래로 묶어 매 슬롯 시작에 ρ_r 자리를 남긴다. 지금 슬롯의 도착은 슬롯 시작 시점에 모두 알므로 $\sigma = t$에는 예약을 두지 않는다. ρ_r의 값은 채점 구간 앞의 준비 구간(1월 1~7일)에 홈 리전 즉시 실행으로 잰 미룰 수 없는 작업의 리전별 최대 동시 실행 수이며, 리전별 최댓값 가운데 가장 큰 값을 전 리전에 같게 쓴다. 합성 워크로드는 3이다. 같은 규칙으로 Azure를 재면 13이 나온다. 그러나 이 값에는 이전 슬롯에서 넘어와 아직 도는 긴 VM이 들어 있고, 이 VM은 식 (12)의 $n_r(\sigma)$에도 세어지므로 두 번 센 과대값이다. 또 이 규칙은 두 워크로드 모두 슬롯당 새 도착이 아니라 동시 실행을 잰다. 새 도착으로, 곧 준비 구간에 한 슬롯 동안 한 리전에 새로 도착하는 미룰 수 없는 작업 수로 재면, 슬롯·리전별 분포에서 Azure는 99 백분위수 5, 최댓값 7이고 합성은 6~8이다. Azure는 이중 계산을 피하려 예약 없이(ρ=0) 돌리고, Azure 상한 0.5배와 0.6배의 높은 부하도 예약 없이만 돌렸다. 위반을 0으로 지키는 데 예약이 필요 없었으므로(4.7절) 설계 비교는 두 워크로드 모두 예약 없이 한다. ρ_r을 0에서 6까지 바꾼 결과는 4.7절에 있다.

예약이 있어도 초과는 날 수 있다. 마감이 코앞인 작업이 상한보다 많이 몰리면 식 (11)의 z^A가 양수가 된다. 그래서 스케줄러가 하는 일은 용량을 강제하는 것이 아니라 초과에 가장 큰 벌점을 두어 줄이는 것이다. 다만 벌점 크기가 엄밀한 순서를 보장하는 충분조건보다

작으므로, 초과가 최소임을 보장하지는 않는다. 생긴 초과는 건수와 시간으로 모두 보고한다(4.4절). 실행 시간을 제출 때 정확히 안다는 가정 아래 스케줄러는 세 가지를 보장한다. 첫째, 후보 시작 시각이 애초에 $D_j - d_j$ 이하로만 잡히고 여유가 한 슬롯보다 적게 남으면 i=0만 허용되므로, 마감을 넘긴 배정은 탐색 공간에 없다. 둘째, 슬롯이 하나 지날 때마다 남은 여유는 정확히 1씩 줄고 처음 여유가 유한하므로 유한한 슬롯 안에 i=0만 남는다. 어떤 작업도 무한히 연기되지 않는다. 셋째, 초과 변수가 있어 식 (9)~(12)는 언제나 해를 가진다.

3.4 로드밸런서와 스케줄러의 결합

CAST가 제안하는 것은 매 슬롯 다시 푸는 배정 구조이고, 가중치 α는 그 구조를 어디에서 운용할지 정하는 값, 곧 운용점이다. α를 정하는 두 운용 규칙, 무릎점 규칙과 고정 α는 이 절 끝에 함께 둔다.

식 (9)~(12)는 리전과 시작 슬롯을 한 문제에서 정한다. 둘을 나누어 정하는 설계와 비교하면 이렇게 한 까닭이 드러난다. 작업이 도착할 때 리전을 정하고 시작 시각은 그 리전 안에서 나중에 고르며 용량은 입장 때만 확인하는 2단계 설계에서는, 각 작업이 상한을 보지 않고 제 창, 곧 시작할 수 있는 시각의 범위 안에서 예측 탄소가 가장 낮은 쪽의 시각을 고른다. 그 시각이 지금인 작업만 빈자리만큼 들어가고 나머지는 다음 슬롯에 다시 고른다. 같은 리전의 작업들은 같은 예측을 보므로 같은 골, 곧 제 창 안에서 예측 탄소집약도가 가장 낮은 구간을 고르고, 예측 곡선의 골이 뾰족할수록 그 시각에 들어가려는 줄이 길어진다. 줄에 선 채 여유가 다한 작업은 자리가 없어도 들어가므로 위반이 생긴다(4.4절). CAST에서는 용량이 입장 조건이 아니라 모든 대기 작업이 함께 보는 제약이어서, 한 골에 상한보다 많이 몰리는 배정은 식 (9)의 초과 항 때문에 애초에 비싸다.

남은 하나는 식 (7)의 가중치 α다. 왕복 지연은 왕복 지연 상한 안에서도 낮을수록 좋다. 원격 리전을 점유하는 동안 자료가 오가고 장애에 노출되므로, 지연 비용은 그 점유 시간에 비례하게 둔다. 옮긴 작업이 보내는 자료도 비용이 되어, 작업당 약 9~12 GB를 넘기면 전송 배출이 공간 이동의 이득을 지운다(4.3절). 그래서 탄소와 지연은 서로 다른 두 목적이고, α가 둘의 맞바꿈을 정한다. α=1이면 지연을 보지 않고 탄소만 줄이며, 본 자료에서는 작업시간의 99.9%가 프랑스로 가 지연이 121 ms가 된다(4.3절). α=0이면 홈 리전에서 즉시 실행하는 것과 같다. CAST는 가중치를 손으로 정하지 않는 기본값으로 α를 슬롯마다 고른다. 기준은 파레토 곡선이 가장 크게 꺾이는 점이라는 본래 뜻의 무릎점[29]이 아니라, 배출과 지연이 모두 가장 작은 이상점에 가장 가까운 후보, 곧 타협해[30]다. 이렇게 고른 점을 편의상 무릎점, 이 규칙을 무릎점 규칙이라 부른다. 구체적으로는 0부터 1까지 0.1 간격의 후보 11개, 곧 $\mathcal{A}$ = {0, 0.1, …, 1}의 각 α에 대해 식 (9)~(12)를 푼다. 그 배정마다 지연 $\bar{\lambda}_\alpha = \Sigma_j\, d_j\, \ell(o_j, r(j)) \,/\, \Sigma_j\, d_j$(실행 시간 가중 평균 왕복 지연)와 예측 배출량 $\bar{e}_\alpha$를 얻는다. 두 축을 후보들 사이에서 각각 0에서 1로 정규화하면 이상점이 원점에 오므로, 원점에 가장 가까운 후보의 α와 배정을 택한다.

$$\alpha^* = \operatorname*{argmin}_{\alpha \in \mathcal{A}} \sqrt{\tilde{\lambda}_\alpha^2 + \tilde{e}_\alpha^2} \tag{13}$$

여기서 $\tilde{\lambda}_\alpha$와 $\tilde{e}_\alpha$는 각각 0~1로 정규화한 평균 지연과 예측 배출이다. 이 규칙은 다중목적 최적화에서 이상점까지의 거리로 타협해를 고르는 방법이다. 물리 초과가 생기는 후보가 있으면 초과가 최소인 후보들 안에서만 고르고, 거리가 같으면 작은 α를 택한다. 무릎점은 정수 조건을 뺀 선형 계획 해(선형 계획 완화)의 (지연, 배출) 좌표로 고른다. 분수해가 난 슬롯만 고른 α에서 정수 계획으로 다시 풀므로, 그 슬롯들에서는 정수해의 좌표로 골랐을 때의 무릎점과 다를 수 있다. 홈 리전에 고정된 경우처럼 모든 후보의 지연이 같거나 배출이 같아 한 축이 한 점으로 줄면 α=1로 둔다. 배출 축만 한 점으로 줄 때도 α=1을 두므로, 그 슬롯에서는 지연이 가장 짧은 배정을 고르지 못할 수 있다. 대표 상한 17에서 37일 893 슬롯을 세어 보니 그런 슬롯은 없었다. 두 축을 모두 실행 시간으로 가중하는 것은 목적함수 식 (1)(2)와 같은 척도로 재기 위해서다. 건수로 평균하면 1초짜리 작업 한 건이 6시간짜리 작업 한 건과 같은 표를 갖는다.

리전과 시각을 한 문제로 둔 것이 배정을 어떻게 바꾸는지는 한 슬롯을 열어 보면 분명하다. 그림 4의 슬롯에서 한국에서 제출된 작업 하나를 보면, α를 0에서 1까지 옮길 때 배정이 다섯 구간으로 갈린다(표 2). α가 0이면 한국에서 지금 시작하고(352 gCO_2/kWh), 0.1~0.3이면 한국에 남되 6~7시간 뒤로 미뤄 329~330으로 낮춘다. 0.4~0.5에서는 왕복 지연 30 ms를 들여 일본으로 옮기고 7시간 뒤에 시작해 204로, 0.6~0.9에서는 캘리포니아로 옮겨 2~3시간 뒤에 58~80으로, 1.0에서는 프랑스에서 즉시 시작해 12로 간다. 리전만 바뀌는 것이 아니라 리전마다 고르는 시작 시각도 다르며, α 하나가 로드밸런서와 스케줄러의 선택을 함께 움직인다. 무릎점은 이 슬롯에서 0.5를 골라, 이 작업은 왕복 지연 30 ms를 들여 예측 탄소집약도를 한국에서 지금 시작할 때보다 42% 낮춘다.

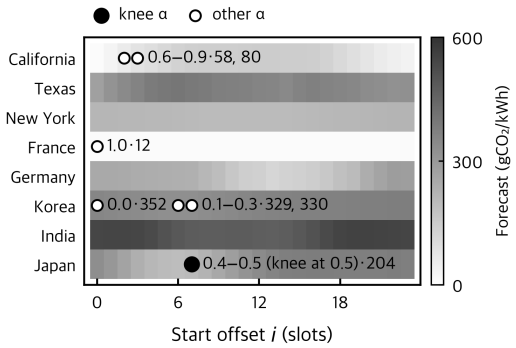


그림 4 한 슬롯에서 α에 따른 배정. 2025년 5월 26일 19시 48분(UTC)에 한국에서 제출된 k=1 작업 하나(실행 17분, 마감까지 14.5시간)이며, 칸은 리전 × 향후 24 슬롯이고 진할수록 예측 탄소집약도가 높다. ●는 무릎점(α=0.5), ○는 다른 α가 고른 칸이며, 옆 숫자는 고른 α의 범위 · 그 칸의 예측 탄소집약도(gCO₂/kWh)다.

Fig. 4 Assignment by α at one slot. One k=1 job submitted in Korea at 19:48 UTC on 26 May 2025 (17 min run, 14.5 h to deadline); cells are region × next 24 slots, darker for higher forecast carbon intensity. The filled dot is the cell chosen at the knee (α=0.5), hollow dots those chosen at other α, labelled with the range of α that chose them, followed after · by the cell's forecast intensity (gCO₂/kWh).

표 2 α에 따른 배정 변화. 그림 4와 같은 작업(홈 리전 한국)이며, 시작은 지금부터 몇 슬롯 뒤인지, 탄소는 예측 탄소집약도(gCO₂/kWh), 지연은 ms다.

Table 2 Assignment change by weight α. Same slot and job as Fig. 4 (origin: Korea); start is in slots from now, carbon is forecast intensity in gCO₂/kWh, latency in ms.

α	Region	Start	Carbon	Latency
0.0	Korea	0	352	0
0.1–0.3	Korea	6–7	329–330	0
0.4–0.5	Japan	7	204	30
0.6–0.9	California	2–3	58–80	134
1.0	France	0	12	238

이 작업 하나만 보면 α가 커질수록 탄소가 단조롭게 줄지는 않는다. 같은 슬롯의 다른 작업 29건과 캘리포니아의 자리를 나눠 쓰기 때문이다. α=0.7에서는 캘리포니아의 t+2 자리가 다른 작업 여섯 건에 배정되어 이 작업이 t+3으로 밀렸다. 작업마다 따로 골랐다면 이 작업은 예측 탄소집약도가 더 낮은 t+2(58)를 골랐을 것이다(t+3은 80).

무릎점 규칙은 가중치 값 대신 후보 α의 격자, 정규화, 거리 규칙을 정해 두고 슬롯마다 α를 고른다. 그래서 무릎점은 곡선의 모양에 따라 움직인다. 예측된 탄소 곡선의 골이 뾰족할수록 탄소 축의 정규화가 달라져, 같은 슬롯이라도 더 낮은 α, 곧 지연을 더 무겁게 보는 쪽을 고른다(4.5절). 슬롯들이 실제로 고른 α의 분포는 4.5절에 있다.

운용 규칙은 쓸 수 있는 자료에 따라 고른다. 자료가 없으면 무릎점 규칙을 쓴다. 지연 목표를 미리 정하지 않아도 첫 슬롯부터 α를 정해 주기 때문이다. 첫 주 자료가 있으면 그 주에서 무릎점 규칙과 같은 지연이 나오도록 고른 고정 α(0.5687)를 쓰며, 이것이 무릎점 규칙보다 약 0.9%포인트 더 줄인다(4.5절). 예약은 선택이며 시험한 부하에서는 필요 없었다(4.7절). 무릎점은 배출이 가장 낮은 지점이 아니라 지연도 함께 줄이는 쪽의 한 점이다. 같은 지연 짝, 곧 지연이 같은 고정 α와 견주면 합성 워크로드의 아홉 조건 가운데 여덟에서 더 배출하고 대기 시간은 짧다. 또 슬롯마다 선형 계획을 열한 번 푼다(4.5절, 4.9절). 정규화가 슬롯마다 달라 α 하나가 (P0)의 지연 예산 $\bar{\Lambda}$ 하나와 1:1로 대응하지 않으므로, 4장은 같은 지연 짝으로 비교한다. 알고리즘 1은 세 모듈이 한 슬롯에서 하는 일을 차례로 적은 것이다.

비교를 위해 여덟 가지를 함께 잰다. 첫째는 작업이 도착하는 슬롯에서 로드밸런서가 리전을 정해 고정하고, 그 뒤로는 스케줄러가 시작 슬롯만 같은 문제로 고르게 한 것이다. 비용, 상한, 예약, 예측, 가중치 규칙은 모두 같고 리전을 정하는 시점만 다르며, 4장에서는

알고리즘 1 결합 롤링 배정

Algorithm 1 Joint rolling assignment with the knee rule (in the realistic-information setting of Section 4.4, non-deferrable jobs are instead assigned greedily at their arrival instant)

Input: cap$_r$, ρ_r, H, M_A, M_B, ε, $\mathcal{A}$ = {0, 0.1, …, 1}, forecasts $\hat{C}_r^{(t)}(\cdot)$, job stream
Output: ($r(j)$, τ_j) for all j

$Q \leftarrow \emptyset$
for $t \leftarrow$ 0, 1, 2, … do
  for each j with $\lfloor s_j \rfloor = t$ do $Q \leftarrow Q \cup \{j\}$
  for each $j \in Q$ do
    $h_j \leftarrow \lfloor \min(D_j - d_j,\ t + H) - t \rfloor$
    if j is non-deferrable or $(D_j - d_j) - t < 1$ then $h_j \leftarrow 0$
  for each $\alpha \in \{0, 0.1, \ldots, 1\}$ do
    $x^\alpha \leftarrow$ LP solution of (9)–(12)
    $(\bar{\lambda}_\alpha, \bar{e}_\alpha) \leftarrow$ duration-weighted mean latency and forecast emissions of x^α
  $\alpha^* \leftarrow$ knee point by (13) among the candidates with minimum physical excess, tie rules of Sec. 3.4; $x \leftarrow x^{\alpha^*}$
  if x is fractional then $x \leftarrow$ MILP solution of (9)–(12) at α^* (HiGHS, relative gap 0.01%, no time limit; at most 1.22 s (synthetic) and 1.52 s (Azure) per slot in the main runs; Section 4.9)
  for each (j, r) with $j \in Q$ and $x_{jr0} = 1$ do
    $r(j) \leftarrow r$; $\tau_j \leftarrow \max(t, s_j)$; record $[\tau_j, \tau_j + d_j)$ in the execution record of r; $Q \leftarrow Q \setminus \{j\}$

표 3 대조군의 이름과 정의. 2단계와 탐욕을 빼면 모든 대조군이 CAST와 같은 선형 계획, 예측, 비용, 상한, 무릎점 규칙을 쓰고 용량을 확인하는 방식이나 리전을 정하는 시점만 다르다(탐욕은 선형 계획 대신 LLF 순서로 자리를 잡는다). 본문과 표, 그림은 이 이름을 쓴다.

Table 3 Names and definitions of the control designs. Except for the two-stage and greedy designs (greedy uses LLF ordering instead of the linear program), all share CAST's linear program, forecasts, costs, ceilings and knee rule and differ only in how capacity is checked or when regions are fixed. Text, tables and figures use these names.

Name (Korean)	Name in tables	Fixed on arrival	Capacity checked against	Re-solved
분리형	Decoupled	Region	All plans in the same LP	Start slot, every slot
용량 무시	Capacity ignored	—	Nothing (rows (11)(12) removed)	Every slot
탐욕	Greedy	—	All plans, job by job in LLF order	Every slot
2단계	Two-stage	Region, by a separate balancer	Internal ceiling at admission	No
도착 고정형	Arrival-fixed	Region and start	All committed plans	No
시작한 작업만 확인형	Started-only	Region and start	Started jobs only	No
입장 통제형	Admission control	Region and start	Started jobs, re-checked at start	No; waits a slot if full
대피형	Evacuation	Region and start	Started jobs, re-checked at start	No; moves to a free region if full
미래 행 제거형	No future rows	—	Current-slot instants only	Every slot

분리형이라 부른다. 둘째는 같은 배정에서 식 (11)(12)를 지운 것으로, 저탄소 시각에 몰려도 아무것도 막지 않는다. 용량을 지운 기준점이며 용량 무시라 부른다. 셋째는 같은 비용, 상한, 무릎점 규칙을 쓰되 선형 계획 대신 남은 여유가 적은 작업부터(LLF, least laxity first) 각자 가장 싼 (리전, 시작 슬롯)을 차례로 잡게 한 것으로, 탐욕(greedy)이라 부른다. 넷째는 위에서 말한 2단계 설계로, 예측, 물리 상한, 한 시간 앞 정보, 채점은 CAST와 같고 나머지가 다르다. 리전은 작업이 도착할 때 슬롯 단위 정수 계획(가중치는 슬롯마다 무릎점)으로 정해 고정한다. 이 리전 배정은 본 연구가 따로 구현한 2단계 대조군의 로드밸런서(리전당 상한 $12 = \lfloor 0.8 \times 16 \rfloor$, 16은 가장 큰 기준 피크 13의 약 1.2배)가 LSTM 예측으로 한 번 계산한 것이며, 예측기나 내부 상한(이 대조군이 입장 때 쓰는 자체 상한)을 바꾼 실행에서도 이 배정 하나를 그대로 쓴다(그래서 평균 지연이 모두 같다). 시작 시각은 그 리전 안에서 작업마다 따로 고른다. 후보 창의 예측 탄소를 최솟값–최댓값으로 0~1로 바꾼 값과 창 길이 대비 미룸의 비율을 $(6-k)/5$ 대 $1-(6-k)/5$로 더한 점수가 가장 낮은 시각이며, 첫 시각은 예측 대신 그 시각의 실측을 쓴다. 그 시각이 지금인 작업만 남은 여유가 적은 순으로 내부 상한까지 들여보내고, 여유가 다한 작업과 미룰 수 없는 작업은 자리가 없어도 들여보낸다. 다시 풀지 않고 예약도 두지 않으며, PuLP와 CBC로 푼다. 용량을 확인하는 시점 말고도 다른 점이 많아, 확인 시점만 다른 대조군은 아래 넷으로 따로 만든다. 다섯째는 Zanotto 등[9]처럼 작업이 도착한 슬롯에서 이미 정한 배정을 모두 보고 리전과 시작 슬롯을 한 번 정한 뒤 다시 풀지 않는 것으로, 도착 고정형(Arrival-fixed)이라 부른다. 원 설계보다 불리하지 않도록 한 슬롯에 도착한 작업을 묶어 CAST와 같은 무릎점 선형 계획으로 정한다. 여섯째는 도착 고정형과 같되 미래 슬롯의 용량을 이미 시작한 작업만 보고 확인하는 것으로, 시작한 작업만 확인형이라 부른다. 확정했지만 아직 시작하지 않은 다른 작업의 계획을 보지 않는다는 한 줄만 도착 고정형과 다르다. 일곱째는 시작한 작업만 확인형에 시작 때의 확인을 더한 것으로, 정한 시각이 오면 그 순간의 실행 기록으로 다시 확인해 자리가 있으면 시작하고 없으면 한 슬롯 미루며, 여유가 다하면 자리가 없어도 들인다. 7a 입장 통제형이라 부른다. 7a와 같되 여유가 다해 들여야 할 때 정한 리전이 꽉 찼으면 그 순간 자리가 있는 리전 가운데 예측 탄소가 가장 낮은 곳으로 보내는 변형은 7b 대피형이라 부른다. 7a와 7b는 시작 때 다시 확인하는 설계다. 여덟째는 CAST에서 미래 슬롯 행(식 (11b)(12))만 지운 것으로, 미래 행 제거형이라 부른다. 이 아홉 대조군을 표 3에 모았다.

3.5 탄소 회계

작업 j의 배출량은 실행 리전의 실측 탄소집약도를 실행 구간에 대해 적분한 값에 소비 전력을 곱한 값이며, 그 합이 식 (1)이다. 실측 계열이 1시간 해상도이므로 적분은 슬롯 단위로 근사한다. 실행이 걸치는 슬롯들의 평균 탄소집약도에 실행 시간을 곱한다.

$$E_j \approx Pd_j \frac{1}{|T_j|} \sum_{\sigma \in T_j} C_{r(j)}(\sigma) \tag{14}$$

여기서 T_j는 시작 슬롯 $\lfloor \tau_j \rfloor$부터 $\lceil d_j \rceil$개 슬롯의 집합이다. 슬롯마다 실행과 겹친 길이로 가중해 정확히 적분해도 절감은 무릎점 규칙 71.93%에서 71.80%로, α=1은 95.65%에서 95.64%로 움직일 뿐이다. 소비 전력은 작업당 1 kW 상수로 두고 유휴 전력·냉방 전력과 장비 제조에서 나온 내재 배출은 넣지 않으므로, 절감률의 분모는 작업이 실행 중에 쓰는 전력의 배출이다. 상대 비교를 위한 단순화이며, 그 덕에 배출량 차이는 전적으로 실행 리전과 시각에서 온다. 4장의 모든 정책과 대조군은 이 회계 하나로 채점한다.

4. 실험과 분석

4.1 실험 설정

평가는 8개 리전의 2025년 탄소집약도 1년치 위에서 수행한다. 이 값은 Electricity Maps가 발전원 구성과 배출계수로 계산해 사후에 확정한 CO_2 환산량(gCO_2eq/kWh)이며, 이하 예측과 구별해 실측이라 부르고 단위는 gCO_2/kWh로 적는다. 탄소집약도는 Electricity Maps API를 통해 1시간 해상도로 수집하였다[31]. Electricity Maps의 값은 수입과 수출을 따라간 소비 기준이며[32], 받은 파일에는 직접 배출과 전과정 가운데 어느 기준인지, 추정값인지, API 판과 받은 날짜가 남아 있지 않고 수집 스크립트도 없어 다시 받아 대조하지 못했다. 프랑스의 최솟값 2 gCO_2/kWh는 직접 배출 기준과 맞으나, 받은 값이 전과정 기준이라면 본 결과가 곧 그 기준의 값이다.

회계를 바꾸면 절감의 크기는 크게 달라지고, 한계 근사(프랑스만)에서는 합성의 설계 비교 순서까지 뒤집힌다. 먼저 무탄소 전원에 전과정 배출(원자력 12, 재생 30 gCO_2/kWh)을 더한 근사로 다시 채점한다. 화석 상류 배출은 넣지 않은 한쪽 근사이며, 배정은 그대로 둔다. 이때 CAST의 절감은 3.2~5.4%포인트 준다(무릎점 71.93%가 67.07%, α=1 95.65%가 90.28%, Azure 무릎점 51.35%가 48.13%). 그래도 설계의 순서와 용량 비용(용량 무시와의 절감 차)의 부호는 같다. 평균 배출계수로 센 결과는 프랑스가 받은 부하가 깨끗하게 공급된다는 가정에 기댄다. 프랑스로 옮긴 작업만 같은 시각 독일의 평균 탄소집약도로 세는 한계 근사(프랑스만)에서는 CAST 무릎점이 38.99%, α=1이 18.11%(홈 리전에서 시각만 옮기는 시간만은 18.78%), Azure 무릎점이 22.37%로 준다(Azure는 예약 없이). 이 근사로 4.4절의 같은 지연 짝(지연을 맞춘 두 실행의 짝으로, 이 절 뒤에서 정의한다)을 다시 세면 합성에서는 순서가 뒤집힌다. 도착 고정형이 CAST보다 0.89%포인트 앞서고(CAST 37.59% 대 38.49%), 대피형과의 짝에서도 CAST가 2.15%포인트 뒤진다. Azure에서는 CAST가 도착 고정형보다 1.48%포인트 앞서고 대피형과는 차이가 없다(회계별 짝은 4.7절). 이 근사는 프랑스로 간 몫만 바꾼 것이다. 대칭 한계 근사는 출발지, 목적지, 기준의 탄소집약도를 모두 $C' = C / \max(1 - f, 0.05)$로 바꿔 센다($f$는 무탄소 몫, 0.05는 발산을 막으려 임의로 둔 하한). 이 근사에서 무릎점 절감은 36.15%, Azure 무릎점은 28.96%다. 4.4절의 같은 지연 짝으로는 CAST가 도착 고정형보다 1.19%포인트 앞서(34.81% 대 33.62%) 평균 배출계수일 때와 순서가 같고, 하한을 0.02, 0.1, 0.2로 바꾸어도 0.8~1.3%포인트 앞선다. 다만 값은 하한에 크게 좌우된다. 프랑스의 등가 탄소집약도 C'는 채점 구간(지표를 세는 기간) 평균 약 220 gCO_2/kWh로 가스 발전보다 낮고, 하한이 걸리는 시간은 76%다. 같은 짝의 CAST 절감은 하한에 따라 25~52%로 움직이며, 시간만의 절감은 0.42%로 거의 사라진다. 하한이 걸리는 시각에 하한 대신 그 리전의 하한 밖 시각 중앙값(프랑스 약 400 gCO_2/kWh, 가스 발전 수준)을 쓰면 무릎점 절감은 21.20%(Azure 17.93%)다. 이때 같은 지연 짝의 차는 합성에서 도착 고정형과 0.47%포인트, 대피형과 0.42%포인트, Azure에서 1.38과 0.08%포인트로, 순서는 같으나 작아진다. 두 한계 근사 모두 프랑스로 간 작업을 실제보다 깨끗하게 세는 쪽이라 공간 이동에 유리하다. 리전 설비 효율(PUE)을 1.1~1.6으로 두고 작업이 몰리는 프랑스와 캘리포니아에 1.6, 나머지에 1.1을 곱하면(기준 배출도 같이) 예약 없는 무릎점 절감이 72.06%에서 68.66%로 준다. 거꾸로 두면 74.54%로 는다. 4.4절의 같은 지연 짝 차는 각각 1.07%포인트와 0.89%포인트로 순서가 같다. 위반과 필요 용량(위반 없이 돌기 위한 상한)은 회계와 무관해 바뀌지 않는다. 평균 배출계수, 정확 겹침(실행이 슬롯과 겹친 길이로 가중한 적분), 전송 에너지를 목적지로 센 것과 출발지로 센 것, 전과정, 한계 근사 둘을 일곱 회계라 부른다. 모두 평균 배출계수로 정한 배정을 그대로 두고 기준 배출까지 다시 센 것이다. 4.7절의 회계별 짝 표는 이 일곱에 대칭 한계 근사의 분모 하한 셋, PUE를 둔 두 방식, 불리한 가정을 겹친 경우를 더한다.

리전과 Electricity Maps 구역, 지연 행렬의 Azure 리전은 표 4와 같다. 미국 동부 리전은 탄소 자료로 뉴욕 전력망(NYISO) 구역을, 지연 자료로 Azure East US 행을 쓰며, 이후 뉴욕이라 부른다. 뉴욕에는 Azure 리전이 없어 버지니아의 East US를 썼으므로, 이 리전은 지연과 탄소가 서로 다른 곳(PJM과 NYISO)에서 온 혼합이다. LSTM의 학습(2021~2023년), 검증(2024년), 시험(2025년) 자료는 같은 수집 파일 묶음이며, 스케줄러가 쓰는 1월 8일부터의 2025년 실측은 시험 파일과 리전별 평균 절대

표 4 리전과 자료 구역. 뉴욕은 탄소를 뉴욕 전력망에서, 지연을 버지니아의 East US에서 가져온 혼합이다. 일본과 인도는 나라 단위 구역이라 데이터센터가 있는 하위 계통과 다를 수 있다.

Table 4 Regions and data zones. New York mixes carbon from the New York grid with latency from East US (Virginia); Japan and India are country-level zones and may differ from the sub-grids hosting the data centers.

Region	Electricity Maps zone	Azure region (RTT)
California	US-CAL-CISO	West US
Texas	US-TEX-ERCO	South Central US
New York	US-NY-NYIS	East US
France	FR	France Central
Germany	DE	Germany West Central
Korea	KR	Korea Central
Japan	JP	Japan East
India	IN	Central India

표 5 미룸 등급별 미룸 허용치와 실행 시간. 허용치는 등급 안에서 로그균등, 실행 시간은 로그정규에 상·하한을 두어 뽑는다. 중앙값과 상·하한은 초 단위, s.d. (log)는 로그정규의 로그 표준편차이며 중앙값과 함께 생성 모수이고 건수는 1년 전체다.

Table 5 Deferral allowance and job duration by tier. Allowances are log-uniform within each tier; durations are log-normal and clamped (seconds); the median and s.d. (log), the standard deviation of the log, are generation parameters. Counts cover the full year.

k	Deferral allowance	Median d	s.d. (log)	Clamp	Jobs
1	12–24 h	5,400	0.9	300–21,600	43,830
2	2–8 h	1,800	0.9	60–14,400	14,458
3	2–10 min	180	0.8	5–1,800	14,716
4	15–60 s	30	0.7	1–300	29,244
5	1–5 s	5	0.6	1–60	43,752

차 0.27 gCO_2/kWh 안에서 같다. LSTM이 168시간을 입력받아 2025년 예측이 1월 8일 0시부터 나오므로, 1월 1~7일은 1월 8일의 하루 모양을 되풀이해 채운 준비 구간이고 그 기간에는 예측과 실측이 같다. 2024년 말 자료로 입력을 채우면 학습과 검증의 분할이 섞이므로 그렇게 하지 않았다. 2025년 12월 31일 1~23시는 자료가 비어 있어 채운 값이다. 시뮬레이션은 1월 1일부터 돌리되 모든 지표는 1월 8일 0시부터 연말까지 8,592시간(이하 채점 구간)에 대해 세고, 그 전에 제출된 작업은 채점에서 뺀다.

네트워크 지연은 8×8 리전 간 왕복 지연 실측 행렬(최댓값 244 ms, 비대각 최솟값 12 ms)을 Azure 통계[33]에서 가져왔다. 전 구간에서 고정이며, 부하에 따라 지연이 바뀌는 효과는 모델에 없다.

워크로드는 둘이다. 합성 워크로드는 총 146,000건이며 리전당 하루 50건이 제출되고, 채점 구간에 들어오는 것은 143,200건이다. 등급별 비율(k=1부터 3:1:1:2:3)은 공개 트레이스에 미룸 등급 표시가 거의 없어(Azure 트레이스는 0.5%) 정한 합성 파라미터다. 미룸 허용치는 등급 안에서 로그균등으로, 실행 시간은 로그정규 분포에 상·하한을 두어 뽑는다(표 5). 도착에는 하루 주기가 있다. k=3~5는 현지 13시, k=1~2는 현지 2시를 봉우리로 하는 표준편차 3시간의 정규분포에서 70%가 오고 나머지 30%는 봉우리 반대쪽 12시간에 고르게 온다(봉우리를 한낮으로 옮긴 결과는 4.7절). 현지 시각은 일광절약시간 없는 고정 시차로 둔다. 미룰 수 없는 작업이 건수로 59.1%지만 작업시간으로는 1.3%라, 시간 이동의 여지는 작업시간 기준으로 거의 전부에 있다. 공간 이동은 식 (5)가 걸리지 않아 모든 작업에 열려 있다.

두 번째 워크로드는 Azure의 2019년 VM 트레이스[34, 35]에서 만들었다. 29일씩 13장을 이어 붙여 1년을 만들되 장마다 다른 VM 11,600개를 뽑았고(추출률 0.49%), 마지막 장은 연말에서 잘렸다. 전체는 146,016건이다. 실행 시간은 VM이 생성되어 삭제될 때까지의 간격 그대로이고(대부분 300초 단위, 5.9%는 150초), VM 크기는 쓰지 않고 모든 VM을 같은 전력과 같은 용량 한 칸으로 셌으므로 Azure의 숫자는 VM 시간으로 가중한 값이다. 도착 시각은 트레이스의 하루 모양을 그대로 따르되, 그 VM의 홈 리전(아래에서 무작위로 준다)의 현지 시각이라고 가정해 일광절약시간 없는 고정 시차로 UTC로 옮겼다. 기준 배출은 213,513.4 kg으로 합성의 7.5배이고, 이 워크로드의 %포인트는 이 값에 대한 차다. 트레이스에 없는 두 값은 모형으로 채웠다. 홈 리전은 여덟 리전에 고르게 무작위로 준다. 미룸 등급은 트레이스의 미룸 등급 표시(0.52%, 그 가운데 대화형 0.17%)만 따르고, 나머지는 합성 워크로드와 같은 비율로 준다. 도착과 실행 시간은 실측이고 미룸 허용치는 모형이다. 시각 값은 그 한 시간의 시작을 뜻하고(UTC, 2025년 1월 1일 0시가 t=0), 연말을 넘는 작업과 지평에는 마지막 시각의 값을 이어 쓴다.

두 워크로드는 실행 시간에서 서로 다른 끝을 덮는다. 실행 시간의 중앙값은 합성 81초, Azure 1,500초이고, 95백분위수는 1년 전체 작업 기준으로 합성 13,074초, Azure 74,400초(20.7시간)다. 합성은 VM 트레이스가 300초 해상도라 보지 못하는 짧은 작업을, Azure는 하루를 넘는 긴 작업을 담는다. 총 작업시간은 Azure가 합성의 7.8배다. 평균 동시 실행은 여덟 리전 합으로 합성 11.5건, Azure 89.9건이다. 이를 총 상한으로 나눈 값, 곧 1년 동안 모든 작업의 실행 시간 합을 (상한 × 리전 수 ×

8,760시간)으로 나눈 값을 이용률이라 한다. 이용률은 대표 상한(아래 문단에서 정한다)에서 합성 8.4%, Azure 26.1%다(1년 전체 기준). 미룰 수 없는 작업이 차지하는 작업시간 비중은 합성 1.3%, Azure 51%로 정반대다.

등급 비율 3:1:1:2:3은 두 원칙을 따랐다. 첫째, 시간 이동이 가장 잘 듣는 등급 1과 지연에 가장 민감한 등급 5를 같은 30%로 두었다. 다만 주 실험은 왕복 지연 상한을 두지 않아 공간 이동이 모든 작업에 열려 있으므로, 이 워크로드는 공간 축에 유리하다(4.3절의 공간만 61.57% 대 4.4절의 시간만 18.78%). 네트워크 지연에 상한을 두어 공간 이동을 막았을 때의 결과는 4.7절에 있다. 둘째, 1초에서 하루까지 다섯 자릿수에 걸친 미룸 허용치의 어느 구간도 비지 않도록 가운데 세 등급을 1:1:2로 얇게 두었다. 그 결과 합성 워크로드는 건수로는 짧은 작업이, 작업시간으로는 긴 작업이 대부분인 꼴을 갖는다. 15초 미만 작업은 건수의 32.2%지만 작업시간의 0.1%이고, 1시간 이상 작업은 건수의 22.4%로 작업시간의 85.2%를 차지한다. Google 트레이스에서도 대다수 작업이 몇 분 안에 끝나지만 전체 기간 내내 도는 작업이 2,000개를 넘고 전체의 7%인 생산 우선순위 작업이 나머지 전부보다 많은 자원을 쓴다[36]. 미룰 수 있는 작업의 비율을 20~60%로 바꾼 결과는 4.7절에 있다.

상한은 3.3절대로 실험 변수다. 기준 피크는 채점 구간 작업을 홈 리전에서 즉시 돌렸을 때의 리전별 최대 동시 실행으로, 합성 워크로드에서 캘리포니아 11, 텍사스 10, 뉴욕 10, 프랑스 10, 독일 11, 한국 12, 인도 13, 일본 12이고, Azure 워크로드에서는 가장 큰 것이 34다. 상한 자체는 가장 큰 기준 피크의 0.5~3배(Azure는 0.5~2배)로 바꿔 본다(4.4절, 4.7절). 대표 상한은 가장 큰 기준 피크의 1.25배를 올림한 값으로 합성 17, Azure 43이며, 모든 리전에 같게 주므로 리전마다 자기 피크의 1.3~1.7배가 된다. 어떤 정책이든 채점은 이 물리 상한으로 한다. CAST가 안에서 쓰는 예약 ρ는 채점 기준이 아니다.

지표는 다음과 같다. 총 배출량은 식 (1)을 3.5절의 회계로 잰 값이고, 절감률은 기준(홈 리전 즉시 실행)의 배출에 대해 줄어든 백분율이다. 지연에 관한 값은 네 가지로 갈라 쓴다. 왕복 지연은 홈 리전과 실행 리전 사이의 왕복 시간이다. 실행 시간 가중 지연은 그것을 실행 시간으로 가중한 평균으로, 목적식과 같은 척도이고 같은 지연 짝의 기준이며, 이하 따로 밝히지 않은 지연은 이것이다. 작업당 지연은 작업 수로 낸 평균이고, 대기 시간은 제출에서 시작까지의 시간(3장의 미룸)이다. 상한 위반은 상한을 넘긴 배정 건수다. 작업 j가 시작하는 순간 같은 리전에서 이미 돌고 있는 작업이 상한 이상이면 j를 위반으로 센다. 같은 시각에 시작한 작업은 서로를 보고, 같은 시각에 끝나는 작업은 먼저 뺀다. 실제 클러스터에서는 상한을 넘겨 실행할 수 없으므로 위반은 대기 연장, 마감 놓침, 추가 용량 빌림 가운데 하나로 나타나며, 본 연구는 그 뒤처리를 모형에 넣지 않고 건수로만 센다.

강제 시작은 미룰 수 있는 작업 가운데 여유(마감에 맞추려면 시작해야 하는 마지막 시각까지 남은 시간)가 한 슬롯 안으로 줄어 지금 시작해야 하는 작업 수다. 초과 시간은 리전별로 동시 실행이 상한을 넘은 시간의 합이다. 필요 용량은 그 정책의 배정이 만든 리전별 최대 동시 실행 수의 최댓값으로, 그 정책이 위반 없이 돌려면 필요한 상한이다. 채점 상한과 무관한 값이라 선행 정책 비교의 자로 쓰며, 1초짜리 순간 피크의 영향을 가리기 위해 시간 가중 99.9 백분위수를 함께 적는다. 이 밖에 마감 위반 건수, 공간 이동이 더한 왕복 지연의 평균과 95 백분위수, 제출에서 시작까지 미룬 시간을 잰다.

예측 곡선의 골의 깊이는 예측의 하루 최대−최소를 실측의 하루 최대−최소로 나눈 값(납작함)으로 잰다. 몰림, 곧 여러 작업이 같은 깨끗한 시각을 함께 노리는 정도는 세 값으로 설명한다. 리전-일마다 실측 탄소가 가장 낮은 1시간의 최대 동시 실행을 평균한 값, 그 시각에 동시 실행이 상한에 닿은 리전-일의 비율, 미룬 작업 가운데 그날 탄소가 가장 낮은 세 시각에 시작한 작업의 비율이다. 이 셋은 몰림의 모양을 보일 뿐이며, 몰림이 해로운지는 절감과 위반으로 가린다.

두 실행의 비교에는 같은 지연 짝을 쓴다. 같은 지연 짝은 비교 대상과 지연이 0.6 ms 안에서 같아지는 고정 α를 이분법으로 찾아 1년 전체를 돌린 실행을 비교 대상과 묶은 것이다. 설계 비교(4.4절)는 두 워크로드 모두 예약 없이 한다. 위반 0에는 예약이 필요 없고, 주 합성 워크로드에서는 예약을 빼면 두 설계 모두 조금 나아지기 때문이다(같은 지연에서 CAST 0.03, 도착 고정형 0.27%포인트). 두 실행의 차는 무릎점이 더 배출한 양으로 적는다. 판정에는 7일 묶음을 되뽑아 2,000번 다시 계산한 블록 부트스트랩의 95% 신뢰구간을 쓴다. 신뢰구간이 0보다 위에 있으면 무릎점이 뒤진다고, 0을 포함하면 차이가 없다고, 0보다 아래에 있으면 이긴다고 판정한다. 이 신뢰구간은 한 실행 안의 주 단위 변동을 잰 것이며, 워크로드를 다시 뽑는 변동은 시드를 바꾼 다섯 벌의 실행으로 따로 잰다(4.7절). %포인트 차는 반올림 전 kg 차를 기준 배출(합성 28,625.5 kg, Azure 213,513.4 kg)로 나눈 값이라, 반올림한 절감률끼리 뺀 값과 0.01 다를 수 있다.

실험은 Python 3.14.7과 SciPy 1.18.0의 HiGHS 풀이기로 Apple M1 Max(성능 코어 8, 효율 코어 2, 메모리 32 GB, macOS 15.7.7)에서 돌렸다. 4.4절의 2단계 설계는 PuLP 3.3.2와 CBC 2.10.3으로 같은 기계에서 돌렸다. 모든

표의 숫자는 작업별 기록에서 별도 스크립트로 다시 계산해 대조했다.

본 연구의 코드, 합성 워크로드 생성기, Azure 트레이스 변환 스크립트, 실행 설정은 심사용 익명 저장소(투고 때 기재)에 공개한다. 탄소집약도 자료는 Electricity Maps 이용 약관에 따라 재배포하지 않고 전처리·학습 코드만 공개하며, 수집 스크립트는 남아 있지 않다.

4.2 예측기 평가

예측기는 값을 얼마나 맞히는가와 그 예측이 배정을 얼마나 잘 이끄는가로 나누어 본다. 값만 보면 LSTM의 오차는 가장 나은 단순 예측과 비슷하다. 2025년 시험 연도 전 지평의 평균 절대 오차는 LSTM이 여덟 리전 평균 26.10 gCO_2/kWh로, 직전값을 그대로 쓰는 예측(48.41)과 168시간 전 값을 쓰는 예측(35.46)보다 낮고 24시간 전 값을 쓰는 예측(SN-24h, 25.27)과는 비슷하다. LSTM이 이 예측보다 조금 뒤지는 차는 인도 하나에서 온다. 인도의 하루 변동(최댓값−최솟값) 월평균이 2024년 11월부터 거의 두 배가 되어 학습 연도와 하루 모양이 달라졌고(인도만 LSTM 30.54 대 11.48), 인도를 빼면 LSTM이 25.47로 24시간 전 값(27.24)보다 낮다. 이 변화는 망의 변화라기보다 자료 산정 방식의 변화로 보인다(추정, 일본도 2024년 1월에 비슷한 변화가 보인다). 연평균이 학습 연도에서 시험 연도로 갈수록 크게 내려간 리전이 많아(프랑스 41.3, 14.8, 13.6 gCO_2/kWh, 마지막은 채점 구간) 학습과 시험의 수준 차이가 크며, 이것이 아래에서 보는 LSTM의 얕은 골의 한 까닭일 수 있다. 미래 기상 실측을 입력으로 쓰는 세 리전(캘리포니아, 텍사스, 독일)의 예측을 기상 입력이 없는 24시간 전 값으로 바꾸어도 CAST의 절감은 무릎점에서 0.37%포인트, α=1에서 0.01%포인트 움직이고 위반은 0이다(캘리포니아만 바꾸면 무릎점이 조금 더 낮은 α를 골라 71.08%).

배정에 미치는 효과는 값의 오차 순서와 다르다. 예측기를 24시간 전 값, LSTM, 완전예지로 바꾸어 CAST를 돌린 결과가 표 6에 있다. α를 0.5로 고정하면 절감이 61.82%, 62.99%, 63.46%로 예측이 좋을수록 높다. 평균 절대 오차가 LSTM보다 조금 낮은 24시간 전 값이 절감은 1.17%포인트 낮으므로, 배정 결과를 정하는 것은 값의 오차만이 아니다. 곡선의 모양을 재면 24시간 창 안의 순위상관은 LSTM 0.629, 24시간 전 값 0.677이고, 예측 최저 시각이 실측 최저 시각과 맞는 비율은 21.5%와 30.3%로 오히려 24시간 전 값이 높다. 둘이 크게 갈리는 것은 골의 깊이다. 예측한 하루 최대−최소를 실측의 것으로 나눈 납작함이 24시간 전 값은 1.001, LSTM은 0.647이다. LSTM은 골을 실측의 65% 깊이로 납작하게 그린다. 탄소만 보는 α=1에서는 95.59%, 95.65%, 95.87%로 차이가 0.28%포인트 안이다. 무릎점에서는 64.44%, 71.93%, 67.20%(작업당 지연 39.2, 47.8, 40.7 ms)로 LSTM이 가장 높아 순서가 뒤집힌다. 이는 예측의 질 때문이 아니라 무릎점 규칙이 곡선의 모양에 따라 다른 α를 고르기 때문이며, 4.5절에서 가른다. 어느 예측기에서든 CAST의 위반은 0이다. 탄소 쪽의 한 시간 앞 정보, 곧 지금 슬롯의 첫 한 시간에 사후 확정값을 쓰는 것을 없애고 한 시간 앞서 발행된 예측만 써도 위반은 그대로 0이다. 절감은 α=1에서 0.17%포인트, Azure 무릎점(예약 13 판)에서 0.52%포인트 낮아진다. LSTM 무릎점은 2.42%포인트 낮은 69.50%가 되고(24시간 전 값은 오히려 1.01%포인트 높은 65.45%), 이 감소는 무릎점이 고르는 α가 달라진 몫이 크다(작업당 지연 47.85 ms에서 45.89 ms).

표 6 예측기와 설계에 따른 절감률(%)과 작업당 평균 왕복 지연(ms). 무릎점끼리는 지연이 달라 같은 조건의 비교가 아니다(같은 지연 짝은 4.4절). 2단계 행은 확인 시점 말고도 다른 점이 많아 참고값이다. 괄호는 상한 17을 넘긴 배정 건수이고 —는 돌리지 않은 칸이다. 예약 행이 없는 2단계와 용량 무시를 뺀 실행은 대표 설정인 예약 3이며, 예약 없는 같은 지연의 설계 비교는 4.4절에 있다. 2단계의 η는 내부 상한을 물리 상한의 몇 배로 두었는지이며, η=1.0이면 내부 상한이 물리 상한과 같고, η=0.83은 내부 상한 14로 CAST의 미래 슬롯 상한 17−ρ와 같다. Arrival-fixed는 도착 고정형(3.4절)이다. 탄소 쪽 한 시간 앞 정보를 없애면 LSTM의 CAST 무릎점은 69.50%다(4.2절). 표의 영문 이름은 Oracle이 완전예지, Two-stage가 2단계, Decoupled가 분리형, Capacity ignored가 용량 무시다.

Table 6 Savings (%) and per-job mean round-trip latency (ms) by forecaster and design. Knees differ in latency, so knee-to-knee gaps are not like-for-like (latency-matched pairs are in Section 4.4); two-stage rows are for reference. Parentheses give assignments past the ceiling of 17; a dash marks a run not performed. All runs except two-stage and capacity ignored, which have no reservation rows, use the representative reservation of 3; latency-matched design comparisons without reservation are in Section 4.4. For the two-stage design, η is the internal ceiling as a fraction of the physical one; at η=1.0 the two coincide, and η=0.83 gives an internal ceiling of 14, equal to CAST's future-slot ceiling 17−ρ. Arrival-fixed is the arrival-fixed control of Section 3.4: each job's region and start are fixed once in its arrival slot against all existing plans and never re-solved. Without the one-hour carbon lookahead, CAST at the knee with LSTM saves 69.50% (Section 4.2).

Design	SN-24h	LSTM	Oracle
Two-stage, η=0.8	63.04 (0) / 36.2	63.78 (6) / 36.2	64.38 (19) / 36.2
Two-stage, η=0.83	—	64.31 (6) / 36.2	64.92 (31) / 36.2
Two-stage, η=1.0	64.54 (125) / 36.2	65.26 (18) / 36.2	66.02 (152) / 36.2
Arrival-fixed, knee	62.43 (0) / 40.1	64.28 (0) / 41.2	64.23 (0) / 40.1
Arrival-fixed, α=1	—	93.03 (0) / 120.3	—
Decoupled, knee	43.77 (0) / 19.5	48.88 (0) / 23.4	46.62 (0) / 19.9
Decoupled, α=1	95.57 (0) / 119.9	95.34 (0) / 120.0	95.85 (0) / 119.9
CAST, fixed α=0.5	61.82 (0) / 32.7	62.99 (0) / 33.9	63.46 (0) / 32.6
CAST, knee	64.44 (0) / 39.2	71.93 (0) / 47.8	67.20 (0) / 40.7
CAST, α=1	95.59 (0) / 120.0	95.65 (0) / 120.0	95.87 (0) / 119.9
Capacity ignored, knee	66.29 (29,086) / 40.3	73.39 (24,993) / 47.8	69.25 (33,964) / 41.5
Capacity ignored, α=1	—	95.97 (46,354) / 120.4	—

4.3 로드밸런서 평가

본 연구의 일곱 방식을 같은 회계와 같은 상한 17로 비교하였다. 기준은 홈 리전 즉시 실행이다. 시간만은 리전을 홈 리전에 고정하고 스케줄러만 쓴 것, 공간만은 시각을 그대로 두고 로드밸런서만 쓴 것이며, 분리형, CAST, 용량 무시, 탐욕은 3.4절에서 정의했다. 이 일곱 이름과 3.4절의 2단계, 도착 고정형을 이후 모든 표와 그림에서 그대로 쓴다. 각 방식은 무릎점과 α=1 두 가중치로 돌렸고 결과는 표 7에 있다.

로드밸런서만으로도 절감의 대부분이 나온다. 공간만은 무릎점에서 61.57%(작업당 지연 40.4 ms), α=1에서 93.96%(120.1 ms)를 줄이며, 두 경우 모두 최대 동시 실행이 상한 17을 넘지 않는다. 리전 상한을 보지 않는 CASPER는 자체 서버 용량에도 사실상 묶이지 않는다. CASPER는 작업당 지연이 120.4 ms로 95.30%를 줄여 공간만의 α=1보다 1.34%포인트 더 줄이지만, 그 배정은 리전 하나가 동시에 29건을 돌려야 성립한다(4.8절). 다만 그 1.34%포인트에는 용량을 지킨 대가와 함께 목적함수와 구현의 차이도 섞여 있다. 제약 두 줄만 다른 짝은 아래의 용량 무시다. 공간 이동이 실제로 만든 흐름은 그림 5에 있다. CAST(무릎점)는 채점 구간 작업의 58.4%인 83,665건을 홈 리전 밖에서 돌렸고, 리전별로 실행된 몫은 프랑스 51.6%, 캘리포니아 27.3%로 두 리전이 79.0%를 차지한다. 가장 큰 흐름은 독일에서 프랑스(옮긴 물량의 21.3%)와 인도에서 프랑스(20.2%)이며, 텍사스와 일본, 한국의 작업은 주로 캘리포니아로 간다.

이용률이 8.4%인 이 워크로드에서 CAST가 용량을 지키는 비용은 작다. 용량 무시는 CAST(예약 3)보다 무릎점에서 1.47%포인트(73.39%), α=1에서 0.31%포인트(95.97%) 더 줄인다. 그러나 최대 동시 실행이 63건과 117건으로 상한의 3.7배와 6.9배이고, 상한 위반이 24,993건과 46,354건이다. 넘친 자리는 거의 모두 로드밸런서가 작업을 모으는 리전에서 난다(4.6절). α=1의 CAST는 작업시간의 99.9%를 프랑스에서 돌려(무릎점은 프랑스 49.2%, 캘리포니아 31.2%, 일본 10.3%) 서론의 Sukprasert 정책과 같은 모습이며, 다른 점은 상한 안에서 그렇게 한다는 것이다(필요 용량 17 대 129). 그래서 α=1에서의 1%포인트 미만 차(이 0.31%포인트, 4.5절의 α=1 비교, CASPER와의 차)는 프랑스 평균 배출계수로 약 1 gCO_2/kWh 수준의 차로

표 7 일곱 방식의 절감률, 작업당 평균 지연, 최대 동시 실행. 각 칸은 무릎점 / α=1이며, 최대 동시 실행은 리전별 최댓값 가운데 가장 큰 값으로 4.8절의 필요 용량과 같은 척도다. 상한은 17, 예약은 3이며, CAST·분리형·용량 무시 행은 표 6의 LSTM 열과 같은 실행이고, 시간만은 지연 축이 없고 탐욕은 무릎점에서만 돌려 한 값만 적었다. 무릎점끼리의 차에는 설계마다 다른 α가 섞여 있으며, 같은 α에서의 설계 차는 4.5절에 있다.

Table 7 Savings, mean latency and peak concurrency of the seven methods. Each cell is knee / α=1; Req. cap. is the largest per-region maximum concurrency, the same measure as the required capacity in Section 4.8. The ceiling is 17; the CAST, decoupled and capacity-ignored rows are the same runs as the LSTM column of Table 6; Temporal-only has no latency axis and Greedy was run at the knee only, so each has a single value. Differences between knees mix in each design's different α; same-α design differences are in Section 4.5.

Method	Savings (%)	Per-job RTT (ms)	Req. cap.
Baseline	0 / 0	0 / 0	13 / 13
Temporal-only	18.78 / —	0 / —	17 / —
Spatial-only	61.57 / 93.96	40.4 / 120.1	17 / 17
Decoupled	48.88 / 95.34	23.4 / 120.0	17 / 17
CAST	71.93 / 95.65	47.8 / 120.0	17 / 17
Capacity ignored	73.39 / 95.97	47.8 / 120.4	63 / 117
Greedy	71.08 / —	47.6 / —	17 / —

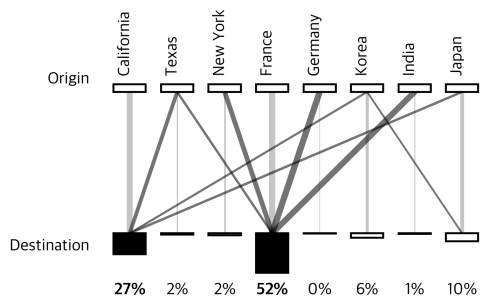


그림 5 로드밸런서가 만든 리전 간 흐름. CAST(무릎점)의 채점 구간 143,200건에서 옮긴 물량의 93.9%를 차지하는 흐름 8개이며, 회색 세로선은 홈에 남은 몫, 선 굵기는 옮긴 건수에 따라 굵어지고, 아래 상자 높이는 리전별로 실행된 몫에 정비례하며(몫이 2% 남짓 이하인 리전은 납작한 선으로 보인다) 검은 칠은 그 몫이 20%를 넘는 리전이다.

Fig. 5 Inter-region flow created by the load balancer. The 8 flows carrying 93.9% of moved jobs among the 143,200 scored jobs under CAST (knee α). Gray verticals are the share kept at the origin, line width grows with the number of jobs moved, lower box heights are strictly proportional to each region's share of executions (regions at about 2% or less appear as flat lines), and black fill marks regions above 20%.

읽어야 한다. 작업당 1 kW 가정에서 CAST는 1년에 무릎점에서 약 20.6 t, α=1에서 약 27.4 t을 덜 내보낸다.

옮긴 작업마다 보내는 자료가 작으면 전송 에너지를 더해도 설계의 순서는 바뀌지 않지만, 크면 공간 이동의 이득이 사라진다. 위의 회계는 리전 사이로 옮긴 작업의 전송 에너지를 넣지 않았다. 옮긴 작업마다 데이터 1 GB를 보내는 데 드는 0.06 kWh[37]를 목적지 리전의 탄소집약도로 더해도, CAST의 절감은 무릎점에서 71.93%가 71.16%로, α=1에서 95.65%가 95.20%로 줄어 1%포인트 안이다. 비교한 아홉 실행(표 7에서 기준과 탐욕을 뺀 실행)의 순서도 바뀌지 않는다. 전송 에너지를 출발지(홈 리전)의 실행 구간 평균 탄소집약도로 세면 무릎점은 66.01%(5.92%포인트 감소), α=1은 87.65%(8.00%포인트 감소)로 더 줄지만 순서는 여전히 같다. 작업이 긴 Azure에서는 무릎점이 51.35%에서 50.69%로 줄 뿐이다. 출발지로 셀 때 작업당 약 9 GB면 CAST 무릎점이 시간만(18.78%)과 같아지고, 약 12 GB면 절감이 0이 된다. 작업당 1 kW 회계에서 작업이 긴 Azure는 시간만과 같아지는 손익분기가 약 75.5 GB다. 이 가정에 매이지 않게 옮긴 작업마다 같은 GB를 보낸다고 두고 적으면, 시간만과 같아지는 전송량은 옮긴 작업의 계산 에너지 1 kWh당 합성 12.9 GB, Azure 13.8 GB이며, 작업 전력이 1 kW보다 작으면(Azure에서 코어 수로 전력을 줄이면) 작업당 손익분기도 같은 비율로 준다. VM의 실제 전력(대부분 2~4코어)으로 두면 손익분기는 VM당 1 GB 안팎으로 줄어, VM을 옮기는 순이득은 디스크 이미지 크기에 따라 부호가 바뀔 수도 있다(추정). 0.06 kWh/GB는 2015년 값이라 지금 값으로는 손익분기가 더 크다. 건수로는 옮긴 작업의 55.2%가 1 GB 전송 에너지가 계산 에너지보다 큰 짧은 작업이지만, 이들의 배출은 옮긴 작업 배출의 0.69%다. 목적식에 이동 비용이 없는 것은 한계다.

4.4 스케줄러 평가

스케줄러만 쓴 시간만은 홈 리전을 떠나지 않고 시각만 옮겨 18.78%를 줄인다. 평균 2.17시간 미루며, 최대 동시 실행은 상한 17에 닿되 넘지 않는다. 스케줄러가 용량을 어떻게 다루는지는 캘리포니아의 한 날을 열어 보면 분명하다. 그림 6은 용량 무시 실행에서 캘리포니아의 동시 실행이 연중 가장 높았던 날인 7월 7일의 탄소 곡선, 그날 시작해 미뤄진 작업 가운데 창이 가장 넓은 9건, CAST의 시각별 동시 실행을 위아래로

겹친 것이다. 그날 CAST의 동시 실행은 하루 24시간 가운데 8시간 동안 상한 17에 닿아 있고, 그 시간은 현지 9시에서 16시, 곧 태양광이 만드는 한낮의 저탄소 구간이다.

작업들이 가고 싶어 하는 시각은 모두 같은 저탄소 구간이며, 용량을 보지 않았다면 받았을 시각(파선 상자)과 CAST가 실제로 준 시각(실선 상자)이 어긋나는 것은 그 구간이 이미 상한까지 찼기 때문이다. 굵은 테두리로 표시한 작업은 현지 0시 58분에 제출된 2.36시간짜리 작업이다. 용량을 보지 않았다면 그날 탄소가 가장 낮은 시각(현지 11시와 12시가 같다) 가운데 12시에 시작했겠지만, 그 시각이 이미 상한에 닿아 있어 CAST는 이 작업을 원하던 자리보다 세 시간 앞당긴 현지 9시에 시작했다.

용량을 언제, 무엇과 함께 확인하느냐가 결과를 정한다. CAST와 같은 선형 계획, 예측, 무릎점 규칙, 풀이기 위에서 확인 방식만 다른 대조군(3.4절)을 견주었다. 도착 슬롯에 리전과 시작 시각을 확정하되 미래 슬롯의 용량을 이미 확정한 모든 계획과 함께 보는 도착 고정형은 LSTM, 완전예지, Azure에서 모두 위반이 0이다. 시작한 작업만 확인형은 다른 작업의 확정 계획을 보지 않고 확정하며, 시작 때 다시 확인하지도 않는다. 그 결과 합성 워크로드에서는 채점 작업의 9.0~17.0%가 상한 위반으로 시작한다. 이 범위는 원래 워크로드와 다시 뽑은 네 벌을 모두 포함하며, 예약은 각 워크로드에 준비 구간 규칙(3.3절)을 적용한 2~3건이다. Azure에서는 예약 없이 11.4~14.7%였다. 상한을 바꾸어도 이 차이는 남는다. 상한을 가장 큰 기준 피크의 1배(13)와 2배(26), 리전별 상한(깨끗한 세 리전을 작게)으로 두면 시작한 작업만 확인형(무릎점, 예약 3)은 작업의 12.03%, 4.76%, 14.97%가 상한 위반이다(상한 17에서는 9.01%). 같은 세 상한에서 CAST는 고정 α=0.5375(예약 없이)로 모두 위반이 0이고(절감 67.71%, 69.19%, 66.38%), 무릎점·예약 3으로도 위반이 0이다. 시작 때 다시 확인하는 입장 통제형에서는 위반이 LSTM 227건, 완전예지 387건으로 준다(0.16~0.27%, 시드 포함 0.16~0.56%, Azure 예약 없이 5.2~5.3%). 남은 위반은 모두 여유가 다한 작업을 처음 정한 리전에 그대로 밀어 넣는 규칙에서 나왔고, 두 워크로드 모두 그 순간 다른 리전에는 늘 빈자리가 있었다. 거절된 작업이 리전과 시각을 다시 고르게 해도 이 규칙이 남으면 위반이 160~299건(Azure ρ=13에서 608건) 생긴다. 꽉 찬 리전의 작업을 그 순간 자리가 있는 리전으로 보내는 대피형은 예측을 섞은 다섯 판, 24시간 전 값, Azure의 두 예측기 모두에서 위반이 0이다. 실제 클러스터 스케줄러가 시작 때 자리를 확인하면 이 두 설계에 가깝다. 예측이 정확할수록 대기 시간이

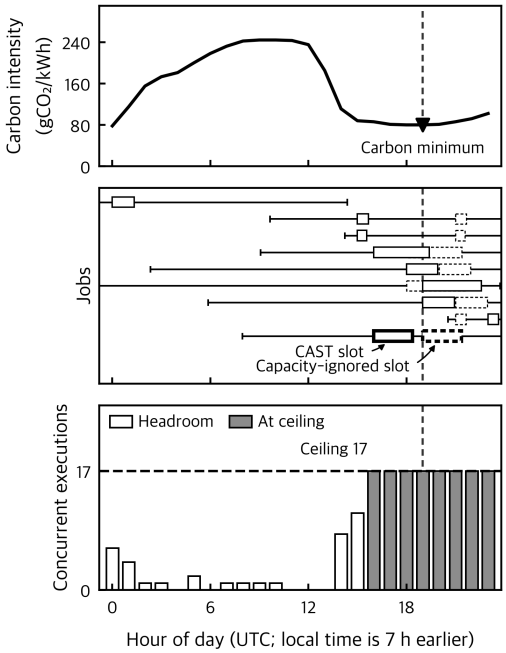


그림 6 용량이 저탄소 시각으로의 몰림을 막는 방식. 캘리포니아의 한 날(7월 7일)이며 그날 시작해 미뤄진 작업 중 창이 19.48시간 이상인 9건이다. 가는 선이 창, 실선 상자가 CAST의 실행 구간, 파선 상자가 용량 무시에서 받은 시각이며(맨 위 행은 전날 19시 UTC라 그림 밖), 굵은 테두리(맨 아래 행)가 본문의 예다. 아래 단은 CAST의 동시 실행과 상한 17이다.

Fig. 6 How capacity keeps jobs from piling into the low-carbon hours. One day in California (7 July), showing the 9 deferred jobs started that day with windows of at least 19.48 h. Thin lines are windows, solid boxes CAST's executed intervals and dashed boxes the slots received under capacity ignored (off the frame for the top row, at 19:00 UTC the previous day), with the bold outlines (bottom row) marking the example in the text; the bottom panel shows CAST's concurrency against the ceiling of 17.

는다(대피형 2.06시간에서 2.48시간). 이는 대피형만의 일이 아니라 CAST(1.47시간에서 2.13시간)와 도착 고정형(2.05시간에서 2.49시간)에서도 같으므로 몰림의 결과로 볼 수 없고, 정확한 예측이 더 깊은 저탄소 구간을 보여 주기 때문으로 보인다(이 대기 시간 비교는 합성, 예약 3).

매 슬롯 다시 풀며 지금 시작만 확정하면 미래 슬롯 행 없이도 위반이 없다. 미래 행 제거형은 LSTM, 완전예지, 24시간 전 값, Azure에서 모두 위반 0이다. 위반 0은 두

가지에서 따라 나온다. 하나는 지금 시작만 확정하고 지금 슬롯을 순간 행으로 세는 구성이고, 다른 하나는 리전을 모두 합친 용량의 여유다. 이용률은 합성 8.4%, Azure 26.1%이고, 리전을 모두 합친 동시 실행의 최댓값은 합성 46, Azure 160으로 상한의 합(136, 344)보다 작다. 4.7절의 상한 실험에서 이용률이 가장 높았던 조건(합성 20.5%, Azure 43.2%)에서도 위반은 0이었다. 이보다 더 조인 Azure 상한 0.6배(21, 이용률 53.5%)에서도 CAST와 도착 고정형 모두 위반이 0이었다(무릎점 27.17%와 26.94%). 0.5배(17, 이용률 66.1%)에서는 처음으로 위반이 나서 CAST는 465건(0.32%), 도착 고정형은 847건(0.59%)이다. CAST의 465건은 모두 그 순간 다른 모든 리전도 상한까지 차 옮길 자리가 없을 때였다. 미룰 수 없는 작업이 271건, 여유가 다한 작업이 187건, 그 밖이 7건이며, 그 7건도 모든 리전이 찬 순간이었다. 물리 초과 z^{4}도 이 상한에서 처음으로 71 슬롯에서 양수가 되었다. 이 위반을 미래를 알았다면 피할 수 있었는지 보려고, 위반이 가장 많았던 주를 골라 앞 주에서 넘어온 작업은 고정한 채 그 주를 다아는 선형 계획으로 다시 풀었다. 결과는 초과를 세는 방법에 따라 갈렸다. 슬롯을 조금이라도 덮는 실행을 모두 세면 다 알고 풀어도 초과가 224 작업·슬롯 남았고(CAST의 실제 실행은 같은 셈으로 1,582), 슬롯을 통째로 덮는 실행만 세면 초과가 없었다. 그래서 이 위반을 사후에 피할 수 있었는지는 확정하지 못했다. 도착 고정형은 847건 가운데 65건을 다른 리전에 빈자리가 있는데도 넘겼다.

미래 슬롯 행이 하는 일은 위반을 막는 것이 아니라 붐빌 슬롯을 미리 피해 절감을 조금 더하는 것이다. 예약 없는 CAST는 미래 행 제거형보다 LSTM에서 0.67, 완전예지에서 0.72, 24시간 전 값에서 0.44, Azure에서 0.12%포인트를 더 줄인다(같은 지연 짝은 아니다). 예약 13을 둔 Azure에서는 미래 행 제거형(51.22%)이 CAST(46.14%)보다 높은데, 미래 행 제거형에는 예약 행인 식 (12)가 없어 예약 비용이 없기 때문이다. 예약 없는 CAST(51.35%)와는 0.12%포인트 차다.

도착 고정형과 CAST는 둘 다 위반이 없고, 다시 푸는 것의 몫은 절감에 있다. 위반 0에 이르는 길은 이미 정한 배정을 모두 보는 것[9](도착 고정형)과 확정하지 않고 매 슬롯 다시 푸는 것(CAST) 둘이다. 두 설계를 4.1절의 같은 지연 짝으로, 두 워크로드 모두 예약 없이 견준 결과가 표 8이다. 한 점이 아니라 곡선 전체에서 보아도 같다(그림 7). 두 설계의 고정 α 열한 점은 모두 위반이 0이다. 지연이 10 ms 아래인 $\alpha \le 0.4$에서는 시간 이동이 주이고 두 곡선이 겹친다(차 −0.3~0.0%포인트). 지연이 30 ms 위인 $\alpha \ge 0.5$에서는 CAST가 같은 지연에서 0.9~2.1%포인트 위에 있고 대기 시간도 짧다(α=0.6에서 1.65 대 2.04시간). 같은 지연 짝으로는 합성 워크로드에서 CAST가 도착 고정형보다 기준 배출의 0.96%포인트를 덜 배출한다. 워크로드를 다시 뽑은 네 벌에서도 고정 α=0.575의 짝으로 1.23~1.56%포인트 덜 배출하며, 신뢰구간은 모두 0 밖이다. 0.96%포인트를 실행 리전별로 나누면 일본 +1.37, 독일과 캘리포니아 각 +0.24, 프랑스 +0.07이고 한국(−0.59)과 텍사스(−0.33)는 반대다. 프랑스의 작업시간 비중은 47.5% 대 46.8%로 거의 같고 CAST는 모든 리전에서 작업 kWh당 배출이 더 낮으므로, 이점은 프랑스에 더 몬 데서가 아니라 리전마다 더 깨끗한 시각을 다시 고른 데서 온다.

다시 푸는 몫은 깨끗한 리전의 상한이 빠듯할수록 크고 넉넉하면 사라진다. 상한을 바꾼 같은 지연 짝(예약 없이)에서 CAST가 덜 배출한 양은 상한 13에서 1.46%포인트(1.13~1.83), 26에서 0.34%포인트(−0.06~0.77, 0 포함), 깨끗한 세 리전을 작게 둔 리전별 상한에서 2.10%포인트(1.83~2.36)다. 대기 시간은 짝을 잰 네 상한 설정 모두에서 CAST가 짧다. 완전예지에서도 같은 짝으로 1.01%포인트(0.80~1.25)가 남아, 이 몫은 예측이 아니라 새로 도착한 작업과 함께 다시 짜는 데서 온다. 대기 시간도 다섯 벌 모두 CAST가 짧다. 도착 고정형은 한 번 정한 자리를 나중에 더 깨끗한 자리가 보여도 바꾸지 않기 때문이다. 그 대신 도착 고정형은 시작 시각이 도착 때 정해져 사용자에게 미리 알려 줄 수 있고(CAST는 시작하기 전까지 알려 줄 수 없다), 계산이 적으며, 예약에 덜 흔들린다. 다른 작업의 확정 계획을 보지 않고 확정하되 시작 때 빈 리전으로 보내는 대피형도 CAST보다 0.91%포인트 더 배출한다. 기본 운용 규칙인 무릎점 규칙의 CAST(예약 없이 72.06%)는 같은 지연의 도착 고정형과 배출 차가 유의하지 않고(점추정은 도착 고정형이 0.94%포인트 낮다), 대기 시간은 25% 짧다.

표 8 위반 없는 가장 가까운 설계와의 같은 지연 짝. Δ는 상대 설계 배출 − CAST 배출(각 워크로드 기준 배출 대비 %포인트, 양수면 CAST가 덜 배출)이며 괄호는 7일 블록 부트스트랩 95% 신뢰구간이다. 모든 짝은 예약 없이 실행 시간 가중 왕복 지연을 0.6 ms 안으로 맞췄고(마지막 행만 Azure ρ=13), 대기는 CAST / 상대의 평균 대기 시간(h)이다. 무릎점 행의 CAST는 슬롯별 무릎점이다.

Table 8 Same-latency pairs with the closest designs that also have no violations. Δ is the other design's emissions minus CAST's (percentage points of each workload's baseline; positive = CAST emits less), with 95% seven-day block-bootstrap intervals. All pairs are without reservation and matched within 0.6 ms of duration-weighted round-trip latency (last row: Azure, ρ=13); wait is mean waiting time of CAST / the other design (h). In the knee row CAST uses its per-slot knee.

Workload	Other design	CAST α	Other α Δ pp (95% CI)	Wait (h)
Synthetic	Arrival-fixed	0.5375	0.5444 +0.96 (0.51–1.46)	1.72 / 2.03
Synthetic, seeds 1–4	Arrival-fixed	0.575	0.5812–0.5844 +1.23 to +1.56 (all > 0)	1.71–1.79 / 2.07–2.15
Synthetic	Evacuation	0.5375	0.5353 +0.91 (0.67–1.15)	1.72 / 2.07
Synthetic	Arrival-fixed	knee	0.5875 −0.94 (−2.48 to 0.51)	1.52 / 2.04
Azure	Arrival-fixed	0.5375	0.5381 +2.53 (1.68–3.42)	2.51 / 1.81
Azure	Evacuation	0.5375	0.5184 +1.99 (0.86–3.01)	2.51 / 2.25
Azure, ρ=13	Arrival-fixed	0.5375	0.5329 −2.10 (−3.35 to −1.01)	—

Azure에서는 CAST가 도착 고정형보다 2.53%포인트, 대피형보다 1.99%포인트 덜 배출하지만 대기 시간은 CAST가 더 길다. 준비 구간 규칙의 예약 13을 두면 같은 지연의 고정 α 짝에서 도착 고정형이 2.10%포인트 앞선다. 같은 α=0.5375에서 예약 행(식 (12))이 CAST의 절감을 6.18%포인트 깎기 때문이다. 다만 13은 계속 도는 VM을 두 번 센 과대값이다(3.3절). CAST의 이점은 대부분 매 슬롯 다시 풀며 지금 시작만 확정하는 데서 온다. 미래 슬롯 행은 0.12~0.72%포인트를 더하고(같은 지연 짝은 아니다), 초과 벌점과 예약은 이 워크로드에서 몫이 없다.

합성 워크로드에서는 미래 정보를 모두 빼도 같은 지연 짝에서 CAST의 이점이 남는다. 현실 정보 설정은 미래 정보 셋을 모두 뺀 것으로, 슬롯 안 도착을 미리 알지 않고, 지금 슬롯 탄소는 한 시간 전 예측을, 기상 세 리전은 24시간 전 값을 쓴다. 이 설정에서도 모든 판의 위반이 0이고, 같은 지연에서 CAST는 도착 고정형보다 0.96%포인트(신뢰구간 0.74~1.18), 대피형보다 1.13%포인트(0.83~1.36) 덜 배출하며 대기 시간도 짧다(2.28 대 2.40, 2.45시간). 고정 α=0.5375의 CAST 절감은 정보를 빼도 68.60%로 모든 정보를 쓸 때와 같고, 무릎점은 예약 없는 72.06%에서 68.88%로 3.18%포인트 준다. 그러므로 줄어드는 몫은 대부분 무릎점이 고르는 α에서 오며, CAST의 이점은 미래 정보에서 오지 않는다. 표 6의 무릎점끼리의 차는 지연이 달라 같은 조건이 아니다.

2단계 대조군(3.4절)은 시작 때 내부 상한으로 들이는 입장 통제의 한 꼴이며, 예측이 정확할수록 위반이 느는 방향도 입장 통제형과 같다. 내부 상한을 물리 상한 17과 같게 두면 위반이 LSTM 18건에서 완전예지 152건으로 늘고(표 6), LSTM 예측에 완전예지를 섞는 비율을 높여 가면 대체로 늘어 최대 164건에 이른다. 이 설계는 확인 시점 말고도 리전 단계, 시각 점수, 예약과 재풀이가 없다는 점, 풀이기가 달라 교란 요인이 많다. 내부 상한을 0.8배로 두면 위반이 0~19건으로 대부분 흡수되고(표 6), 24시간 전 값, 하루 한 번 발행한 LSTM, 잡음을 더한 완전예지 사이의 순서는 한 방향이 아니다.

용량을 모든 대기 작업이 함께 보는 제약으로 두면 예측이 정확해져도 위반이 늘지 않는다. 리전을 도착 때 고정하는 분리형과 CAST 모두, 무릎점과 α=1 각각에서 예측기를 세 가지로 바꾼 열두 실행이 전부 위반 0이다. 예약을 0으로 두어도 두 설계와 세 예측기의 여섯 조합 모두 위반이 0이고, 절감은 예약 3일 때와 0.3%포인트 안에서 같다. Azure 워크로드에서도 예약 없이 위반이 0이다(4.7절). 이 결과는 슬롯 안에 도착하는 작업을 슬롯 머리에서 모두 안다는 가정 아래의 것이며, 특히 미룰 수 없는 작업이 이 한 시간 앞 정보의 덕을 본다. 그래서 이 정보를 없앤 판(미룰 수 있는 작업은 다음 슬롯 머리에, 미룰 수 없는 작업은 도착 순간 하나씩 배정)을 합성과 Azure, 예약 유무, 예측기를 바꾸어 다섯 번 돌렸고, 모두 위반이 0이었으며 절감은 0.56~1.62%포인트 줄었다. 리전을 먼저 정하는 분리형에서는 미룬 작업이 그날 탄소가 가장 낮은 세 시각에 시작하는 몫이 LSTM 23.9%에서 완전예지 52.5%로 늘어 몰림이 커진다. 그래도 여덟 예측기 변형(24시간 전 값, LSTM, 완전예지, 하루 한 번 발행한 LSTM, 잡음을 더한 완전예지, 완전예지를 25, 50, 75% 섞은 LSTM) 전부에서 위반이 0이다. 탄소만 보는 α=1에서는 두 설계 모두 완전예지가 가장 높은 절감을 내고, 세 예측기의 차이도 0.6%포인트 안이다.

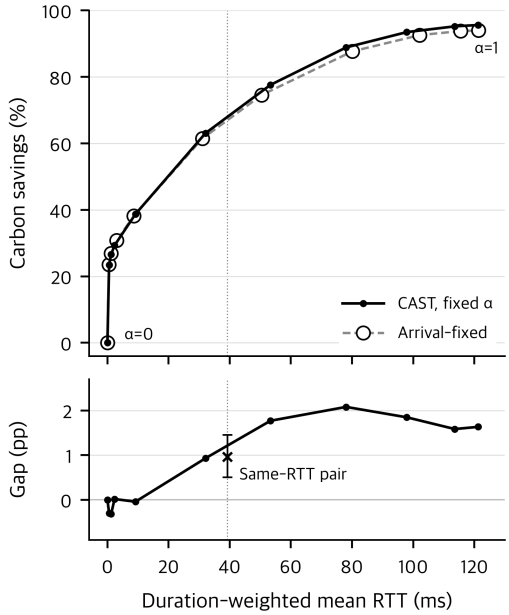


그림 7 예약 없는 합성 워크로드에서 α를 고정한 두 설계의 곡선(LSTM). 위 칸에서 ●와 실선은 매 슬롯 다시 푸는 CAST, ○와 파선은 도착 때 정한 자리를 다시 풀지 않는 도착 고정형이며, 점은 α=0, 0.1, …, 1이고 가로축은 실행 시간 가중 왕복 지연이다. 아래 칸은 CAST 점마다 도착 고정형 곡선과의 세로 간격(%포인트, 선형 보간)이고, 점선은 지연을 맞춘 실측 짝(39.2 ms, α=0.5375 대 0.5444)의 자리, ×와 막대는 그 차 0.96%포인트와 95% 신뢰구간(0.51~1.46)이다. 39.2 ms에서 보간 간격(1.21)과 실측 짝이 다른 것은 보간 때문이다. 모든 점에서 위반은 0이다.

Fig. 7 Fixed-α curves of the two designs on the synthetic workload without reservation (LSTM). Top: filled dots and solid line are CAST, which re-solves every slot; hollow dots and dashed line are the arrival-fixed design, which does not re-solve placements made on arrival; points are α = 0, 0.1, …, 1 and the x axis is duration-weighted round-trip latency. Bottom: vertical gap between each CAST point and the arrival-fixed curve (pp, linear interpolation); the dotted line marks the measured latency-matched pair (39.2 ms; α = 0.5375 vs 0.5444), and the cross and bar give its difference, 0.96 pp (95% CI 0.51–1.46). The interpolated gap at 39.2 ms (1.21) differs from the measured pair because of interpolation. No point has a violation.

용량을 보지 않을 때와 CAST의 차이를 한 주로 늘려 보면 그림 8과 같다. 7월 7일이 든 주(day 184~190)에서

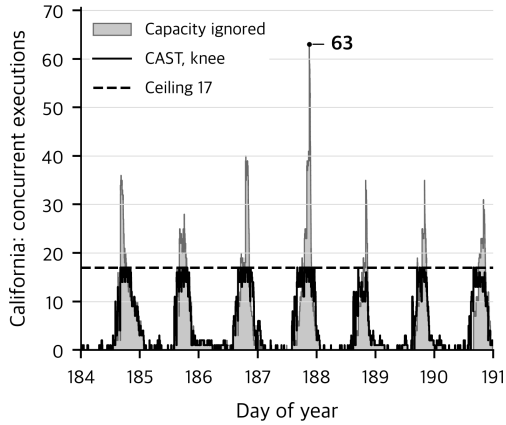


그림 8 캘리포니아의 동시 실행, 용량 무시와 CAST. 연중 한 주(day 184~190, 1월 1일을 0으로 센 날로 7월 4~10일)이며 회색 채움이 용량 무시, 검은 선이 CAST, 파선이 상한 17이다.

Fig. 8 California concurrent executions under capacity ignored and CAST. One week of the year (days 184–190 counted from 0 on 1 January, i.e. 4–10 July); gray fill is capacity ignored, the black line CAST, and the dashed line the ceiling of 17.

용량 무시는 한낮마다 상한을 넘겨 최대 63건에 이르고, CAST는 같은 시각을 17건에서 멈춘다.

분리형에서 용량 행은 절감을 깎지 않고 위반만 없앤다. 분리형에서 용량 행을 지우면 위반이 13,845건, 초과 시간이 599시간 생기고 최대 동시 실행이 45건에 이르지만, 절감은 48.42%로 용량이 있을 때의 48.88%보다 오히려 조금 낮다. 상한이 없으면 무릎점 규칙의 후보 열한 개가 모두 비교 대상에 들어가 정규화가 바뀐다. 그래서 무릎점이 α 평균 0.438에서 0.434로, 작업당 평균 지연이 23.4 ms에서 22.9 ms로 지연이 짧은 쪽으로 옮겨 간다. 저탄소 구간에 몰린 작업이 더 깨끗한 시각을 얻는 것도 아니다.

작업 143,200건은 모든 방식에서 전량 마감 이내에 처리되었다. 후보 시각을 마감 이내에서만 찾으므로 설계상의 결과다.

4.5 결합과 무릎점

로드밸런서와 스케줄러를 한 문제로 합쳐서 더 얻는 몫은 가중치에 따라 다르다. 같은 무릎점 규칙을 쓴 실행끼리 비교하면 CAST는 공간만보다 10.35%포인트를 더 줄이고, α=1에서는 1.70%포인트를 더 줄인다(표 7). α=1에서는 작업 대부분이 가장 깨끗한 리전으로 가므로 시각을 옮겨 더 얻을 것이 적다. 다만 두 무릎점은

표 9 같은 고정 α에서 CAST와 분리형의 비교. 칸은 절감률(%)이고 예약은 3이며, 두 실행의 실행 시간 가중 지연 차는 최대 1.71 ms(α=0.6)라 같은 지연의 짝은 아니다. 차는 분리형 배출 − CAST 배출(kg, 양수면 CAST가 덜 배출)이며 괄호는 7일 블록 부트스트랩 95% 신뢰구간, pp는 기준 배출 대비 %포인트다.

Table 9 CAST versus the decoupled design at the same fixed α. Cells are savings (%) with reservation 3; the two runs differ in duration-weighted latency by up to 1.71 ms (α=0.6), so they are not latency-matched pairs. Diff. is decoupled minus CAST emissions in kg (positive = CAST emits less) with 95% 7-day block-bootstrap intervals; pp is the gap in percentage points of baseline emissions.

α	CAST	Decoupled	Diff. (95% CI)	pp
0.4	38.64	37.80	238 (66 to 434)	0.83
0.5	62.99	62.02	276 (205 to 351)	0.96
0.6	77.69	76.15	441 (354 to 528)	1.54
0.7	88.99	88.29	200 (114 to 282)	0.70
0.8	93.51	93.31	59 (9 to 109)	0.20
0.9	95.24	94.98	73 (34 to 115)	0.25
1.0	95.65	95.34	91 (52 to 135)	0.32

슬롯마다 고른 α의 평균이 공간만 0.488, CAST 0.556으로 달라서 같은 α의 비교가 아니다. 분리형의 무릎점 절감이 공간만보다 낮은 것(48.88% 대 61.57%)도 시간 축을 더해서가 아니라 무릎점 규칙이 고른 α 때문이다.

설계끼리는 같은 α에서 비교해야 하며, α=0.4 이상의 같은 α에서는 CAST가 분리형보다 근소하게 더 줄인다. 상한이 가장 큰 기준 피크의 1.25배인 이 조건에서 CAST와 분리형의 고정 α 곡선은 거의 겹친다. 그러나 같은 α끼리 비교하면 CAST가 α=0.4부터 1까지 일곱 점에서 0.20~1.54%포인트 덜 배출하고 신뢰구간이 0을 포함하지 않는다(표 9). 차이는 α=0.6에서 가장 크고 α=0.8 이상에서는 0.2~0.32%포인트로 작아진다. 다만 지연을 맞춘 짝은 아니어서, α=0.6에서는 CAST가 실행 시간 가중 지연을 1.71 ms 더 쓴다. 분리형 곡선을 CAST 점의 실행 시간 가중 지연에서 선형 보간해 이 지연 차를 보정하면 차는 α=0.4~1에서 0.32~0.74%포인트이고, α=0.6의 봉우리(1.54%포인트)는 절반 넘게 지연 차에서 온다. $\alpha \le 0.3$에서는 작업당 지연이 3 ms 아래이고 부호가 섞인다(0.3에서 분리형이 1.11%포인트 앞선다). 이 짝 차가 워크로드 시드에 따라 얼마나 흔들리는지는 재지 않았다. 또 이 차는 합성 워크로드의 결과이며, 시간 이동이 거의 절감을 내지 못하는 Azure 워크로드에서는 성립하지 않는다(4.7절).

선형 계획이 더하는 몫은 작업마다 차례로 가장 싼 자리를 잡는 탐욕과 비교하면 드러나며, CAST가 탐욕보다 근소하게 덜 배출한다. 탐욕은 무릎점에서 71.08%를 내며 위반이 없고 최대 동시 실행도 상한과

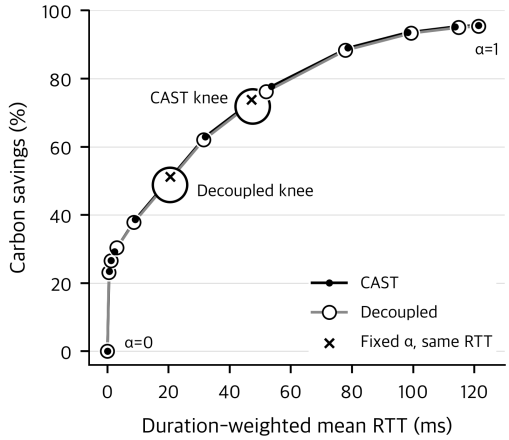


그림 9 고정 α 곡선과 슬롯별 무릎점. ●와 검은 선은 CAST, ○와 회색 선은 분리형의 고정 α 열한 점(0, 0.1, …, 1)이며, 선은 점을 이은 보조선이고 과녁 표식이 각 설계의 무릎점이고(CAST의 ×는 무릎점과 거의 같은 자리라 과녁 안에 있다), ×는 무릎점과 같은 실행 시간 가중 왕복 지연을 내는 고정 α(CAST 0.575: 47.08 ms, 73.79%, 무릎점 47.48 ms; 분리형 0.4156)이며, 가로축은 실행 시간 가중 왕복 지연이다.

Fig. 9 Fixed-α curves and per-slot knee points. Filled dots with the black line are CAST and hollow dots with the gray line the decoupled design at eleven fixed α (0, 0.1, …, 1); lines only connect the points, target markers are each design's knee point (CAST's cross lies inside its target), and crosses are the fixed α giving the same duration-weighted round-trip latency as each knee (CAST 0.575: 47.08 ms, 73.79%, knee 47.48 ms; decoupled 0.4156); the x axis is duration-weighted round-trip latency.

같은 17이다. 두 실행의 작업당 평균 지연은 0.29 ms 안으로 같다. 다만 짝을 맞춘 것은 아니며, 실행 시간 가중 지연은 0.73 ms 다르다. 이때 CAST는 탐욕보다 242 kg, 기준 배출의 0.84%포인트를 덜 배출한다(95% 신뢰구간 160~322 kg).

무릎점이 고정 α보다 나은지는 한 축으로 물을 수 없다. α=1이 탄소의 최솟값이고 α=0이 지연의 최솟값이기 때문이다. 고정 α 열한 개를 각각 1년 동안 돌려 얻은 (실행 시간 가중 왕복 지연, 절감)의 점이 그림 9에 있다. 열한 점을 이은 선은 보간일 뿐이라 그림의 보조선으로만 두고 판정에는 쓰지 않는다. 판정은 같은 지연 짝으로 한다(4.1절). 예약 없이 CAST와 도착 고정형의 고정 α 곡선을 견준 것은 그림 7(4.4절)에 있다.

표 10 무릎점과 같은 지연의 고정 α 짝(CAST). 짝은 목적식과 같은 실행 시간 가중 평균 지연이 무릎점과 0.6 ms 안에서 같은 고정 α이고, DW RTT 열은 무릎점 / 짝의 실행 시간 가중 평균 왕복 지연, Δ는 무릎점이 짝보다 더 배출한 양(괄호는 각 워크로드 기준 배출 대비 %포인트), 신뢰구간은 7일 블록 부트스트랩 2,000회의 95% 구간이다. 마지막 열은 작업당 평균 지연으로 맞춘 짝과의 차(%포인트)다. 위 세 행은 합성 워크로드의 상한 17, 다음 두 행은 LSTM에서 상한을 10과 7로 줄인 것, 그다음 네 행은 합성 워크로드를 시드 1~4로 다시 뽑은 것, 마지막 행은 Azure 워크로드(상한 43, 예약 없이)다.

Table 10 Knee point versus the fixed α at the same latency (CAST). The pair is the fixed α whose duration-weighted mean latency, the measure used by the objective, is within 0.6 ms of the knee's; DW RTT gives knee / pair duration-weighted latency, Δ is the knee's extra emission over the pair (percentage points of each workload's baseline in parentheses), and the CI is the 95% interval of 2,000 seven-day block bootstrap draws. The last column is the gap to the pair matched on per-job mean latency (percentage points). The top three rows use the synthetic workload at the ceiling of 17, the next two LSTM with the ceiling cut to 10 and 7, the next four the synthetic workload re-drawn with seeds 1–4, and the last row the Azure workload (ceiling 43, no reservation).

Condition	Knee (%)	DW RTT (ms)	Pair α	Δ kg (pp)	95% CI (kg)	Per-job Δ (pp)
LSTM	71.93	47.48 / 47.08	0.5750	534 (1.87)	31 to 1,072	0.93
SN-24h	64.44	36.74 / 36.69	0.5281	537 (1.88)	12 to 1,117	3.51
Oracle	67.20	38.83 / 38.48	0.5375	628 (2.20)	96 to 1,157	3.25
LSTM, cap 10	66.11	41.00 / 41.26	0.5540	756 (2.64)	336 to 1,191	2.64
LSTM, cap 7	56.79	30.63 / 30.15	0.5250	235 (0.82)	−9 to 484	2.19
LSTM, seed 1	71.39	46.86 / 47.02	0.5750	650 (2.28)	154 to 1,163	1.41
LSTM, seed 2	70.92	46.00 / 45.79	0.5750	555 (1.94)	118 to 999	1.04
LSTM, seed 3	71.95	47.36 / 46.86	0.5750	509 (1.77)	2 to 1,042	0.87
LSTM, seed 4	71.37	46.22 / 46.26	0.5750	584 (2.04)	132 to 1,054	1.19
Azure	51.35	39.35 / 38.94	0.5525	1,342 (0.63)	−2,247 to 5,094	−1.78

분리형에서는 무릎점이 같은 지연의 고정 α에 뒤진다. 무릎점의 실행 시간 가중 왕복 지연은 20.45 ms(작업당 23.4 ms)다. 같은 실행 시간 가중 지연으로 맞춘 짝은 고정 α=0.4156이며, 20.57 ms에서 51.20%를 낸다. 무릎점(48.88%)은 이 짝보다 666 kg, 기준 배출의 2.33%포인트를 더 배출한다(95% 신뢰구간 188~1,141 kg 더, 0.66~3.99%포인트). 작업당 지연으로 맞춘 짝(α=0.4375, 22.95 ms, 55.49%)과 견주면 차는 6.62%포인트(5.39~7.88)로 더 크다. 분리형의 무릎점은 α를 너무 낮게 골라 작업을 홈 리전에 남긴다. 분리형 무릎점이 예측기에 따라 CAST와 다른 방향으로 움직이는 까닭(표 6)은 가르지 않았다.

CAST에서도 무릎점은 실행 시간 가중 지연으로 맞춘 고정 α보다 더 배출하지만, 대기 시간은 짧다. 무릎점 규칙이 쓰는 실행 시간 가중 지연(47.48 ms)으로 맞춘 짝은 α=0.575(47.08 ms, 73.79%)이고, 무릎점은 이 짝보다 534 kg, 기준 배출의 1.87%포인트를 더 배출한다(95% 신뢰구간 31~1,072 kg 더). 작업당 지연으로 맞춘 짝(α=0.5687)과 견주면 차는 0.93%포인트로 줄고 신뢰구간이 0을 포함한다(표 10 마지막 열). 0.5687과 0.575는 1년을 돌려 찾은 값이지만, 아래에서 보듯 첫 주만 보고 골라도 비슷한 자리에 온다. 무릎점의 평균 대기 시간은 1.47시간으로 작업당 지연 짝의 1.63시간보다 짧고, 95 백분위수도 9.98시간 대 10.95시간이다. 대기 시간은 목적식에 없는 지표이고, 짧은 대기는 시간 이동을 덜 한 것의 다른 면일 수 있다. 같은 규칙이 분리형에서는 실행 시간 가중 짝보다 2.33%포인트 뒤진다.

무릎점이 실행 시간 가중 짝보다 더 배출하는 것은 합성 워크로드의 거의 모든 조건에서 되풀이되지만, Azure에서는 차이가 없다. 합성 워크로드의 아홉 조건에서 짝을 찾으면(표 10), 무릎점은 여덟 조건에서 1.77~2.64%포인트 더 배출하고 상한 7에서만 차이가 없다(0.82%포인트, 신뢰구간 −9~484 kg). 대표 조건과 시드 네 벌, 모두 다섯 벌을 묶으면 평균 1.98%포인트(시드 사이 표준편차 0.19)다. 같은 다섯 벌을 작업당 지연 짝으로 재면 평균 1.09%포인트이나, 벌마다의 신뢰구간은 모두 0을 포함한다. Azure에서는 실행 시간 가중 짝과 차이가 없다. 예약 ρ=13에서 0.20%포인트(신뢰구간 −2,389~3,491 kg), 예약 없이도 0.63%포인트(신뢰구간 −1.05~2.39%포인트)다. 작업당 평균 지연으로 맞춘 짝으로는 Azure에서 무릎점이 1.78%포인트 덜 배출한다(신뢰구간 0.03~3.50, 예약 없이). 평균 대기 시간은 Azure 세 벌을 포함한 모든 조건에서 무릎점 쪽이 짧다(Azure 시드 두 벌은 준비 구간 규칙으로 정한 예약 14와 15). 이 결과들은 무릎점이 고르는 자리가 예측 곡선의 모양에 따라 움직인다는 아래의 결과와 같은 방향이다.

작업당 지연으로 맞춘 고정 α=0.5687 하나는 예측기를 바꿔도 작업당 지연 46~48 ms, 절감 72.6~74.0%에서 안정하다. LSTM에서 작업당 지연으로 맞춘 짝 α=0.5687을 다른 두 예측기에 그대로 주어 보았다. 이

값은 아래에서 첫 주로 고른 실행 시간 가중 α와 우연히 같다. 24시간 전 값에서는 72.58%(작업당 46.4 ms), 완전예지에서는 73.97%(작업당 46.0 ms)를 낸다.

같은 지연의 고정 α는 1년을 다 보아야만 찾을 수 있는 값이 아니다. 채점 구간의 첫 주(1월 8~14일)만 보고, 무릎점과 같은 실행 시간 가중 지연이 나오도록 고른 고정 α는 0.5687이다. 이 값은 연간 72.86%(45.61 ms)로, 무릎점(71.93%, 47.48 ms)보다 더 줄이고 지연도 짧다. 다만 첫 주의 예측 입력에는 준비 구간의 되풀이 자료가 섞여 있다. 그래도 무릎점을 기본 운용 규칙으로 두는 것은, 운영자가 가중치를 정하거나 첫 주를 돌려 맞추지 않아도 슬롯마다 탄소와 지연 전선의 한가운데를 고르기 때문이다. 그 대가는 첫 주 고정 α 대비 약 0.9%포인트, 같은 지연의 고정 α 대비 1.77~2.64%포인트다. 합성 워크로드에서 무릎점이 같은 지연의 고정 α보다 낮지 않은 까닭의 하나는 식 (8)에 있는 것으로 보인다. 이 식은 탄소 항을 그 슬롯에 발행된 예측의 최댓값, 곧 슬롯마다 다른 분모로 나눈다. 그래서 같은 α라도 슬롯마다 kg과 ms의 교환비가 다르다. 슬롯별 무릎점은 여기에 슬롯마다 다른 α까지 더해, 교환비가 서로 다른 점들을 모은다. 같은 이유로 고정 α 열한 점을 이은 곡선도 연간 파레토 전선은 아니다.

무릎점 규칙이 슬롯마다 고른 α는 가운데 몇 값에 몰리고, 시간대에 따라 조금 움직인다. α는 슬롯마다 하나를 골라 그 슬롯의 모든 리전과 작업에 같이 쓴다. 이 실행의 리전 선택은 4.3절에 적은 그대로다(옮긴 작업 58.4%, 실행 몫 프랑스 51.6%, 캘리포니아 27.3%). 슬롯마다 고른 α는 평균 0.556이고, 채점 구간 8,592 슬롯 가운데 0.6이 48.3%, 0.5가 40.7%, 0.7이 6.3%, 0.4가 4.1%다. 시간대(UTC)별 평균은 3~8시 0.52~0.53, 13~17시 0.58~0.61이다.

이 분포는 예측 곡선의 모양에 따라 움직인다. 완전예지에서는 LSTM보다 0.5를 고른 슬롯이 늘고, 아시아의 낮인 UTC 3~10시에 평균 α가 LSTM의 0.53에서 0.47~0.52로 가장 많이 내려간다. 완전예지의 예측 곡선은 저탄소 구간이 뾰족해 탄소 축의 정규화가 달라지고, 그래서 무릎점이 더 낮은 α를 고른다.

4.2절에서 무릎점의 순서가 뒤집힌 것도 대부분 이렇게 무릎점이 고른 α의 차이에서 온다. 완전예지 무릎점(67.20%)은 LSTM 무릎점(71.93%)보다 4.72%포인트 낮다. 이 차이를 가르기 위해, 한 실행이 슬롯마다 고른 α의 열(이하 α 경로)을 다른 예측기의 실행에 그대로 주었다. LSTM 예측에 완전예지의 α 경로를 주면 69.23%(작업당 43.8 ms), 완전예지 예측에 LSTM의 α 경로를 주면 71.33%(작업당 45.2 ms)다. 두 순서를 평균하면 차이의 약 72%가 α 경로에서, 약 28%가 예측에서 온다. 상한을 없앤 용량 무시 실행에서도 무릎점 절감은 LSTM의 73.39%에서 완전예지의 69.25%로 내려간다. 따라서 이 차이는 용량이 아니라 무릎점이 고르는 자리에서 온다.

용량을 모든 대기 작업이 함께 보는 제약, 곧 공유 제약은 몰림을 지우지 않고 해롭지 않게 만든다. 완전예지 예측에 LSTM의 α 경로를 준 실행은 리전-일마다 탄소가 가장 낮은 시각의 최대 동시 실행이 평균 6.67건으로, LSTM 실행의 4.68건보다 많다. 그 시각에 상한에 닿은 리전-일도 23.92%로 LSTM 실행의 11.31%의 두 배다. 그런데 절감은 71.33%로 LSTM과 0.59%포인트밖에 차이 나지 않고 위반은 0이다. 몰린 작업은 저탄소 구간을 상한까지 채우는 데서 멈추고, 나머지는 다음으로 깨끗한 자리로 간다.

합성 워크로드의 네 주에서 완전예지·고정 α로 재면, 매 슬롯 다시 풀고 지금 시작할 작업만 확정하는 온라인 구조가 내주는 몫은 0.13~0.64%포인트로 작다. 이 몫은 한 주를 한 번에 푼 하한, 곧 참조 문제 (P0)를 한 주로 자르고 용량을 슬롯 단위로 완화한 문제와 비교해 잰다. 계절이 다른 네 주(2월 중순, 4월 하순, 7월 초, 10월 중순; 합성, 이용률 8.4%)마다 그 주에 제출된 작업을 상한 17 아래에서, 탄소집약도를 실측 그대로 아는 완전예지로 한 문제로 풀었다. 이 문제는 시작 후보를 마감까지 전부 연다. 용량은 슬롯을 통째로 덮는 실행만 세고(슬롯의 일부만 덮는 실행은 그 슬롯에서 세지 않는다), 그 주 이전의 점유는 넣지 않는다. 그래서 같은 격자 위의 어떤 배정보다도 배출이 낮다. 지연은 연간 실행에서 그 주 작업이 쓴 실행 시간 가중 지연을 예산으로 두어, 같은 지연에서 배출만 최소화한다. 하한과의 격차(각 주 기준 배출 대비)는 CAST 무릎점이 완전예지로 1.87~2.77%포인트, LSTM으로 3.11~6.49%포인트다. 탄소만 보는 α=1에서는 완전예지로 0.27~1.14%포인트, LSTM으로 0.33~1.47%포인트다. 같은 네 주에서 무릎점과 같은 지연의 고정 α 짝(LSTM α=0.5750, 완전예지 α=0.5375)의 격차는 완전예지로 0.13~0.64%포인트, LSTM으로 1.19~2.36%포인트다. 도착 고정형과도 같은 네 주에서 LSTM, 고정 α, 예약 3으로 견주었다. 격차는 CAST(α=0.5375)가 1.16~2.22%포인트, 도착 고정형(α=0.5437)이 2.56~3.63%포인트로, 네 주 모두 CAST가 하한에 더 가깝다. 따라서 다시 풀기와 확정 방식이라는 온라인 구조의 대가는 작다(합성 작업의 시작 창이 모두 24시간 안이라 지평의 대가는 이 비교로 잴 수 없다). 무릎점의 격차는 대부분 무릎점 규칙에서, LSTM 쪽의 더 큰 격차는 예측 오차에서 온다. 하한은 통째로 덮는 실행만 세는 완화라 실제 최적보다 낮을 수 있어, 이 격차는 크게 잰 값이다.

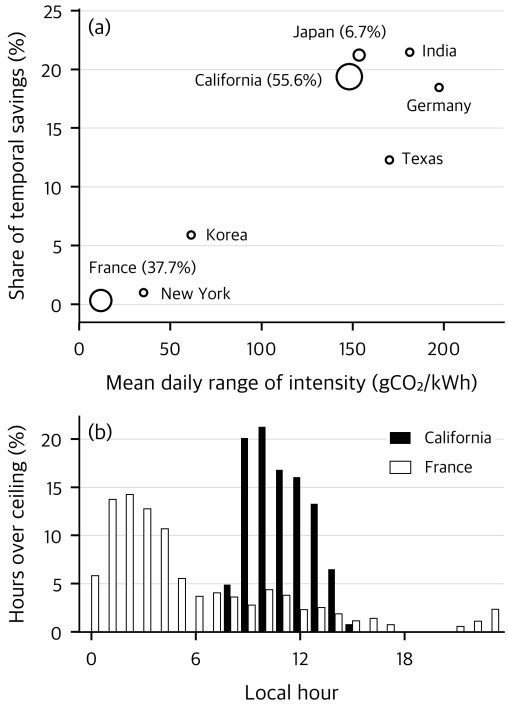


그림 10 상한이 걸리는 자리와 시간 이동의 절감이 큰 자리. (a) 여덟 리전의 하루 절대 변동(가로, 채점 구간의 일별 최대−최소 평균)과 시간 이동 절감 몫(세로, 시간만의 출발 리전별 감소량)이며, 원 넓이는 용량 무시(무릎점)에서 상한 17을 넘긴 시간의 몫에 따라 커지며 괄호 안이 그 몫이고, 작은 ○는 넘긴 시간이 거의 없는 리전이다. (b) 같은 실행에서 상한을 넘긴 시각의 현지 시간(일광절약시간 반영) 분포로, 리전별 합을 100%로 두었고 채운 막대가 캘리포니아, 빈 막대가 프랑스이며 0.5% 미만 막대는 생략했다.

Fig. 10 Where capacity binds and where temporal shifting pays. (a) Daily absolute swing (x; mean daily max−min over the scored window) against share of temporal-shift savings (y; temporal-only reduction by origin) for the eight regions; circle area grows with the share of hours over the ceiling of 17 under capacity ignored (knee α), given in parentheses, and small circles mark regions that barely exceed it. (b) Local-hour distribution of those hours (daylight saving applied), normalised per region; California filled, France hollow, with bars below 0.5% omitted.

4.6 상한이 걸리는 자리

평균 배출계수로 셀 때 상한은 주로 캘리포니아와 프랑스에서 걸린다. 이는 용량 무시 실행에서 상한을 넘긴 시간을 리전별로 나누어 보면 드러난다. 상한이 모든 리전에 같으면 작업이 모이는 리전에서 넘치는 것은 거의 정해진 일이다. 실제로 상한을 넘긴 시간의 55.6%가 캘리포니아, 37.7%가 프랑스, 6.7%가 일본에서 나며, 나머지 리전은 거의 넘기지 않는다(인도 0.02시간). 캘리포니아와 프랑스 두 리전이 93.3%다. 리전마다 상한을 달리 두어도 이 두 리전의 비중이 가장 큰지는 4.7절에서 본다.

상한이 걸리는 이 자리를 시간 이동의 절감이 큰 자리와 견주어 보면, 시간 이동의 절감은 하루 안의 절대 변동이 큰 리전에서 난다(그림 10(a)). 홈 리전을 고정하고 시각만 옮긴 시간만의 절감 5,377 kg을 출발 리전별로 나누면 인도 21.5%, 일본 21.2%, 캘리포니아 19.4%, 독일 18.5%, 텍사스 12.3%다. 인도와 일본은 자료 산정 방식이 바뀐 것으로 보이는 리전이라(4.2절) 이 순위는 자료에 기댄다. 이 다섯은 하루 안의 절대 변동, 곧 날마다 최댓값과 최솟값의 차를 평균한 값이 148.1~197.4 gCO_2/kWh로 가장 큰 다섯 리전이고, 시간 이동 절감의 92.8%를 낸다. 절대 변동이 클수록 절감이 큰 것은 작업당 1 kW로 절대 배출량을 세는 회계의 정의에서 따라 나온다. 캘리포니아는 날마다 최댓값이 최솟값의 평균 3.93배다. 연평균(채점 구간)은 인도 543.2 gCO_2/kWh에서 프랑스 13.6 gCO_2/kWh까지 약 40배 벌어진다. 프랑스의 상대 변동, 곧 날마다 (최댓값−최솟값)/일평균을 1년 평균한 값은 0.97로 캘리포니아(1.06) 다음으로 크다. 그러나 연평균이 13.6 gCO_2/kWh로 낮아 절대 변동이 11.8로 가장 작고, 시간 이동 절감의 0.3%만 낸다.

상한이 걸리는 리전과 시간 이동의 절감이 큰 리전, 두 목록에 함께 드는 리전은 캘리포니아와 일본이다. 캘리포니아는 CAST에서 실행된 몫의 27.3%를 받는 목적지이면서 시간 이동 절감의 19.4%를 내는 출발지이고, 용량을 두지 않으면 초과 시간의 55.6%가 나는 리전이다. 일본은 시간 이동 절감의 21.2%와 초과 시간의 6.7%를 낸다. 모든 리전에 같은 상한을 두면 상한이 공간 이동이 모이는 곳에서 걸리는 것은 상한의 정의에서 거의 따라 나온다. 시간 이동의 절감이 절대 변동이 큰 곳에서 나는 것도 kg 회계에서 거의 따라 나온다. 이 절의 측정이 보태는 것은 셋이다. 두 목록이 캘리포니아와 일본(나라 단위 구역 자료)에서 겹친다는 것, 인도, 독일, 텍사스에서는 상한이 거의 걸리지 않는다는 것, 그리고 상한 배치를 바꿔도 캘리포니아와 프랑스의 비중이 가장 크다는 것(4.7절)이다.

캘리포니아와 프랑스는 넘치는 시각도 다르다(그림 10(b)). 캘리포니아는 상한을 넘긴 시간이 모두 일광절약시간을 반영한 현지 8시부터 18시 전 사이에 있고, 그 74.2%가 9시부터 13시 전까지 네 시간에 몰린다.

표 11 결론이 유지되는 범위(합성, CAST). 모든 행에서 위반은 0이고 —는 돌리지 않은 칸이다. 현실 정보 행은 슬롯 안 도착을 미리 알지 않고, 지금 슬롯 탄소는 한 시간 전 예측, 기상 세 리전은 24시간 전 값을 쓴 것이다. 무릎점 열은 행마다 지연이 다르다. Cost of cap은 상한 없음 무릎점 − CAST 무릎점(%포인트)이며 음수면 CAST가 덜 배출한다. 회계 행의 계산식은 4.1절에 있고, 전송 행의 Cost of cap은 목적지 회계다.

Table 11 Robustness (synthetic, CAST). No row has a violation and — marks runs not performed; the realistic-information row drops in-slot arrival lookahead, uses the one-hour-old forecast for the current slot and SN-24h for the three weather regions; knee latency differs by row. Cost of cap is the no-cap knee saving minus CAST's knee saving (pp), negative when CAST emits less. Accounting formulas are in Section 4.1; the transfer row's cost uses destination accounting.

Condition	Knee (%)	α=1 (%)	Cost of cap (pp)
Main (LSTM, cap 17)	71.93	95.65	1.47
Forecaster: SN-24h	64.44	95.59	1.86
Forecaster: oracle	67.20	95.87	2.05
5 workload draws	71.51±0.43	95.40±0.28	1.83±0.53
Cap 7 (0.5×)	56.79	76.30	16.61
Cap 13 (1×)	70.16	94.18	3.24
Cap 26 (2×)	73.21	95.90	0.19
Cap 39 (3×)	73.42	95.96	−0.03
Per-region caps, 3 layouts	63.75–73.05	87.38–95.77	0.35–9.64
Arrivals ×2 (cap 23)	69.45	92.58	3.85
Unequal arrival rates	74.53	95.42	0.71
Deferrable share 20–60%	70.01–72.54	93.35–95.70	1.10–3.09
Arrival peak at noon	70.26	95.58	0.31
Horizon 6–12 h	71.04–71.63	95.58–95.65	0.93–1.18
No carbon lookahead	69.50	95.48	2.54
SN-24h in 3 weather regions	71.55	95.66	1.32
RTT cap 100 ms	32.98	46.83	0.11
Transfer 1 GB, dest. / origin	71.16 / 66.01	95.20 / 87.65	1.46
Life-cycle approx.	67.07	90.28	1.33
Marginal: France only	38.99	18.11	1.03
Marginal: symmetric	36.15	61.51	0.04
PUE 1.1–1.6, adverse / favorable (ρ=0)	68.66 / 74.54	—	—
Realistic info (ρ=0)	68.88	—	—

태양광이 만드는 한낮의 저탄소 구간으로 작업이 함께 모인 결과다. 프랑스는 반대로 현지 밤에 넘친다. 가장 큰 시간대가 2시로 14.3%이고, 0시부터 5시 전까지가 57.3%다. 프랑스의 하루 최저는 수요가 낮은 밤에 오고, 독일·인도·뉴욕에서 온 작업이 그 시각으로 모인다.

4.7 결론이 성립하는 범위

이 절에서 조건을 바꾼 합성 실행에서는 CAST의 위반이 모두 0이다. 바꾼 조건은 표 11에, 회계를 바꾼 같은 지연 짝은 표 12에 모았다. 아래는 그 가운데 설명이 필요한 것들이다.

상한을 바꾸어 결과가 상한값에 얼마나 매였는지 보았고, 어느 상한에서도 CAST는 위반이 없다. 상한을 모든 리전에 같게 가장 큰 기준 피크 13의 0.5배(7)부터 3배(39)까지 일곱 단계로 바꾸어 CAST를 돌렸다. 예약 ρ는 미룰 수 없는 작업이 준비 구간에서 필요로 한 자리이므로 상한과 무관하게 모든 단계에서 3으로 고정했다. 일곱 단계 모두 위반과 초과 시간이 0이다. 가장 큰 기준 피크의 절반인 상한 7에서는 홈 리전 즉시 실행조차 713번 상한을 넘기지만, CAST는 무릎점에서 56.79%를 줄이며 한 번도 넘기지 않는다.

절감은 상한을 늘릴수록 오르다가 가장 큰 기준 피크의 2배 근처에서 멈춘다. 무릎점 절감은 상한 7, 10, 13, 17, 20, 26, 39에서 56.79%, 66.11%, 70.16%, 71.93%, 72.72%, 73.21%, 73.42%다. 3배에서는 73.42%로 용량 무시(73.39%)를 0.03%포인트 넘는다. 무릎점이 상한에 따라 α를 다시 고르기 때문이며, 용량 무시가 CAST 절감의 위 끝이 되는 것은 같은 고정 α에서뿐이다. 용량을 지키는 비용, 곧 용량 무시와의 차는 대표 상한 17에서 1.47%포인트, 가장 큰 기준 피크와 같은 13에서 3.24%포인트, 절반인 7에서 16.61%포인트이고, 2배인 26에서는 0.19%포인트로 거의 사라진다. α=1에서도 상한 7, 10, 13에서 76.30%, 89.71%, 94.18%, 20에서 95.80%, 39에서 95.96%로 같은 꼴이며, 여섯 판 모두 위반이 0이다.

상한을 조여도 무릎점이 고르는 α는 거의 움직이지 않는다. 슬롯마다 고른 α의 평균은 상한 7에서 0.530, 17에서 0.556, 39에서 0.554다. 상한이 조이면 무릎점이 더 낮은 α를 고르는 것이 아니라, 갈 자리가 없는 작업이 홈 리전에 남는다. 그래서 홈 리전에서 도는 작업이 상한 39의 41%에서 상한 7의 50%로 는다. 리전을 도착 때 고정하는 분리형은 상한을 풀어도 무릎점 절감이 상한 7~26에서 48.44~49.71%에 머문다. 분리형의 무릎점은 α를 0.44 안팎으로 두어 작업을 홈 리전에 남기므로, 상한이 결과를 정하지 않는다.

표 12 회계별 같은 지연 짝(예약 없이, 고정 α의 짝). Δ는 상대 설계 배출 − CAST 배출(기준 배출 대비 %포인트, 양수면 CAST가 덜 배출)이고 괄호는 7일 블록 부트스트랩 95% 신뢰구간이다. 배정은 평균 배출계수로 정한 그대로 두고 기준 배출까지 다시 셌으며, %포인트의 분모는 회계마다 다시 센 기준 배출이고 계산식은 4.1절에 있다. 불리한 가정을 겹친 행은 프랑스만 한계 근사, 출발지 전송 1 GB, PUE 불리를 함께 두되 PUE는 실행 전력에만 곱했다. CAST는 두 워크로드 모두 α=0.5375(Azure 49.56%)로 Azure 무릎점(51.35%)과 다르다.

Table 12 Same-latency pairs under each accounting (no reservation, fixed-α pairs). Δ is the other design's emissions minus CAST's (percentage points of each workload's baseline; positive = CAST emits less), with 95% seven-day block-bootstrap intervals. Assignments are those made with average factors; emissions and baselines are rescored, and each row's pp use its own rescored baseline (formulas in Section 4.1). The adverse combination stacks France-only marginal, origin transfer 1 GB and adverse PUE, with PUE applied to execution power only. CAST uses α = 0.5375 on both workloads (Azure 49.56%), not the Azure knee (51.35%).

Accounting	Synthetic, arrival-fixed	Synthetic, evacuation	Azure, arrival-fixed	Azure, evacuation
Average	+0.96 (0.51–1.46)	+0.91 (0.67–1.15)	+2.53 (1.68–3.42)	+1.99 (0.86–3.01)
Exact overlap	+0.88 (0.42–1.37)	+0.80 (0.55–1.04)	+2.52 (1.67–3.41)	+1.98 (0.85–3.01)
Transfer 1 GB, dest.	+1.03 (0.57–1.52)	+0.97 (0.73–1.21)	+2.50 (1.66–3.40)	+1.97 (0.83–2.99)
Transfer 1 GB, origin	+1.01 (0.57–1.50)	+0.88 (0.65–1.11)	+2.59 (1.73–3.48)	+2.01 (0.89–3.03)
Life-cycle approx.	+0.95 (0.53–1.41)	+0.94 (0.71–1.17)	+2.40 (1.60–3.23)	+1.85 (0.79–2.81)
Marginal: France only	−0.89 (−1.35 to −0.43)	−2.15 (−2.62 to −1.68)	+1.48 (0.78–2.17)	−0.16 (−0.93 to 0.55)
Marginal: symmetric, floor 0.05	+1.19 (0.61–1.88)	+1.58 (0.90–2.39)	+2.05 (1.40–2.75)	+0.87 (0.11–1.67)
floor 0.02	+0.84 (0.38–1.43)	+0.95 (0.40–1.62)	+1.59 (1.07–2.12)	+0.30 (−0.28 to 0.89)
floor 0.1	+1.33 (0.70–2.02)	+1.89 (1.19–2.69)	+2.47 (1.73–3.24)	+1.34 (0.40–2.26)
floor 0.2	+0.82 (0.31–1.40)	+1.38 (0.93–1.88)	+2.72 (1.96–3.51)	+1.75 (0.72–2.75)
Marginal: symmetric, floor hours at median (FR ≈ 400 g)	+0.47 (0.15–0.84)	+0.42 (0.07–0.83)	+1.38 (0.96–1.81)	+0.08 (−0.40 to 0.61)
PUE adverse (FR, CAL 1.6)	+1.07 (0.66–1.49)	+1.19 (0.97–1.43)	+2.32 (1.57–3.07)	+2.21 (1.20–3.16)
PUE favorable (FR, CAL 1.1)	+0.89 (0.39–1.43)	+0.70 (0.43–0.98)	+2.68 (1.74–3.69)	+1.83 (0.59–2.98)
Adverse combined (France only + origin 1 GB + PUE adverse)	−1.49 (−2.12 to −0.86)	−3.13 (−3.72 to −2.54)	+0.89 (0.30–1.47)	−0.79 (−1.76 to 0.11)

예약이 없어도 대표 상한 17에서는 위반이 나지 않는다. 예약 ρ를 0, 2, 3, 4, 6으로 바꾸면 절감은 72.06%, 72.16%, 71.93%, 72.00%, 71.53%로 ρ=3과 0.40%포인트 안에서 오르내리고, 위반은 모두 0이다. 매 슬롯 지금 시작하는 작업(i=0)만 확정하므로 아직 확정하지 않은 미래 슬롯의 계획은 언제든 양보할 수 있고, 이미 시작해 돌고 있는 작업만으로 슬롯이 찬 때가 없었기 때문이다. 예약이 바꾸는 것은 강제 시작, 곧 미룰 수 있는 작업 가운데 마감까지 남은 여유가 한 슬롯 아래로 떨어져 그 자리에서 시작해야 했던 작업의 수다. ρ를 0에서 6으로 늘리면 강제 시작은 4,805건에서 3,931건으로 준다. 상한을 조여도 마찬가지다. 상한 7과 10에서 예약을 0으로 두면 절감은 58.30%와 66.85%로 예약 3일 때보다 1.51%포인트와 0.74%포인트 높고, 위반은 여전히 0이다. 대신 강제 시작이 상한 7에서 13,479건으로, 예약 3일 때(4,625건)의 세 배 가까이 된다. 어느 상한에서도 위반이 없는 것은 예약에 기대지 않는다. 예약은 위반을 막은 적이 없으며, 마감 직전으로 밀리는 작업을 줄이는 선택 사항으로 대표 실행에 남겼다. Azure에서는 예약을 빼면 강제 시작이 5,491건에서 10,815건으로 두 배가 되지만 절감이 5.20%포인트 늘어, 이 워크로드에서 예약은 쓸모를 보이지 못했다.

같은 α에서 CAST와 분리형의 차는 상한을 조일수록 벌어진다. α=1에서 CAST와 분리형의 차는 대표 상한 17에서 0.32%포인트(표 9, 모두 예약 3)이고, 13에서 0.89%포인트, 10에서 1.77%포인트, 7에서 3.04%포인트다. 리전을 도착 때 고정하면 그 리전의 자리가 찼을 때 다른 리전으로 옮겨 갈 수 없기 때문으로 보인다. 무릎점끼리는 두 설계가 고르는 α가 달라 같은 α의 비교가 되지 않으므로 여기서 견주지 않는다.

리전마다 상한을 달리 두어도 CAST는 위반이 없고, 용량 무시의 배정이 상한을 넘기는 시간은 여전히 캘리포니아와 프랑스에서 가장 많다. 상한의 총합을 113~114로 거의 같게 두고 세 가지로 나누었다. CAST 무릎점 절감은 리전마다 자기 기준 피크의 1.25배를 주면 70.04%, 깨끗한 세 리전(캘리포니아·프랑스·뉴욕)을 크게 주면 73.05%, 작게 주면 63.75%이고, 모두 위반이 없다. 같은 세 배치로 용량 무시의 배정을 채점해 상한을 넘긴 시간을 리전별로 나누었다. 자기 피크의 1.25배에서 캘리포니아 47.5%·프랑스 46.6%, '작게'에서 프랑스 57.5%·캘리포니아 40.1%, '크게'에서 캘리포니아 44.8%·프랑스 31.4%·일본 21.3%로, 어느 배치에서도 캘리포니아와 프랑스의 비중이 가장 크다. 다만 시간만의 리전별 절감은 상한 17에서 잰 것이라, 상한 배치를 바꿀 때 그 목록이 그대로인지는 재지 않았다.

워크로드를 다시 뽑아도 절감은 거의 흔들리지 않고 설계의 순서도 같다. 워크로드 추첨에 따른 변동은 합성 워크로드를 시드 넷으로 다시 뽑아 잰다. 시드마다 가장 큰 기준 피크가 달라 상한과 예약도 같은 규칙으로 다시 정했으므로(상한 14~17, 예약 2~3), 아래 폭에는 상한의 변동도 들어 있다. 주 실험을 포함한 다섯 벌에서 CAST 무릎점은 71.51 ± 0.43%, α=1은 95.40 ± 0.28%, 분리형은 49.01 ± 0.28%, 공간만은 61.20 ± 0.22%, 시간만은 18.72 ± 0.05%, 용량 무시는 73.34 ± 0.27%다(평균 ± 표준편차). 공유 제약이나 다시 풀기를 쓰는 실행에서는 모두 위반이 0이다. 무릎점끼리의 차는 CAST와 분리형이 22.50 ± 0.66%포인트, CAST와 공간만이 10.32 ± 0.34%포인트이며 다섯 벌 모두에서 순서가 같다. 이 차의 대부분은 설계가 아니라 무릎점 규칙이 설계마다 다른 α를 고른 결과이며, 같은 α에서의 설계 차는 표 9에 있다.

예측 지평을 줄여도 절감은 거의 그대로이고 대기 시간만 짧아진다. 지평을 6, 12, 24시간으로 바꾸면 무릎점 절감은 71.63%, 71.04%, 71.93%로 0.9%포인트 안에서 움직이고 위반은 모두 0이다. 지평이 짧을수록 미룰 수 있는 범위가 좁아져 평균 대기 시간이 0.89, 1.25, 1.47시간으로 짧아진다.

왕복 지연에 상한을 두면 결과가 크게 바뀐다. 식 (5)의 왕복 지연 상한을 모든 작업에 50, 100, 150 ms로 두면 무릎점 절감은 28.41%, 32.98%, 66.37%, α=1 절감은 38.09%, 46.83%, 89.85%다. 모두 지연 상한이 없을 때(71.93%, 95.65%)보다 낮고, 위반은 모두 0이다. 50 ms와 100 ms에서는 한국, 일본, 인도에서 출발한 작업이 가장 깨끗한 캘리포니아와 프랑스에 닿지 못한다(한국·일본은 캘리포니아까지 109~134 ms, 인도는 프랑스까지 137 ms). 그래서 홈 리전에 남는 작업이 84.2%와 78.2%에 이르고, 절감이 지연 상한이 없을 때의 절반 아래로 꺾인다.

미룰 수 있는 작업의 비율 φ를 20%에서 60%까지 바꾸어도 결론은 그대로다. 등급 안의 비율은 유지하고, 상한과 예약은 각 φ의 가장 큰 기준 피크에서 같은 규칙으로 다시 정했다. CAST 무릎점은 φ 20, 30, 40, 50, 60%에서 72.30%, 72.54%, 71.93%, 71.45%, 70.01%, 분리형은 47.97~49.98%이고, 두 설계 모두 모든 φ에서 위반이 없다. 미룰 수 있는 작업이 늘수록 용량 무시의 위반은 늘어, 각 φ의 상한으로 세어 15,233건에서 46,994건으로 세 배가 된다. 용량을 지키는 비용은 1.10~3.09%포인트로 φ=60%에서 가장 크다. 미룰 수 있는 작업이 많을수록 같은 저탄소 구간으로 더 몰린다.

미룰 수 있는 작업의 도착 봉우리를 현지 새벽 2시에서 한낮 13시로 옮겨도 결론은 같다. 가장 큰 기준 피크가 바뀌어 상한은 같은 규칙으로 23이 되며, CAST는 70.26%를 위반 없이 줄인다. 시간만은 18.78%에서 13.92%로, 용량을 지키는 비용은 1.47%포인트에서 0.31%포인트로 작아진다. 그래도 시간 이동의 절감이 큰 출발 리전은 인도, 독일, 일본, 캘리포니아로 거의 같다.

작업 수와 리전별 발생률을 바꾸어도 결론은 같다. 리전당 하루 도착을 100건으로 두 배 늘리면 가장 큰 기준 피크가 18, 상한이 23으로 커지며, CAST는 무릎점에서 69.45%, α=1에서 92.58%를 위반 없이 줄인다. 용량 무시는 73.30%를 내지만 위반이 81,799건, 최대 동시 실행이 122건이며, 상한의 정수 단위가 커져도 용량을 지키는 비용은 3.85%포인트로 오히려 커진다. 미국 세 리전에 하루 80, 70, 70건, 나머지 다섯 리전에 30~40건을 주어 발생률을 달리하면(하루 합은 같다) 상한은 18이 되고, CAST는 무릎점 74.53%, α=1 95.42%를 위반 없이 내며 용량을 지키는 비용은 0.71%포인트다.

Azure 워크로드에서도 공유 제약이나 다시 풀기를 쓰는 실행(CAST, 분리형, 도착 고정형, 대피형 등)은 모두 위반 없이 돈다. 용량 행을 지운 두 실행만 위반을 낸다. 용량 무시는 위반 61,400건, 초과 시간 5,911시간을 내고 최대 동시 실행이 107건으로 상한 43의 2.5배에 이르며, 용량 행을 지운 분리형은 30,599건이다.

결과의 모양은 합성과 다르다(이하 Azure는 예약 없이). 시간 이동으로 얻을 것이 거의 없고, 용량을 지키는 비용이 훨씬 크다. 작업이 길고 미룰 수 없는 작업시간이 51%라, 시간만은 1.53%를 줄이는 데 그친다. 공간만(52.10%)은 CAST 무릎점(51.35%)보다 0.76%포인트 더 줄인다. 다만 실행 시간 가중 왕복 지연을 7.2 ms 더 쓰므로, 지연이 다른 무릎점끼리의 차다. CAST는 분리형보다 무릎점에서 4.46%포인트(46.89% 대 51.35%), α=1에서 2.22%포인트(65.52% 대 67.74%) 더 줄인다. 트레이스를 다시 뽑은 두 시드(상한 47, 39)에서도 CAST 무릎점은 분리형보다 5.92%포인트와 2.80%포인트, 도착 고정형보다 2.53%포인트와 1.38%포인트 더 줄인다(지연이 다른 무릎점끼리, 위반은 모두 0). 준비 구간 규칙으로 정한 예약(13~15)에서는 분리형과의 차가 시드에 따라 부호가 바뀌었다. 반대로 용량을 지키는 비용은 훨씬 크다. 탄소만 보는 α=1에서 CAST(67.74%, 작업당 지연 115.1 ms)와 용량 무시(95.11%, 120.5 ms, 위반 139,527건)의 차는 27.37%포인트로, 합성의 0.34%포인트(예약 없이)와 크게 다르다. 합성은 평균 동시 실행 11.5건이 가장 깨끗한 프랑스 한 리전의 상한 17 안에 들지만, Azure는 89.9건이 프랑스의 상한 43을 넘기 때문이다. CAST와 용량 무시의 무릎점끼리의 차 11.35%포인트(51.35% 대 62.70%)는 실행 시간 가중 왕복 지연이 12 ms 다른 두 실행의 차다. VM 크기를 전력에 반영해 코어 수에 비례하게 다시

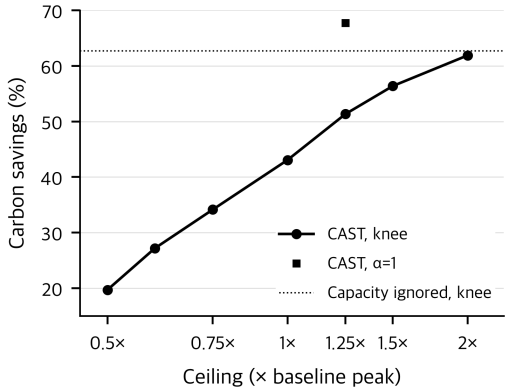


그림 11 Azure 워크로드에서 상한에 따른 절감(예약 없이, ρ=0). 가로는 상한을 가장 큰 기준 피크 34의 배수로 적은 것이며(로그 눈금, 0.5~2배), ●와 실선은 CAST의 무릎점, ■는 대표 상한(1.25배)에서의 CAST α=1, 검은 점선은 용량 무시(무릎점)다. 위반은 0.5배에서만 0.32%이고(4.4절) 나머지 점은 0이다. 계열 이름은 표 6·표 7과 같다.

Fig. 11 Savings against the capacity ceiling on the Azure workload (no reservation, ρ=0). The x axis is the ceiling as a multiple of the baseline peak of 34 (log scale, 0.5× to 2×); filled circles with the solid line are CAST at the knee, the square is CAST at α=1 at the representative ceiling (1.25×), and the black dotted line is capacity ignored at the knee. Violations occur only at 0.5× (0.32%, Section 4.4); all other points have none. Series names follow Tables 6 and 7.

채점해도(배정은 그대로) Azure의 절감은 많아야 1.1%포인트 움직이고 비교의 부호는 모두 같다. 합성과 달리 무릎점의 순서가 예측기에 따라 뒤집히지도 않는다. 완전예지 51.81%, LSTM 51.35%, 24시간 전 값 50.98%로 0.83%포인트 안에 있다.

상한과 예약에 대한 반응도 다르다. 상한을 늘리면 절감이 멈추지 않고 오르며, 예약은 절감을 크게 깎는다. 상한을 가장 큰 기준 피크 34의 0.75배, 1배, 1.25배, 1.5배, 2배로 바꾸면 무릎점 절감은 34.15%, 43.07%, 51.35%, 56.39%, 61.89%다(그림 11, 0.5배와 0.6배는 4.4절의 높은 부하 조건). 합성과 달리 2배에서도 멈추지 않고, 용량 무시(62.70%)까지 0.80%포인트가 남는다. 예약 ρ를 6, 13, 19, 25로 바꾸면 절감은 50.44%, 46.14%, 42.49%, 39.88%로 준다. 13은 준비 구간 규칙으로 정한 값으로, 이전 슬롯에서 넘어온 긴 VM을 두 번 센 과대값이다(3.3절). Azure에서는 ρ가 상한 43의 14~58%라 예약이 미래 슬롯의 자리를 직접 깎아, 예약 13의 비용은 5.20%포인트로 합성(0.13%포인트)보다 훨씬 크다. 도착 고정형은 예약의 영향을 0.37%포인트만 받는다. 그래서 ρ=13에서는 무릎점끼리 도착 고정형이 CAST보다 앞선다(48.67% 대 46.14%, 실행 시간 가중 지연 4.0 ms 차). 그러나 ρ를 6으로만 줄여도 무릎점끼리 CAST가 50.44%로 앞서고, 예약을 빼면 CAST가 실행 시간 가중 지연을 3.6 ms 덜 쓰고도 2.31%포인트 앞선다(51.35% 대 49.04%, 4.4절).

4.8 선행 정책 재현 비교

CICS, CASPER, Sukprasert, Caspian의 정책을 원문의 수식을 따라 본 연구의 자료 위에 다시 구현하고 같은 회계(식 (1), 3.5절)로 채점하였다. 원문의 설정과 본 연구의 설정, 바꾼 이유는 표 13에 모았다. CASPER는 서버 대수를 함께 정하는 웹 서비스용 정책이다. 그래서 시각을 옮기지 않고, 본 연구의 상한 대신 스스로 정한 서버 용량 안에서 배정한다. 표 14의 필요 용량은 그 배정에서 나중에 잰 최대 동시 실행 수다.

비교의 자는 둘이다. 하나는 상한 17에서의 위반 건수이고, 다른 하나는 그 정책의 배정이 위반 없이 돌려면 필요한 리전 상한, 곧 필요 용량이다. 상한 17은 본 연구가 고른 값이므로, 선행 정책이 그 상한을 넘겼다는 사실만으로는 정책의 좋고 나쁨을 가릴 수 없다. 필요 용량은 본 연구가 정한 상한과 무관하게 각 정책이 스스로 요구한 값이며, 1초짜리 순간 피크의 영향을 가리기 위해 시간 가중 99.9 백분위수를 함께 적는다. 결과는 표 14에, 절감과 필요 용량을 두 축에 놓은 것은 그림 12에 있다. 그림 12에는 4.7절에서 상한을 바꾼 CAST의 곡선과 합성 워크로드의 용량 무시 수준(가로 점선)도 함께 그렸다.

표 14 선행 정책을 같은 회계로 재현한 결과(RTT는 실행 시간 가중 평균 왕복 지연). CICS, CASPER, Sukprasert는 상한 없이 돌렸고 Caspian은 자기 단위의 상한 17을 두므로, 위반은 참고값이고 비교는 필요 용량으로 한다. 위반은 상한 17을 넘긴 배정 건수, 마감은 마감 위반 건수이고, 필요 용량은 리전별 최대 동시 실행의 최댓값(괄호는 시간 가중 99.9 백분위수), 설비 총량(Total)은 리전별 최댓값의 합이며 가장 큰 기준 피크는 13이다. Suk.는 Sukprasert 등[6]의 정책으로 (home)은 홈 리전, (ours)는 CAST(무릎점)가 정한 리전 위에서 시각만 옮긴 것이다. CICS는 하루 전 자원 곡선, CASPER는 탄소 인지 프로비저닝(CAP) 정책, Caspian은 슬롯 안 자원량을 실행 시간으로 둔 판이다. CASPER 대비 CAST(α=1, 상한 17, 예약 3)의 차는 회계별로 평균 0.36, 정확 겹침 0.35, 전송 목적지 0.26, 출발지 0.45, 전과정 0.32, 한계 근사 프랑스만 2.02, 대칭 2.52%포인트다(예약 없는 CAST로는 평균 0.33%포인트).

Table 14 Prior policies reproduced under the same accounting (RTT is the duration-weighted mean round-trip latency). CICS, CASPER and Sukprasert run without our cap and Caspian with its own-unit cap of 17, so Viol. is for reference; the comparison uses required capacity. Viol. counts assignments past the ceiling of 17 and Deadl. deadline misses; Req. cap. is the largest per-region peak concurrency (time-weighted 99.9th percentile in parentheses) and Total the sum of per-region peaks; the baseline peak is 13. Suk. is the policy of Sukprasert et al. [6]: (home) shifts time on the home region and (ours) on the regions chosen by CAST at the knee. CICS uses the day-ahead capacity curve, CASPER its CAP policy, and Caspian the variant whose per-slot resource equals the run time. CAST (α=1, cap 17, reservation 3) leads CASPER by 0.36 (average), 0.35 (exact overlap), 0.26 (transfer, destination), 0.45 (transfer, origin), 0.32 (life-cycle), 2.02 (marginal, France only) and 2.52 pp (marginal, symmetric); without reservation the average gap is 0.33 pp.

Policy	Savings (%)	Viol.	Req. cap.	Total	Deadl.	RTT (ms)
Baseline	0	0	13 (8)	89	0	0
CICS	13.89	0	12 (9)	88	0	0
Suk. temporal (home)	22.22	3,180	27 (19)	194	0	0
Suk. temporal (ours)	73.23	37,236	82 (64)	267	0	47.48
CASPER	95.30	11,061	29 (24)	46	0	121.25
CASPER, cap 17	78.53	0	16 (13)	86	0	117.35
Suk. spatial+temporal	96.63	61,539	129 (107)	139	0	121.34
Caspian	67.54	1,538	34 (28)	86	5,582	84.52
CAST, knee	71.93	0	17 (17)	117	0	47.48
CAST, α=1	95.65	0	17 (17)	46	0	121.32

절감률만 보면 CASPER(95.30%)와 Sukprasert 공간+시간(96.63%)이 CAST의 무릎점(71.93%)을 앞서지만, 그 절감은 리전 하나에 큰 용량을 요구한다. 뒤엣것은 1년치 탄소집약도를 모두 미리 안다고 가정한 정책이다. 두 정책의 배정이 위반 없이 돌려면 리전 하나가 동시에 29건과 129건, 곧 가장 큰 기준 피크 13의 2.2배와 9.9배를 돌려야 한다. Sukprasert 공간+시간은 채점 작업 143,200건 가운데 142,841건(99.75%)을 프랑스에서 돌린다. 지연까지 같은 짝으로 좁히면 차이가 분명하다. CASPER와 CAST(α=1)는 둘 다 지연을 보지 않아 실행 시간 가중 왕복 지연이 121.25 ms와 121.32 ms로 거의 같고, 절감도 95.30%와 95.65%로 비슷하다. 그래도 일곱 회계(4.1절) 모두에서 CAST가 0.26~2.52%포인트 앞서며(회계별 값은 표 14 캡션), CAST(α=1)는 이 절감을 상한 17 안에서 낸다. CASPER에 같은 상한 17을 걸면 위반 없이 78.53%(필요 용량 16, 실행 시간 가중 왕복 지연 117.35 ms)를 낸다. 같은 상한과 거의 같은 지연에서 CAST(α=1)와의 차는 17.1%포인트다. 그러나 같은 상한의 공간만도 α=1에서 93.96%를 내므로, 이 차는 용량 인지의 몫이 아니라 CASPER가 시각을 옮기지 않고 시간 단위로 서버 수를 정하는 설계에서 온다. 상한을 걸면 CASPER도 프랑스 한 곳에 몰지 못하고 여러 리전으로 퍼져, 리전별 최댓값을 더한 설비 총량이 46에서 86으로 는다.

표 13 선행 정책과 도착 고정형(Zanotto 등)의 원문 설정과 본 연구의 재구현. 원 논문의 자료와 워크로드가 아니므로 원 수치의 재현은 확인하지 못했다(5장).

Table 13 Original settings of the prior policies and of the arrival-fixed design (Zanotto et al.), and our reimplementation. The original data and workloads were not available, so the original numbers were not reproduced (Section 5).

Policy	Original setting	This study	Reason / difference in objective and workload
CICS [3]	Day-ahead LP on CPU usage; carbon + peak-power cost; no per-job deadlines	Job counts; hourly allowance ≤ 2× daily mean; peak weight 0; day's actual carbon known; earliest deadline released first, last-chance jobs over the allowance	No published peak weight; our jobs have deadlines
CASPER [4]	ILP over carbon and active servers, weight 0.9; 10 jobs/server, 100 servers/region	Code values; hourly submissions as requests; actual carbon; no time shifting	Web-service provisioning; own capacity (8,000 jobs/h) never binds
Sukprasert [6]	Public-code search, one year of carbon known; no capacity; fixed slack ≥ 24 h	Same search at 1 h steps; per-job deadline as slack; temporal on home and on CAST regions, spatial+temporal over all (region, hour)	Upper-bound analysis, not an online policy
Caspian [8]	Hourly 24-slot LP relaxation, three algorithms; whole-slot run time; soft deadlines; weights 1	Transcribed (no code); per-slot resource = run time (≤ 1); hard cap 17; LSTM carbon; jobs 1 h past deadline start at home	Whole slots inflate sub-hour jobs; soft deadlines need a fallback
Zanotto [9] (arrival-fixed)	Per-request ILP at arrival against existing assignments; regional caps; hard deadlines; own forecaster; 1 month of 1,000-job batches	Jobs of one slot batched into CAST's LP with the knee rule; same LSTM, caps and costs; never re-solved	Batching avoids handicapping the original; same solver isolates the commit time

홈 리전을 고정하고 시각만 옮기는 정책끼리 좁혀 보아도 마찬가지다. 이 조건에서 CICS는 13.89%를 내며 필요 용량이 12로 기준보다도 낮다. 하루 전에 정한 자원 곡선 하나로 다음 날의 부하를 묶어 피크를 깎기 때문이다. 그 대신 개별 작업의 마감에 맞춰 시각을 고르지는 않는다. Sukprasert의 시간 이동은 같은 조건에서 22.22%를 내지만 필요 용량이 27로 가장 큰 기준 피크의 2.1배다. 본 연구의 시간만은 18.78%로 둘 사이에 있고 최대 동시 실행이 상한 17을 넘지 않는다(표 7).

Sukprasert의 시간 이동은 어느 리전 위에서 돌리느냐에 따라 두 행으로 갈린다. 홈 리전 위에서는 22.22%이지만, CAST(무릎점)가 정한 리전 위에서 시각만 옮기면 73.23%가 되어 CAST보다도 조금 더 줄인다. 그 차 51.01%포인트는 시간 이동이 아니라 공간 이동의 몫이다. 그러나 이때 필요 용량은 82건으로 가장 큰 기준 피크의 6.3배이고 위반은 37,236건이다. CAST가 상한까지 채워 둔 리전 위에서 Sukprasert의 시간 이동이 작업을 다시 같은 저탄소 구간으로 몰기 때문이다.

Caspian은 용량 17을 자기 슬롯 단위의 하드 제약으로 두고 마감을 양보한다. 절감은 67.54%이고, 마감을 넘긴 작업이 5,582건(3.90%)이다. 탄소 가중치를 0.5~2로 바꿔도 67.34~67.97%로 거의 같다(원문 초록의 33%는 가중치 0.5의 값이다). Caspian은 1시간 미만 작업을 실행 시간만큼의 분수로 세므로 자기 단위로는 상한을 지킨다. 그러나 시작 순간 단위로 다시 세면 1,538건이 상한을 넘고, 필요 용량은 34로 가장 큰 기준 피크의 2.6배다. CAST α=1은 마감과 상한을 모두 지키며 Caspian보다 28.11%포인트 더 줄인다. 다만 실행 시간 가중 왕복 지연은 121.32 대 84.52 ms로 더 쓴다. 지연이 더 짧은 CAST 무릎점(47.48 ms)도 4.39%포인트 더 줄인다. 원 논문은 경량 쿠버네티스 배포 도구(k3d)로 만든 클러스터 넷에 24시간 동안 작업 200건을 도착시켜 40시간을 평가했다[8].

필요 용량을 순간 피크를 가린 99.9 백분위수로 다시 세어도 정책의 순서는 같다. CICS 9, Sukprasert 시간(홈) 19, CASPER 24, Caspian 28, Sukprasert 시간(본 연구 리전) 64, Sukprasert 공간+시간 107이다. CAST는 두 가중치 모두 17로, 순간 피크와 99.9 백분위수가 같다. CAST가 상한까지 채운 시간이 짧은 순간이 아니라는 뜻이다. Caspian을 빼면 모든 정책에서 마감을 넘긴 작업은 없었다. 가장 붐빈 리전으로 재면, CAST 무릎점보다 많이 줄이는 세 정책은 가장 큰 기준 피크의 2.2~9.9배를 요구한다. 그러나 리전별 최댓값을 모두 더한 설비 총량(표 14의 Total)으로는 CASPER(46), Caspian(86), CICS(88)가 CAST 무릎점(117)보다 작다. α=1의 CAST는 46으로 CASPER와 같다. CASPER는 프랑스 29와 캘리포니아 17만 쓰고 나머지 리전은 비운다. 리전 용량이 데이터센터마다 정해져 있다는 본 연구의 가정에서는 한 리전의 크기가 맞는 척도다.

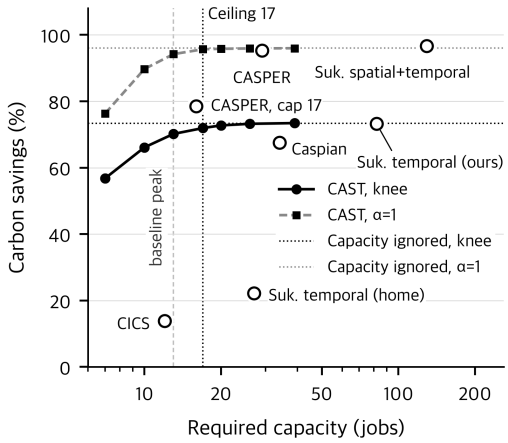


그림 12 선행 정책 재현 결과 비교(표 14). ○는 선행 정책이고 그중 CASPER, cap 17은 같은 상한 17을 건 CASPER(4.8절)다. ●와 ■는 상한을 가장 큰 기준 피크의 0.5~3배로 바꾼 CAST의 무릎점과 α=1을 이은 것이고, 가로 점선은 같은 두 가중치에서 용량을 무시한 절감(검은 선 무릎점 73.39%, 회색 선 α=1 95.97%)이다. 회색 세로 파선은 가장 큰 기준 피크 13, 검은 세로 점선은 상한 17이며 가로축은 로그 눈금이다.

Fig. 12 Comparison of reproduced prior policies (Table 14). Hollow circles are prior policies, where CASPER, cap 17 is CASPER under the same ceiling of 17 (Section 4.8); filled circles and squares connect CAST at the knee and at α=1 with the ceiling swept from 0.5× to 3× the baseline peak, and the horizontal dotted lines are the same two weights with capacity ignored (black: knee, 73.39%; gray: α=1, 95.97%). The vertical gray dashed line marks the baseline peak of 13 and the vertical black dotted line the ceiling of 17; the x axis is logarithmic.

4.9 계산 비용

CAST의 계산 시간은 슬롯 길이에 비하면 무시할 만하다. CAST는 무릎점을 고르려고 매 슬롯 선형 계획을 열한 번 푼다. 1년 8,760 슬롯 동안 96,360번이며, 분수해가 나와 정수 계획으로 다시 푼 슬롯은 341개(3.9%)였다. 한 슬롯의 문제가 가장 컸을 때 변수는 7,848개였고, 정수 계획의 상대 최적성 허용오차는 HiGHS 기본값인 0.01%다. 무릎점 탐색을 포함한 슬롯당 풀이 시간(후보 α 열한 개를 차례로 푼 값)은 평균 223 ms, 95 백분위수 380 ms, 최대 1,223 ms로 슬롯 길이 1시간에 비하면 무시할 만하다. 자료 적재와 채점까지 포함해 1년 전체를 4.1절의 기계 한 대에서 34.4분에 돌렸다. α 후보를 0.1 간격으로 둔 것은 정확도와 계산의 맞바꿈이다. 채점 구간 첫머리의 한 달(채점 작업 12,000건)에서 간격을 0.05로 좁혀 후보 스물한 개를 풀어 보았다. 절감은 66.19%에서 66.30%로 그 달 기준 배출의 0.11%포인트, 작업당 평균 지연은 1.1 ms 달라지지만, 풀이 시간은 6.4분에서 12.4분으로 두 배가 된다.

α=1은 슬롯마다 한 번만 풀지만 정수 계획이 필요한 슬롯이 1,421개(16.2%)로 늘고 최대 변수가 9,488개라 1년에 14.4분이 걸린다. 분리형은 13.6분, 시간만은 7.7분, 공간만은 3.5분이다. 선형 계획 없이 작업마다 차례로 자리를 잡는 탐욕은 같은 무릎점 규칙으로 100.3분이 걸리지만, 순수 파이썬 구현이라 HiGHS를 쓰는 CAST와 속도를 견주지 않는다. 작업시간이 합성의 7.8배인 Azure 워크로드(예약 없이)에서도 한 슬롯 문제의 최대 변수는 10,064개로 합성(7,848개)의 1.3배에 그친다. 변수 수가 작업 수가 아니라 대기열의 길이에 매이기 때문이다. 늘어난 것은 정수 계획으로 다시 푼 슬롯(663개, 7.6%)과 슬롯당 풀이 시간(평균 318 ms, 95 백분위수 580 ms, 최대 1,522 ms)이다. 1년 전체는 49.3분으로 합성의 1.4배다. Azure의 α=1은 최대 변수 11,160개, 정수 계획 슬롯 2,437개(27.8%), 26.7분이다. 합성 워크로드의 도착을 두 배로 늘리면 최대 변수가 15,816개로 두 배가 되고, 1년 풀이 시간은 85.6분으로 2.5배다(슬롯당 평균 556 ms, 최대 4,298 ms). 같은 지연의 고정 α에서 1년 풀이 시간은 CAST 5.3분, 도착 고정형 1.2분(Azure 14.2분 대 1.2분)이다.

5. 결론 및 향후 연구

본 논문은 리전 용량을 넘지 않고 마감을 지키면서 탄소 배출을 줄이는 스케줄러 CAST를 제안하였다(위반 없음은 실험한 이용률 53.5%까지). CAST는 확정 계획을 유지하지 않고 매 슬롯 아직 시작하지 않은 작업 전부의 실행 리전과 시작 슬롯을 선형 계획으로 다시 정하며, 리전 용량의 초과에 가장 큰 벌점을 둔다. 여덟 리전의 1년치 탄소집약도 위에서 평균 배출계수로 센 배출량을 홈 리전 즉시 실행보다 합성 워크로드에서 무릎점 규칙으로 71.93%, 지연을 보지 않는 α=1에서 95.65%, Azure 워크로드에서 무릎점 규칙으로 51.35%(전송 에너지 제외) 줄였다. 모든 작업을 마감 안에 처리하면서 상한 위반은 없었고, 슬롯 안 도착을 미리 알지 않아도 그랬다. 위반이 없는 가장 가까운 두 설계와는 같은 지연 짝, 곧 지연이 같아지도록 고른 고정 α끼리 견주었고, 미룰 수 없는 작업을 위해 미래 슬롯에 비워 두는 예약은 두지 않았다. CAST는 다시 풀지 않는 도착 고정형보다 합성 0.96%포인트(워크로드를 다시 뽑은 네 벌은 고정 α=0.575의 짝으로 1.23~1.56), Azure 2.53%포인트 덜 배출했고, 시작 때 빈 리전으로 보내는 대피형보다 합성

0.91%포인트, Azure 1.99%포인트 덜 배출했다. 도착 고정형과의 차, 곧 다시 푸는 몫은 깨끗한 리전의 상한이 빠듯할수록 커서, 상한 13에서 1.46%포인트, 깨끗한 세 리전만 작게 둔 리전별 상한에서 2.10%포인트였고 상한 26에서는 차이가 없었다. 이 우위는 평균 배출계수 회계의 것이다. 한계 근사(프랑스만), 곧 프랑스로 옮긴 작업만 같은 시각 독일의 탄소집약도로 센 회계에서는 합성의 순서가 뒤집혔고, Azure에서는 도착 고정형보다 1.48%포인트 앞선 우위가 남았다(표 12). 무릎점 규칙의 CAST는 합성에서 같은 지연의 도착 고정형과 배출 차가 유의하지 않으면서 대기 시간이 25% 짧았고, Azure에서는 CAST의 대기 시간이 더 길었다. 채점 전 첫 주인 준비 구간에서 잰 규칙값대로 예약 13(계속 도는 VM을 두 번 센 값)을 두면 Azure에서 도착 고정형이 2.10%포인트 앞섰다. 다른 작업의 확정 계획을 보지 않고 미래 시작을 확정하되 시작 때 다시 확인하지 않는 시작한 작업만 확인형에서는 합성 채점 작업의 9.0~17.0%(준비 구간 규칙값 예약 2~3)가 상한 위반으로 시작했고, 시작 때 확인해 빈 리전으로 보내는 대피형은 위반이 없었다. 탄소만 볼 때 용량을 지키는 비용, 곧 상한을 지운 용량 무시와의 절감 차는 예약 없이 합성 0.34%포인트, 평균 부하가 가장 깨끗한 리전의 상한을 넘는 Azure에서 27.37%포인트였다. 한 주를 미리 다 알고 같은 지연 예산으로 한 번에 푼 오프라인 하한과 견주면, 계절이 다른 네 주(합성, 이용률 8.4%)에서 온라인 구조의 대가, 곧 하한과의 격차는 완전예지(실측을 그대로 예측으로 쓴 경우)·고정 α로 0.13~0.64%포인트로 작았다. LSTM에서도 CAST가 도착 고정형보다 하한에 더 가까웠다(예약 3). 기본 가중치 규칙인 무릎점 규칙은 슬롯마다 이상점에 가장 가까운 후보를 골라 사람이 가중치를 정하지 않아도 된다. 그러나 목적식과 같은 척도인 실행 시간 가중 지연으로 맞춘 같은 지연의 고정 α보다 합성 워크로드의 아홉 조건 가운데 여덟에서 1.77~2.64%포인트 더 배출했고, 첫 주 자료로 고른 고정 α보다도 약 0.9%포인트 더 배출했다.

본 연구의 주된 측정은 용량을 무엇과 함께 확인하느냐가 위반을 가른다는 데 있다. 미래 시작을 확정한 계획들이 서로를 보지 못하면 같은 예측 최저 시각으로 모이고, 시작 때 확인하지 않는 한 넘치며, 시작 때 확인해 빈 리전으로 보내면 넘치지 않는다. 몰림 자체는 COAST[11]와 수요반응 연구[12]도 지적했다. 확정 계획을 모두 보거나 매 슬롯 다시 풀면 두 워크로드와 세 예측기 모두에서 위반이 없다. CAST에서 미래 슬롯 행, 곧 앞으로 올 슬롯의 용량을 세는 제약은 위반을 막는 것이 아니라 붐빌 슬롯을 미리 피해 절감을 0.12~0.72%포인트 더한다(같은 지연의 짝은 아니다). 다만 공유 제약은 몰림을 지우지는 않는다. 몰린 작업은 저탄소 구간을 상한까지 채우는 데서 멈추고, 위반으로도 큰 손실로도 이어지지 않는다(4.5절).

평균 배출계수로 셀 때 상한이 걸리는 자리는 고르지 않다. 용량 행을 지운 대조군인 용량 무시에서는 상한을 넘긴 시간의 93.3%가 공간 이동이 모이는 캘리포니아와 프랑스에서 난다. 총합이 113~114로 거의 같은 세 가지 리전별 상한 배치에서도 이 두 리전의 비중이 가장 크다. 캘리포니아에서 상한을 넘긴 시간은 74.2%가 현지 9시부터 13시 전까지에 몰린다. 시간 이동의 절감은 하루 절대 변동, 곧 날마다 탄소집약도 최댓값과 최솟값의 차가 큰 다섯 리전이 92.8%를 낸다. 상한이 걸리는 리전과 시간 이동 절감이 큰 리전에 함께 드는 것은 캘리포니아와 일본이다(인도와 일본은 자료 산정 방식이 바뀐 것으로 보여 이 결과는 자료에 기댄다). 상한을 모든 리전에 같게 두고 배출을 절대량(kg)으로 세는 한 이 결과의 대부분은 정의에서 따라 나오며, 상한 배치를 바꿔도 캘리포니아와 프랑스의 비중이 가장 크다는 것이 측정이 보태는 몫이다.

이 결과가 실무에 주는 함의는 두 가지다. 첫째, 탄소 인지 스케줄링의 이득은 그 배정이 요구하는 용량과 함께 보고해야 비교할 수 있다. 용량을 두지 않고 1년치를 미리 안다고 가정해 절감의 상한을 구한 Sukprasert 공간+시간의 96.63%는 리전 하나가 동시에 129건을 돌려야 성립하는 숫자로, 가장 큰 기준 피크 13의 9.9배다. CASPER의 95.30%도 가장 큰 기준 피크의 2.2배인 29건을 요구한다. 다만 리전별 최대 동시 실행을 모두 더한 설비 총량으로는 α=1의 CAST와 CASPER가 46으로 같고 무릎점 규칙의 CAST(117)가 더 크다. 그래서 이 비교는 리전마다 용량이 정해져 있다는 가정 위의 것이다.

둘째, 평균 배출계수로 절감을 절대량(kg)으로 셀 때 시간 이동을 도입할지는 하루 안의 상대 변동, 곧 날마다 최댓값과 최솟값의 차를 그날 평균으로 나눈 값이 아니라 절대 변동, 곧 그 차 자체로 판단해야 한다. 이는 회계 정의에서 따라 나오지만 상대 변동으로 판단하면 놓칠 수 있다. 프랑스는 상대 변동이 0.97로 큰 편이지만 절대 변동이 11.8 gCO_2/kWh로 가장 작아 시간 이동 절감의 0.3%만 낸다. 어느 축이 절감을 내는지는 워크로드가 정한다. 본 연구의 두 워크로드에서 짧고 미룰 수 있는 작업이 많으면 시간 이동과 결합 설계가 절감을 더했고, 길고 미룰 수 없는 작업이 많으면 공간 이동과 용량이 결과의 거의 전부였다.

운영자가 할 일은 다섯으로 정리된다. (a) 위반을 없애는 데는 시작 때 확인해 빈 리전으로 보내거나 확정 계획을 모두 보는 것으로 충분하다(4.4절). (b) 매 슬롯 다시 풀면 같은 지연에서 1~2.5%포인트를 더 얻지만 시작

시각을 미리 알려 줄 수 없고 계산이 4~11배 든다(4.4절, 4.9절). (c) 1주 자료가 있으면 무릎점보다 첫 주로 고른 고정 가중치가 약 0.9%포인트 낫다(4.5절). (d) 시험한 부하에서는 예약이 필요 없었다(4.7절). (e) 옮긴 작업의 데이터가 작업당 약 9 GB를 넘으면 리전 이동의 이득이 사라진다(작업당 1 kW 기준, Azure VM의 실제 전력으로는 VM당 약 1 GB, 4.3절).

한계는 다섯 가지다. 첫째, 탄소 회계가 Electricity Maps의 평균 배출계수 위에 있다. 받은 파일에 배출계수 기준(직접 배출인지 전과정인지), 추정값 여부, API 판, 받은 날짜가 남아 있지 않아 다시 받아 대조하지 못했으며, 프랑스 최솟값 2 gCO_2/kWh는 직접 배출 기준과 맞는다. 무탄소 전원에만 전과정 배출을 더하고 화석 상류는 뺀 전과정 근사로 다시 채점하면 CAST의 절감이 3.2~5.4%포인트 줄지만 순서는 같다(4.1절). 작업을 옮겨 실제로 바뀌는 한계 배출은 넣지 않았다. 프랑스로 보낸 밤 작업은 프랑스의 수출을 줄여 이웃 망의 화석 발전을 늘릴 수 있으므로, 공간 이동의 절감은 한계 회계에서 작아질 수 있다. 한계 근사(프랑스만)에서는 무릎점 규칙의 절감이 38.99%로 줄고, α=1은 홈 리전에서 시각만 옮기는 시간만과 비슷해지며, 도착 고정형과의 순서가 뒤집힌다(0.89%포인트, 표 12). 대칭 한계 근사에서는 같은 지연의 비교 순서가 남지만, 프랑스를 가스 발전 수준(약 400 gCO_2/kWh)으로 두면 무릎점 절감은 21.20%(Azure 17.93%)로 준다(4.1절). 불리한 가정 세 가지, 곧 프랑스만 한계 근사, 출발지 기준 전송 1 GB, 불리한 설비 효율(PUE)을 함께 두면 같은 지연 짝(고정 α, 예약 없이)에서 CAST 절감은 17.26%로 줄고, 도착 고정형(18.75%)과 대피형(20.39%)보다 각각 1.49%포인트와 3.13%포인트 더 배출한다. Azure에서는 세 가정을 겹쳐도 도착 고정형보다 0.89%포인트 앞서고 대피형과는 차이가 없다(표 12). 설비 효율(PUE)은 리전별 1.1~1.6의 민감도로만 확인했고(4.1절) 그 시각별 변동과 유휴 전력은 넣지 않아, 절감률의 분모는 작업이 실행 중에 쓰는 전력의 배출이다. 추가 서버의 내재 배출도 세지 않았다. 리전마다 17자리가 이미 있다면 선행 정책이 더 요구하는 용량이 그만큼 배출을 더하지만, 필요한 만큼만 짓는다면 설비 총량이 큰 CAST 무릎점(117)이 불리하다. 미국 동부 리전은 지연을 Azure East US(버지니아)에서, 탄소를 뉴욕 전력망에서 가져온 혼합이고, 일본과 인도 자료는 데이터센터가 있는 하위 계통과 다를 수 있으며, 인도는 2024년 11월부터 하루 변동이 거의 두 배가 되어 학습 연도와 모양이 다르다. 리전 간 전송 에너지는 옮긴 작업마다 1 GB를 더해 보았다. 그 전송 배출을 목적지 탄소집약도로 세면 절감이 1%포인트 안에서, 출발지 탄소집약도로 세면 무릎점 규칙에서 5.92%포인트, α=1에서 8.00%포인트까지 줄었고 순서는 그대로였다. 그러나 작업당 약 9 GB(옮긴 작업의 계산 1 kWh당 합성 12.9 GB, Azure 13.8 GB)를 넘으면 무릎점 규칙의 CAST가 시간만보다 덜 줄인다(4.3절). Azure에서 VM의 실제 전력(대부분 2~4코어)으로 두면 손익분기가 VM당 1 GB 안팎이라, VM을 옮기는 이득의 부호가 디스크 이미지 크기에 따라 바뀔 수 있다(4.3절). 둘째, 상한은 1년 내내 같은 전용 용량으로 두어 시각마다 바뀌는 공용 여유와 다른 테넌트의 부하는 넣지 않았다. 실제 배포가 아닌 시뮬레이션이어서 클러스터 관리 계층과의 연동 비용을 확인하지 못했다. 또 주 실험의 숫자는 미래 정보 셋, 곧 슬롯 안 도착, 지금 슬롯의 탄소, 세 리전의 미래 기상 실측을 미리 안다고 두고 얻은 것이다. 셋을 모두 뺀 현실 정보 설정에서 무릎점 절감은 68.88%(예약 없는 72.06%에서 −3.18%포인트)이고 설계 비교의 순서는 같다(4.4절). 주 실험은 슬롯 안에 도착하는 작업을 슬롯 시작에 안다고 두어 최대 1시간을 미리 보며, 이를 없애면 위반은 그대로 0이고 절감은 0.56~1.62%포인트 준다. 탄소 쪽도 지금 슬롯의 첫 한 시간에 사후 확정값을 쓰고 발행 시각의 값을 예측 입력으로 써 한 시간을 미리 보며, 이를 없애도 위반은 0이고 LSTM 무릎점은 2.42%포인트, α=1은 0.17%포인트 낮아진다(4.2절). 셋째, 미룸 등급, 곧 작업이 시작을 얼마나 미룰 수 있는지 나눈 단계와 합성 워크로드의 도착 시각은 모형값이다(도착 봉우리를 한낮으로 옮겨도 결론은 같다, 4.7절). 또 지연 항은 등급과 무관하게 실행 시간에 비례하는데, 이 형태는 설계 선택이라 등급별로 다른 비용을 두면 결과가 달라질 수 있다. Azure 워크로드도 도착과 실행 시간만 실측이고, 홈 리전은 균등하게 주었다. 또 VM 크기를 버렸으므로 Azure의 숫자는 VM 시간으로 가중한 값이다(코어 수에 비례한 전력으로 다시 채점해도 절감은 많아야 1.1%포인트 움직였으나, 용량은 여전히 VM 한 대를 한 자리로 센다). VM 하나가 생성되어 삭제될 때까지의 수명을 미룰 수 있는 배치 작업 하나로 본 것과 실행 시간 d_j를 제출 때 안다고 둔 것도 가정이다. 실제 실행 시간이 제출 때 안다고 둔 값보다 길면 그 몫만큼 다른 작업의 자리를 침범할 수 있으나 이 오차의 영향은 재지 않았다. 실제 VM 수명은 미리 알 수 없으며 [34]는 그것을 예측하는 연구다. 주 실험은 왕복 지연 상한을 두지 않아 모든 작업에 공간 이동을 열어 두므로, 지연 상한을 100 ms로 두면 무릎점 절감이 32.98%로 꺾인다(4.7절).

넷째, 캘리포니아, 텍사스, 독일의 기상 입력이 예보가 아닌 실측값이다. 이 세 리전의 예측을 기상 입력이 없는 24시간 전 값으로 바꾸어도 절감은 무릎점에서 0.37%포인트, α=1에서 0.01%포인트 움직였다(4.2절).

다만 예측기 전체를 바꾸면 무릎점 절감이 64.44~71.93%로 움직여 무릎점 결과는 예측기에 매여 있으며, 그 차이의 대부분은 무릎점 규칙이 예측 곡선의 모양에 따라 다른 α를 고르는 데서 오고, 작업당 지연으로 맞춘 고정 α 하나는 72.6~74.0%로 덜 흔들린다(4.5절). 다섯째, 탄소 자료가 2025년 한 해이고 예측 지평이 24시간이어서, 하루를 넘는 작업의 리전도 앞 24시간 예측으로 정하며, 평균 창이 지평 끝에서 잘려 Azure의 긴 작업은 지평 안의 평균으로 판단된다. 계산 비용은 하루 약 400건 도착 규모(도착 두 배까지)에서 쟀다. 같은 α에서 CAST와 리전을 도착 때 고정하는 분리형을 견준 비교는 주 실험의 시드 한 벌에서만 했다. 오프라인 하한과의 격차는 네 주와 같은 시작 후보 격자, 상한 17에서만 쟀고, 24시간 지평의 대가는 재지 못했다. 같은 지연 짝의 맞춤 오차(Azure 짝의 지연 차 0.20~0.58 ms만큼)는 부트스트랩 구간에 들어 있지 않다. 최악의 경우에 대한 경쟁비 보장은 없고, 벌점 크기가 엄밀한 사전식 순서의 충분조건보다 작아 초과가 최소임도 보장하지 않는다(확정 시점의 분류는 [27]에서 빌렸을 뿐이다). 위반 0은 지금 시작만 확정하고 지금 슬롯을 순간 행으로 세는 구성과, 리전을 모두 합친 용량의 여유에서 따라 나온다(4.4절). 이용률 53.5%(Azure 상한 0.6배)까지는 위반이 0이었으나, 66.1%(0.5배)에서는 모든 리전이 찬 순간에 작업의 0.32%가 넘쳐 식 (11)의 초과 변수가 실제로 양수가 되었다. 이 위반을 사후에 피할 수 있었는지는 확정하지 못했다. Azure의 예약은 준비 구간 규칙 값이 이중 계산이어서 버리고 예약 없이 비교했다(3.3절). 선행 정책은 원문 수식을 따라 재구현하되 바꾼 점이 있고(4.8절) 합성 워크로드에서만 돌렸으며, 원 논문의 자료가 아니어서 원 수치의 재현은 확인하지 못했다.

향후 과제는 실제 예보 API 연동, 한계 배출계수를 쓰는 회계, 24시간을 넘는 예측 지평[38], 그리고 리전 상한을 수요와 설비에서 추정하는 방법이다. 탄소 비용을 슬롯마다 다른 분모로 정규화한 탓에 슬롯별 무릎점은 슬롯마다 다른 탄소(kg)와 지연(ms)의 교환비를 쓰므로, 교환비를 하나로 맞추는 방법(목표 지연 예산에 맞춘 라그랑주 승수의 갱신)도 향후 연구로 남긴다. 리전 상한을 추정하는 과제는 Zanotto 등이 남긴 과제이기도 하며[9], 본 연구가 상한을 실험 변수로 두고 바꿔 가며 본 결과가 그 출발점이 될 수 있다. 용량을 지키면서 얻은 절감이 실제 클러스터에서도 유지되는지 검증할 계획이다.

참고문헌

[1] International Energy Agency, Energy and AI, World Energy Outlook Special Report, IEA, Paris, 2025.

[2] European Parliament and Council, Directive (EU) 2023/1791 of the European Parliament and of the Council of 13 September 2023 on Energy Efficiency (recast), Official Journal of the European Union, 2023.

[3] A. Radovanović, R. Koningstein, I. Schneider, B. Chen, A. Duarte, B. Roy, D. Xiao, M. Haridasan, P. Hung, N. Care, S. Talukdar, E. Mullen, K. Smith, M. Cottman, and W. Cirne, “Carbon-Aware Computing for Datacenters,” IEEE Transactions on Power Systems, Vol. 38, No. 2, pp. 1270-1280, Mar. 2023.

[4] A. Souza, S. Jasoria, B. Chakrabarty, A. Bridgwater, A. Lundberg, F. Skogh, A. Ali-Eldin, D. Irwin, and P. Shenoy, “CASPER: Carbon-Aware Scheduling and Provisioning for Distributed Web Services,” Proc. of the 14th Int. Green and Sustainable Computing Conf. (IGSC), pp. 67-73, Oct. 2023.

[5] D. Maji, B. Pfaff, V. P R, R. Sreenivasan, V. Firoiu, S. Iyer, C. Josephson, Z. Pan, and R. K. Sitaraman, “Bringing Carbon Awareness to Multi-Cloud Application Delivery,” Proc. of the 2nd Workshop on Sustainable Computer Systems (HotCarbon), pp. 1-6, Jul. 2023.

[6] T. Sukprasert, A. Souza, N. Bashir, D. Irwin, and P. Shenoy, “On the Limitations of Carbon-Aware Temporal and Spatial Workload Shifting in the Cloud,” Proc. of the 19th European Conf. on Computer Systems (EuroSys), pp. 924-941, Apr. 2024.

[7] W. A. Hanafy, L. Wu, D. Irwin, and P. Shenoy, “Flex: Grid-Responsive Provisioning and Scheduling for Elastic Cloud Clusters,” arXiv:2505.18357v2, Oct. 2026 (to appear in ACM SoCC 2026).

[8] T. Bahreini, A. N. Tantawi, and O. Tardieu, “Caspian: A Carbon-aware Workload Scheduler in Multi-Cluster Kubernetes Environments,” Proc. of the 32nd IEEE Int. Symp. on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS), pp. 1-8, Oct. 2024.

[9] M. Zanotto, L. Vicentini, R. Vreto, F. Lumpp, D. Braga, and S. Fiore, “Toward a User-centric Kubernetes-based Architecture for Green Cloud Computing,” Future Generation Computer Systems, Vol. 186, 108749, Jan. 2027 (in press).

[10] Green Software Foundation, Carbon Aware SDK, v1.8.0 [Online]. Available: https://github.com/Green-Software-Foundation/carbon-aware-sdk (downloaded 2026, Oct. 18)

[11] W. Feng, A. Dasgupta, Z. D. Tekler, S. Ahuja, J. Zheng, and X. Jiang, “COAST: Congestion-Aware Start-Time Recommendations for Carbon-Aware HPC Jobs,” arXiv:2609.05443, 2026.

[12] C. Han, D. Witthaut, L. Rydin Gorjão, and P. C. Böttcher, “Collective Effects and Synchronization of Demand in Real-Time Demand Response,” Journal of Physics: Complexity, Vol. 3, No. 2, 025002, 2022.

[13] S. P. Sethi and G. Sorger, “A Theory of Rolling Horizon Decision Making,” Annals of Operations Research, Vol. 29, No. 1, pp. 387-415, Dec. 1991.

[14] D. Q. Mayne, J. B. Rawlings, C. V. Rao, and P. O. M. Scokaert, “Constrained Model Predictive Control: Stability and Optimality,” Automatica, Vol. 36, No. 6, pp. 789-814, Jun. 2000.

[15] S. Chand, V. N. Hsu, and S. Sethi, “Forecast, Solution, and Rolling Horizons in Operations Management Problems: A Classified Bibliography,” Manufacturing & Service Operations Management, Vol. 4, No. 1, pp. 25-43, Jan. 2002.

[16] P. Wiesner, I. Behnke, D. Scheinert, K. Gontarska, and L. Thamsen, “Let's Wait Awhile: How Temporal Workload Shifting Can Reduce Carbon Emissions in the Cloud,” Proc. of the 22nd Int. Middleware Conf. (Middleware), pp. 260-272, Dec. 2021.

[17] W. A. Hanafy, Q. Liang, N. Bashir, D. Irwin, and P. Shenoy, “CarbonScaler: Leveraging Cloud Workload Elasticity for Optimizing Carbon-Efficiency,” Proc. of the ACM on Measurement and Analysis of Computing Systems (POMACS), Vol. 7, No. 3, Article 57, Dec. 2023.

[18] W. A. Hanafy, Q. Liang, N. Bashir, A. Souza, D. Irwin, and P. Shenoy, “Going Green for Less Green: Optimizing the Cost of Reducing Cloud Carbon Emissions,” Proc. of the 29th ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS), Vol. 3, pp. 479-496, Apr. 2024.

[19] K. Xu, D. Sun, H. Tian, J. Zhang, and K. Chen, “GREEN: Carbon-efficient Resource Scheduling for Machine Learning Clusters,” Proc. of the 22nd USENIX Symp. on Networked Systems Design and Implementation (NSDI), pp. 999-1014, Apr. 2025.

[20] G. Attenni, Y. Moawad, N. Bartolini, and L. Thamsen, “Spatio-Temporal Shifting to Reduce Carbon, Water, and Land-Use Footprints of Cloud Workloads,” arXiv:2512.08725, 2025.

[21] A. Lechowicz, N. Christianson, B. Sun, N. Bashir, M. Hajiesmaili, A. Wierman, and P. Shenoy, “Learning-Augmented Competitive Algorithms for Spatiotemporal Online Allocation with Deadline Constraints,” Proc. of the ACM on Measurement and Analysis of Computing Systems (POMACS), Vol. 9, No. 1, Article 8, pp. 1-49, Mar. 2025.

[22] S. Hall, F. Micheli, G. Belgioioso, A. Radovanović, and F. Dörfler, “Carbon-Aware Computing for Data Centers with Probabilistic Performance Guarantees,” IEEE Transactions on Power Systems, Early Access, pp. 1-14, Nov. 2025, doi: 10.1109/TPWRS.2025.3630499.

[23] Z. Liu, M. Lin, A. Wierman, S. H. Low, and L. L. H. Andrew, “Greening Geographical Load Balancing,” Proc. of the ACM SIGMETRICS Joint Int. Conf. on Measurement and Modeling of Computer Systems (SIGMETRICS), pp. 233-244, Jun. 2011.

[24] M. Lin, Z. Liu, A. Wierman, and L. L. H. Andrew, “Online Algorithms for Geographical Load Balancing,” Proc. of the 2012 Int. Green Computing Conf. (IGCC), pp. 1-10, Jun. 2012.

[25] S. Ren, Y. He, and F. Xu, “Provably-Efficient Job Scheduling for Energy and Fairness in Geographically Distributed Data Centers,” Proc. of the 32nd IEEE Int. Conf. on Distributed Computing Systems (ICDCS), pp. 22-31, Jun. 2012.

[26] Í. Goiri, K. Le, M. E. Haque, R. Beauchea, T. D. Nguyen, J. Guitart, J. Torres, and R. Bianchini, “GreenSlot: Scheduling Energy Consumption in Green Datacenters,” Proc. of the Int. Conf. for High Performance Computing, Networking, Storage and Analysis (SC), pp. 1-11, Nov. 2011.

[27] L. Chen, F. Eberle, N. Megow, K. Schewior, and C. Stein, “A General Framework for Handling Commitment in Online Throughput Maximization,” Mathematical Programming, Vol. 183, No. 1-2, pp. 215-247, Sep. 2020.

[28] D. Maji, R. K. Sitaraman, and P. Shenoy, “DACF: Day-Ahead Carbon Intensity Forecasting of Power Grids using Machine Learning,” Proc. of the 13th ACM Int. Conf. on Future Energy Systems (e-Energy), pp. 188-192, Jun.-Jul. 2022.

[29] J. Branke, K. Deb, H. Dierolf, and M. Osswald, “Finding Knees in Multi-objective Optimization,” Proc. of the 8th Int. Conf. on Parallel Problem Solving from Nature (PPSN VIII), LNCS 3242, pp. 722-731, Sep. 2004.

[30] M. Zeleny, “Compromise Programming,” in J. L. Cochrane and M. Zeleny (Eds.), Multiple Criteria Decision Making, University of South Carolina Press, Columbia, pp. 262-301, 1973.

[31] Electricity Maps. API Documentation [Online]. Available: https://app.electricitymaps.com/docs (downloaded 2026, Oct. 18)

[32] Electricity Maps. Methodology [Online]. Available: https://www.electricitymaps.com/data/methodology (downloaded 2026, Sep. 22)

[33] Microsoft. Azure Network Round-Trip Latency Statistics [Online]. Available: https://learn.microsoft.com/en-us/azure/networking/azure-network-latency (downloaded 2026, Sep. 22)

[34] E. Cortez, A. Bonde, A. Muzio, M. Russinovich, M. Fontoura, and R. Bianchini, “Resource Central: Understanding and Predicting Workloads for Improved Resource Management in Large Cloud Platforms,” Proc. of the 26th ACM Symp. on Operating Systems Principles (SOSP), pp. 153-167, Oct. 2017.

[35] Microsoft Azure, Azure Public Dataset V2, 2019 [Online]. Available: https://github.com/Azure/AzurePublicDataset (downloaded 2026, Oct. 17)

[36] C. Reiss, A. Tumanov, G. R. Ganger, R. H. Katz, and M. A. Kozuch, “Heterogeneity and Dynamicity of Clouds at Scale: Google Trace Analysis,” Proc. of the 3rd ACM Symp. on Cloud Computing (SoCC), Article 7, Oct. 2012.

[37] J. Aslan, K. Mayers, J. G. Koomey, and C. France, “Electricity Intensity of Internet Data Transmission: Untangling the Estimates,” Journal of Industrial Ecology, Vol. 22, No. 4, pp. 785-798, 2018.

[38] D. Maji, P. Shenoy, and R. K. Sitaraman, “CarbonCast: Multi-Day Forecasting of Grid Carbon Intensity,” Proc. of the 9th ACM Int. Conf. on Systems for Energy-Efficient Buildings, Cities, and Transportation (BuildSys), pp. 198-207, Nov. 2022.